

รายงานความก้าวหน้า 240-401 โครงงานวิศวกรรมคอมพิวเตอร์ 1 ครั้งที่ 1/2569

Srisurart Autopart POS: An Offline-First Multi-Tenant Point-of-Sale System

ระบบขายหน้าร้านอะไหล่รถยนต์แบบหลายร้านค้าที่ทำงานออฟไลน์ได้

ผู้ดำเนินโครงงาน

.....

Chavatik Thorarit รหัสนักศึกษา 6610110066

.....

Pattarapon Kitcharoen รหัสนักศึกษา 66101xxxxx

.....

ชื่อ-สกุล (LomerAlloys) รหัสนักศึกษา 66101xxxxx

อาจารย์ที่ปรึกษาโครงงาน

.....

(ตำแหน่งวิชาการ ชื่อ-สกุล อาจารย์ที่ปรึกษา)

รายงานความก้าวหน้าโครงงานวิศวกรรมคอมพิวเตอร์นี้เป็นส่วนหนึ่งของการศึกษา
ตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยสงขลานครินทร์

4 ตุลาคม 2569

สารบัญ

สารบัญ ..... **ก**
รายการรูปภาพ ..... **ฉ**
รายการตาราง ..... **ช**
รายการคำย่อ ..... **ฌ**
บทที่ 1 บทนำ **1**
1.1 ที่มาและความสำคัญ 1
1.1.1 ร้านอะไหล่รถยนต์และปัญหาของการทำงานด้วยมือ 1
1.1.2 ระบบเดิมและข้อจำกัด 2
1.1.3 การย้ายมาใช้ Flutter แบบ offline-first 3
1.1.4 เหตุผลที่ต้องมีระบบเซิร์ฟเวอร์แบบหลายร้านค้า 3
1.2 วัตถุประสงค์ของโครงงาน 5
1.3 ขอบเขตของโครงงาน 6
1.3.1 ขอบเขตที่อยู่ในโครงงาน 6
1.3.2 ขอบเขตที่อยู่นอกโครงงาน 7
1.3.3 สิ่งที่รายงานนี้ยังไม่ถือว่าแล้วเสร็จ 8
1.4 แผนการดำเนินงาน 8
1.4.1 ลำดับเหตุการณ์ที่เกิดขึ้นจริง 8
1.4.2 ตารางแผนงานแบบแกนต์ 9
1.4.3 โครงสร้างทีมและความรับผิดชอบ 9
1.5 ประโยชน์ที่คาดว่าจะได้รับ 11
1.6 ลำดับการเรียบเรียงรายงาน 11
บทที่ 2 ทฤษฎีและความรู้พื้นฐาน **12**
2.1 บทนำ 12
2.2 แนวคิดของระบบขายหน้าร้านและโดเมนอะไหล่รถยนต์ 12
2.2.1 การขายและค่าคงที่ของสต็อก 12
2.2.2 การคืนสินค้าและใบลดหนี้ 12
2.2.3 การรับสินค้าเข้าและต้นทุนถัวเฉลี่ยถ่วงน้ำหนัก 13
2.2.4 กะ ลิ้นชักเงินสด และเครดิตช่าง 13
2.2.5 เลขที่เอกสาร 13
2.2.6 เหตุที่ไม่ใช้ระบบ POS สำเร็จรูป 13

2.3 สถาปัตยกรรมออฟไลน์เป็นหลัก (offline-first) ... 14
2.3.1 แนวคิด local-first และ offline-first ... 14
2.3.2 เหตุที่ไม่ใช้ CRDT หรือการทำสำเนา (replicate) ฐานข้อมูลโดยตรง ... 14
2.3.3 รูปแบบกล่องขาออก (outbox pattern) ... 14
2.3.4 ความสอดคล้องในที่สุด ... 15
2.3.5 ความล้มเหลวระหว่างทางและผลลัพธ์ที่ไม่ทราบแน่ชัด ... 15
2.4 เทคโนโลยีฝั่งไคลเอนต์ ... 15
2.4.1 Flutter และ Dart ... 15
2.4.2 SQLite, Drift และการรันบนเว็บ ... 15
2.4.3 การจัดการสถานะด้วย BLoC และการนำทางด้วย go_router ... 16
2.5 การสื่อสารไคลเอนต์-เซิร์ฟเวอร์ ... 16
2.6 ธุรกรรมฐานข้อมูลและการควบคุมภาวะพร้อมกัน ... 16
2.6.1 คุณสมบัติ ACID และระดับการแยกจากกัน ... 16
2.6.2 การล็อกระดับแถว เดดล็อก และลำดับการล็อก ... 17
2.6.3 ธุรกรรมกับกลุ่มการเชื่อมต่อ ... 17
2.7 การแยกข้อมูลผู้เช่าหลายราย (multi-tenancy) ... 17
2.8 การทำซ้ำคำขออย่างปลอดภัยและรหัสกำกับการเขียน (idempotency) ... 20
2.9 การยืนยันตัวตนและการรักษาความปลอดภัย ... 20
2.9.1 JSON Web Token และอายุของโทเค็น ... 20
2.9.2 การแฮชรหัสผ่านด้วย Argon2 ... 20
2.9.3 การผูกอุปกรณ์และการจำกัดอัตราคำขอ ... 20
2.10 เทคโนโลยีฝั่งเซิร์ฟเวอร์ ... 21
2.11 คอนเทนเนอร์และการจัดวาง ... 21
2.12 การบูรณาการและการส่งมอบอย่างต่อเนื่อง (CI/CD) ... 21
2.13 การสังเกตการณ์ระบบ (observability) ... 22
2.14 การบันทึกการตัดสินใจเชิงสถาปัตยกรรม (ADR) ... 22
2.15 สรุป ... 23
บทที่ 3 รายละเอียดการดำเนินงาน ... 24
3.1 ภาพรวมของโครงงาน ... 24
3.1.1 หลักการของสถาปัตยกรรม ... 24
3.1.2 สถาปัตยกรรมโดยรวม ... 24
3.2 ส่วนประกอบของโครงงาน ... 26

3.3 รายละเอียดฝั่งไคลเอนต์ 29
3.3.1 โครงสร้างแบบสามชั้น 29
3.3.2 ฐานข้อมูลภายในเครื่อง (Drift) 30
3.3.3 Repository ที่พอร์ตมาจาก db.js และกติกาที่คงไว้ 31
3.3.4 Repository แบบ API (USE_API_WRITES) 33
3.3.5 หน้าจอ 33
3.3.6 การแยกข้อมูลแคชตามผู้เช่า 34
3.4 รายละเอียดฝั่งเซิร์ฟเวอร์ 34
3.4.1 โมดูลของ NestJS 34
3.4.2 การแยกข้อมูลผู้เช่าด้วย RLS และ TenantService.runTx 35
3.4.3 Idempotency และลำดับการล็อก 37
3.4.4 ฐานข้อมูล PostgreSQL 38
3.4.5 จุดปลายทาง API หลัก 41
3.4.6 การยืนยันตัวตนและการผูกเครื่อง 43
3.4.7 ระนาบผู้ดูแลแพลตฟอร์ม 43
3.4.8 การจำกัดอัตรา งานเบื้องหลัง และ etcd 44
3.4.9 การวัดผลและสังเกตการณ์ 44
3.5 การซิงก์ข้อมูลเฟส 2 45
3.5.1 ภาพรวมของการทำงานออฟไลน์ 45
3.5.2 Outbox ในเครื่อง 45
3.5.3 POST /sync/push 47
3.5.4 เลข RC และ CN ที่ออกบนเครื่อง 47
3.5.5 การดึงข้อมูลแบบ keyset 48
3.6 ความปลอดภัย 49
3.7 การนำระบบขึ้นใช้งาน (deploy) 50
3.7.1 โครงสร้างการ deploy และระดับของ CI/CD 50
3.7.2 Ansible และ self-hosted runner 52
3.7.3 TLS ด้วย CA ส่วนตัวและการสร้างไฟล์ติดตั้ง Android 52
3.8 เทคโนโลยีที่ใช้ 53
3.9 วิธีการทำงานของทีม 55
3.9.1 การแบ่งงานและการติดตามงาน 55
3.9.2 ADR และลำดับความสำคัญของเอกสาร 55

3.9.3 ขั้นตอนการพัฒนาและการตรวจ 55
3.9.4 เครื่องมือช่วยพัฒนาด้วยปัญญาประดิษฐ์ 56
บทที่ 4 ความก้าวหน้าการดำเนินงาน 57
4.1 ผลการย้ายระบบเดิมสู่ Flutter 57
4.1.1 ลำดับความก้าวหน้าของการย้ายระบบ ขั้น 0 ถึง ขั้น 6 57
4.1.2 การนำเข้าข้อมูลสำรองและฐานข้อมูลบนเว็บ 58
4.2 หน้าจอของระบบ 59
4.2.1 เมนูนำทางและหน้าขายสินค้า 59
4.2.2 หน้าสินค้า ช่าง และใบเสนอราคา 62
4.2.3 หน้าคืนสินค้า ลิ้นชัก และรายงานปิดร้าน 64
4.2.4 หน้าจัดการเครื่อง การเข้าสู่ระบบ และสถานะการเชื่อมต่อ 67
4.2.5 การทดสอบใช้งานบนเครื่องสาธิตและการแก้ไข (3–4 ตุลาคม พ.ศ. 2569) 69
4.2.6 ความเชื่อมโยงกับปัญหาของร้าน 73
4.3 ผลการพัฒนาชั้นเขียนข้อมูลผ่าน API 73
4.4 ผลการพัฒนาระบบทำงานออฟไลน์ เฟส 2 ฝั่งอุปกรณ์ 75
4.4.1 การแบ่งงานและตั๋วที่ปิดแล้ว 75
4.4.2 ผลเพิ่มเติมที่เกิดจากการแก้ข้อบกพร่อง 77
4.4.3 งานที่ยังไม่เสร็จของฝั่งอุปกรณ์ 78
4.5 ผลการทดสอบฝั่งไคลเอนต์ 78
4.6 ผลการพัฒนาฝั่งเซิร์ฟเวอร์ 78
4.6.1 ภาพรวมของฝั่งเซิร์ฟเวอร์ เฟส 1 78
4.6.2 การแยกข้อมูลรายร้าน 79
4.6.3 จุดเชื่อมธุรกรรม (transaction seam) และ idempotency 80
4.6.4 ระนาบผู้ดูแลแพลตฟอร์ม 80
4.6.5 เซิร์ฟเวอร์ของเฟส 2 81
4.7 ผลการพัฒนา CI/CD 81
4.7.1 ระดับที่ 1 ถึง 3 81
4.7.2 ระดับที่ 4: การ deploy ด้วย Ansible และ self-hosted runner 82
4.7.3 เวิร์กโฟลว์ Android APK และ TLS แบบ CA ส่วนตัว (3 ตุลาคม พ.ศ. 2569) 83
4.7.4 การ deploy และการทดสอบบนเครื่องสาธิต (3–4 ตุลาคม พ.ศ. 2569) 85
4.7.5 การติดตามผลด้วย Grafana 86
4.7.6 การขยายการเฝ้าสังเกตระบบ (1–4 ตุลาคม พ.ศ. 2569) 86

4.8 ผลการนำระบบขึ้นใช้งานบนเครื่อง mob04 และการติดตามระบบ 89
4.8.1 สแตกบนเครื่องสาธิต 89
4.8.2 การสำรองข้อมูลและ etcd 89
4.9 ผลการทดสอบฝั่งเซิร์ฟเวอร์ 90
4.10 ตรวจสอบเกณฑ์ตรวจรับ (DoD) 90
4.11 สรุปผลงานที่มีความก้าวหน้า 93
บทที่ 5 ผลการดำเนินงาน 97
5.1 สรุปผลการดำเนินงาน 97
5.2 สถานะล่าสุด 2026-10-07 100
5.3 ปัญหาและอุปสรรค 107
5.3.1 พาธที่มีอักษรไทยทำให้ build_runner ล้มเหลว 107
5.3.2 ไฟร์วอลล์ของคณะปิดกั้นการดึงอิมเมจจาก ghcr.io 107
5.3.3 pool ฐานข้อมูลหมดเมื่อมีคำขอพร้อมกัน (#162) 108
5.3.4 การทิ้งรายการในคิวย้อนผลไม่ตรง (#473, #488) 108
5.3.5 เลขที่เอกสารซ้ำระหว่างการออกเลขออนไลน์กับออฟไลน์ (#489) 108
5.3.6 การควบคุมการรวมโค้ดและหลักฐาน 108
5.3.7 การทดสอบที่ผูกกับเดือนปัจจุบัน (#525) 109
5.3.8 ค่า 0 เป็นเท็จในนิพจน์ของ GitHub Actions 109
5.4 งานที่จะดำเนินการต่อไป 110
บรรณานุกรม 113

รายการรูปภาพ

รูปที่ 1.1 วิวัฒนาการของระบบ POS ร้านศรีสุรัตน์จากรุ่นเว็บ localStorage สู่ระบบหลายร้านค้า 5
รูปที่ 2.1 แนวคิด Row-Level Security: คำขอของผู้เช่าหนึ่งเห็นเฉพาะแถวของผู้เช่านั้น 19
รูปที่ 3.1 สถาปัตยกรรมโดยรวมของระบบ Srisurart Autopart POS 25
รูปที่ 3.2 โครงสร้างชั้นของไคลเอนต์ Flutter และเส้นทางไปยังเซิร์ฟเวอร์ 30
รูปที่ 3.3 ลำดับการประมวลผลคำขอเขียนหนึ่งคำขอผ่านชั้นการแยกผู้เช่าและ idempotency 37
รูปที่ 3.4 แผนภาพ ER ของตารางแกนกลางในเซิร์ฟเวอร์ และตาราง outbox_ops ที่อยู่ในเครื่อง 39
รูปที่ 3.5 การไหลของรูปแบบกล่องขาออกในโครงงาน ตั้งแต่การขายออฟไลน์จนถึงการเล่นซ้ำที่เซิร์ฟเวอร์ 46
รูปที่ 3.6 ภาพรวมไปป์ไลน์ CI/CD ของโครงงาน ตั้งแต่คำขอรวมถึงการติดตั้งบนเครื่องสาธิต 51
รูปที่ 4.1 เมนูนำทางบนโทรศัพท์ เรียงครบ 12 หน้า 60
รูปที่ 4.2 หน้าขายสินค้า แสดงเมนู รายการสินค้า และตะกร้า 61
รูปที่ 4.3 การเลือกช่าง ราคาช่าง และการชำระด้วยเครดิตช่าง 61
รูปที่ 4.4 ใบเสร็จเงินสด แสดงเงินทอนและแต้มสะสมของลูกค้า 62
รูปที่ 4.5 หน้าสินค้า/สต็อก แท็บสต็อก 63
รูปที่ 4.6 หน้าช่าง แสดงยอดค้างและวงเงินเครดิตของช่างแต่ละราย 63
รูปที่ 4.7 ใบเสนอราคา A4 ส่วนหัวและรายการสินค้า (ถ่ายก่อนแก้ #479) 64
รูปที่ 4.8 หน้าคืนสินค้า 65
รูปที่ 4.9 แท็บรายการเงินและการ์ดสรุปเงินในลิ้นชัก 66
รูปที่ 4.10 รายงานปิดร้าน (Daily Closing Report, ถ่ายก่อนแก้ #461) 66
รูปที่ 4.11 หน้าจัดการเครื่อง แยกเครื่องขาย (POS Terminal) และเครื่องจัดการทั่วไป (Backoffice) 67
รูปที่ 4.12 ฟอร์มเข้าสู่ระบบด้วย PIN ออฟไลน์บนเครื่องขาย 68
รูปที่ 4.13 สถานะออฟไลน์ ป้ายขายสำรองและแถบเตือนสีเหลือง 69
รูปที่ 4.14 แผงควบคุม Grafana ระหว่างการซ้อมสาธิต (บันทึกเมื่อ 21 กันยายน พ.ศ. 2569) 86
รูปที่ 4.15 สถานะแผงควบคุม 44 แผงจากการทดสอบบนเครื่องผู้พัฒนา (2 ตุลาคม พ.ศ. 2569; 15 แผงของหน้าที่สองมีเฉพาะบนเครื่องผู้พัฒนา) 88

รายการตาราง

ตารางที่ 1.1 ปัญหาของร้านก่อนมีระบบและผลที่ตามมา 2
ตารางที่ 1.2 ขอบเขตของโครงงานจำแนกตามเฟส 7
ตารางที่ 1.3 แผนการดำเนินงานของโครงงาน (■ = ดำเนินการแล้ว, □ = แผนในอนาคต) 9
ตารางที่ 1.4 ความรับผิดชอบของสมาชิกแต่ละเลนตามแผน 10
ตารางที่ 2.1 เปรียบเทียบแบบจำลองการแยกผู้เช่าในฐานข้อมูล 18
ตารางที่ 3.1 เปรียบเทียบเฟส 1 (Architecture A) กับเฟส 2 (offline shell) 26
ตารางที่ 3.2 ส่วนประกอบหลักของที่เก็บโค้ด 27
ตารางที่ 3.3 คอนเทนเนอร์หลักและเพดานหน่วยความจำ 28
ตารางที่ 3.4 กติกาธุรกิจของ repository ที่ทำธุรกรรม 32
ตารางที่ 3.5 หน้าจอหลักของไคลเอนต์ 34
ตารางที่ 3.6 โมดูลหลักของเซิร์ฟเวอร์ 35
ตารางที่ 3.7 ตารางแกนกลางและข้อกำหนดสำคัญ 40
ตารางที่ 3.8 กลุ่มจุดปลายทางหลักของ API 41
ตารางที่ 3.9 รายการ op ที่เข้าคิวและ endpoint ออนไลน์ที่เป็นคู่กัน 45
ตารางที่ 3.10 มาตรการด้านความปลอดภัยและที่มา 49
ตารางที่ 3.11 เทคโนโลยีที่ใช้ในโครงงาน 54
ตารางที่ 4.1 ลำดับการย้ายระบบเดิมสู่ Flutter (ตรวจสอบจากประวัติ git) 58
ตารางที่ 4.2 ข้อค้นพบจากการทดสอบใช้งานบนเครื่องสาธิตและการแก้ไข 70
ตารางที่ 4.3 ปัญหาของร้านกับฟังก์ชันและหลักฐานในรายงาน 73
ตารางที่ 4.4 งานชั้นเชื่อมต่อ API ฝั่งไคลเอนต์ (fe.0 ถึง fe.3) 74
ตารางที่ 4.5 ตั๋วเฟส 2 ฝั่งอุปกรณ์ที่ปิดแล้ว 75
ตารางที่ 4.6 สรุปองค์ประกอบสำคัญของฝั่งเซิร์ฟเวอร์และหลักฐานการพิสูจน์ 79
ตารางที่ 4.7 หลักฐานการ deploy บนเครื่อง mob04 (30 กันยายน พ.ศ. 2569) 83
ตารางที่ 4.8 งาน Android APK และ TLS ที่รวมเมื่อ 3 ตุลาคม พ.ศ. 2569 และสถานะ 84
ตารางที่ 4.9 การ deploy ขึ้นเครื่อง mob04 ระหว่าง 3–4 ตุลาคม พ.ศ. 2569 85
ตารางที่ 4.10 งานขยายการเฝ้าสังเกตระบบและสถานะ ณ 4 ตุลาคม พ.ศ. 2569 87
ตารางที่ 4.11 จำนวนไฟล์และกรณีทดสอบของเซิร์ฟเวอร์และไคลเอนต์ (นับเมื่อ 1 ตุลาคม พ.ศ. 2569) 90
ตารางที่ 4.12 สถานะเกณฑ์ตรวจรับ (DoD) ของเฟส 1 (17 ข้อ) 91
ตารางที่ 4.13 สถิติภาพรวมของโครงงาน (วัดเมื่อ 1 ตุลาคม พ.ศ. 2569) 93

ตารางที่ 4.14 ผลงานของสมาชิกแต่ละคน (นับเมื่อ 1 ตุลาคม พ.ศ. 2569) 95
ตารางที่ 5.1 ผลการดำเนินงานเทียบกับวัตถุประสงค์ 98
ตารางที่ 5.2 การเปลี่ยนแปลงระหว่างวันที่ 5–7 ตุลาคม พ.ศ. 2569 101
ตารางที่ 5.3 การเปลี่ยนแปลงระหว่างวันที่ 8–12 ตุลาคม พ.ศ. 2569 104

รายการคำย่อ

ACID	Atomicity, Consistency, Isolation, Durability
ADR	Architecture Decision Record
AOF	Append-Only File
AOT	Ahead-Of-Time (compilation)
API	Application Programming Interface
APK	Android Package Kit
ASCII	American Standard Code for Information Interchange
BLoC	Business Logic Component
CA	Certificate Authority
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Delivery
CN	Credit Note (ใบลดหนี้)
CORS	Cross-Origin Resource Sharing
CP	Credit Payment (ใบรับชำระเครดิตช่าง)
CRDT	Conflict-free Replicated Data Type
DDL	Data Definition Language
DoD	Definition of Done
e2e	End-to-End
ER	Entity-Relationship
GHCR	GitHub Container Registry
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
JSON	JavaScript Object Notation

JWT	JSON Web Token
LRU	Least Recently Used
MVCC	Multi-Version Concurrency Control
NAS	Network-Attached Storage
NFC	Normalization Form C (Unicode)
OPFS	Origin Private File System
ORM	Object-Relational Mapping
PDF	Portable Document Format
PIN	Personal Identification Number
PO	Purchase Order
POS	Point of Sale
PR	Pull Request
PWA	Progressive Web App
QT	Quotation (ใบเสนอราคา)
RC	Receipt (เลขที่ใบเสร็จ)
REST	Representational State Transfer
RLS	Row-Level Security
SaaS	Software as a Service
SAN	Subject Alternative Name
SFTP	SSH File Transfer Protocol
SHA	Secure Hash Algorithm
SLI	Service Level Indicator
SMB	Server Message Block
SQL	Structured Query Language
SSH	Secure Shell
TLS	Transport Layer Security

UI	User Interface
VM	Virtual Machine
XSS	Cross-Site Scripting
YAML	YAML Ain't Markup Language

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญ

1.1.1 ร้านอะไหล่รถยนต์และปัญหาของการทำงานด้วยมือ

โครงงานนี้เริ่มจากความต้องการใช้งานจริงของร้านจำหน่ายอะไหล่รถยนต์ขนาดเล็กแห่งหนึ่งในประเทศไทย คือ ร้านศรีสุรัตน์อะไหล่ยนต์ ร้านประเภทนี้มีลักษณะงานที่ต่างจากร้านค้าปลีกทั่วไปหลายประการ กล่าวคือ สินค้ามีจำนวนรายการมากและแต่ละรายการมีรหัสอะไหล่เฉพาะ ลูกค้าส่วนหนึ่งเป็นช่างซ่อมรถที่ซื้อซ้ำเป็นประจำและขอซื้อเชื่อ (เครดิตช่าง) สินค้าชนิดเดียวกันอาจเข้าร้านหลายล็อตในต้นทุนที่ต่างกัน และมีทั้งงานเสนอราคา งานรับคืนสินค้า และงานปิดกะเก็บเงินสดในลิ้นชักทุกวัน

ตามที่สรุปไว้ในเอกสารประกอบโครงงาน [1] ร้านประสบปัญหาหลัก 8 ประการก่อนมีระบบ ดังสรุปในตารางที่ 1.1 ตัวเลขเชิงปริมาณของปัญหา เช่น จำนวนบิลต่อวัน หรือยอดหนี้ช่างรวม ยังไม่มีข้อมูลจากร้านที่ยืนยันได้ คณะผู้จัดทำจึงไม่ระบุตัวเลขเหล่านี้ในรายงานฉบับนี้

ตารางที่ 1.1 ปัญหาของร้านก่อนมีระบบและผลที่ตามมา

รหัส	ปัญหา	ผลที่ตามมา
P1	เขียนบิลด้วยมือเป็นส่วนใหญ่	ช้า ลายมืออ่านยาก บิลสูญหาย ไม่มีการรวมยอดอัตโนมัติ
P2	ไม่แน่ใจว่าสินค้าในสต็อกเหลือหรือไม่	รับปากลูกค้าแล้วสินค้าหมด หรือสั่งซื้อซ้ำทั้งที่ยังมีของ
P3	ไม่แน่ใจราคา และสินค้าที่เข้าต่างล็อตมีต้นทุนต่างกัน	ขายขาดทุนโดยไม่รู้ตัว หรือตั้งราคาสูงเกินไป
P4	ไม่ทราบยอดรับ–จ่ายประจำวัน	เงินในลิ้นชักไม่ตรงกับบิลเมื่อปิดร้าน
P5	ไม่ทราบยอดเครดิตที่ช่างใช้ไป	หนี้ค้างสะสม ทวงถามไม่ตรงตัว ไม่รู้ว่าควรหยุดให้เครดิตเมื่อใด
P6	ไม่ทราบกำไร	ขายได้มากแต่ไม่ทราบว่ากิจการได้กำไรหรือขาดทุน
P7	ไม่ทราบยอดขาย	ตัดสินใจเชิงธุรกิจไม่ได้ เช่น การจ้างงานหรือขยายร้าน
P8	ไม่ทราบว่าสินค้าใดขายดี	สั่งสินค้าผิดตัว เงินทุนจมกับสินค้าที่ไม่หมุนเวียน

ปัญหา P1–P3 เป็นปัญหาที่พนักงานหน้าร้านสัมผัสโดยตรง ส่วน P4–P8 เป็นปัญหา "ข้อมูลที่เจ้าของร้านมองไม่เห็น" การแยกกลุ่มผู้ใช้เช่นนี้มีผลต่อการออกแบบ เพราะพนักงานต้องการขั้นตอนที่รวดเร็ว ขณะที่เจ้าของร้านต้องการตัวเลขที่ถูกต้องและตรวจย้อนหลังได้ ซึ่งบางครั้งขัดกัน เช่น การบังคับให้ยืนยันก่อนขายเชื่อเกินวงเงินทำให้ช้าลงหนึ่งขั้นตอน แต่ป้องกันหนี้ค้างได้

1.1.2 ระบบเดิมและข้อจำกัด

ก่อนโครงงานนี้ ร้านมีแอปพลิเคชันจุดขาย (Point of Sale: POS) รุ่นแรกที่พัฒนาด้วย React ทำงานในเว็บเบราว์เซอร์ และเก็บข้อมูลทั้งหมดใน `localStorage` ของเบราว์เซอร์ [2] กติกาทางธุรกิจทั้งหมด เช่น การตัดสต็อก การคิดต้นทุนเฉลี่ย การคืนสินค้า และการคิดเครดิตช่าง อยู่ในไฟล์ `pos/db.js` ของที่เก็บโค้ดเดิม (tag `v1.0-js-localstorage`) ระบบรุ่นนี้ไม่ต้องมีเซิร์ฟเวอร์ และเป็นต้นแบบเชิงพฤติกรรมที่โครงงานนี้ยึดเป็นแหล่งความจริง (source of truth) ของกติกาทางธุรกิจ อย่างไรก็ตาม ระบบรุ่นแรกมีข้อจำกัดสำคัญ [3] ดังนี้

- ข้อมูลผูกอยู่กับเบราว์เซอร์และเครื่องเดียว หากล้างข้อมูลของเบราว์เซอร์หรือเครื่องเสีย ข้อมูลหายพร้อมกัน
- ไม่มีธุรกรรม (transaction) ที่แท้จริงในระดับฐานข้อมูล การทำหลายขั้นตอนให้สำเร็จทั้งหมดหรือไม่ทำเลยต้องอาศัยการเก็บภาพถ่าย (snapshot) และย้อนกลับ (rollback) ที่เขียนเองในโค้ด JavaScript
- ไม่สามารถใช้ข้อมูลร่วมกันระหว่างหลายเครื่อง และที่เก็บข้อมูลของเบราว์เซอร์มีข้อจำกัดด้านขนาดและเงื่อนไขการลบที่เบราว์เซอร์เป็นผู้กำหนด [2]

1.1.3 การย้ายมาใช้ Flutter แบบ offline-first

ขั้นถัดมา โครงงานย้ายแอปไปยัง Flutter [4] เพราะเฟรมเวิร์กนี้สร้างแอปสำหรับ Android, iOS และเว็บได้จากรหัสต้นฉบับชุดเดียว และเปลี่ยนที่เก็บข้อมูลเป็น Drift ซึ่งเป็นไลบรารีที่ห่อ SQLite [5][6] ทำให้ได้ธุรกรรมของฐานข้อมูลจริงแทนกลไก snapshot/rollback เดิม สถาปัตยกรรมนี้เรียกว่า offline-first กล่าวคือ ฐานข้อมูลในเครื่องเป็นที่เก็บข้อมูลตัวจริง ร้านจึงขายสินค้าได้ต่อเนื่องแม้อินเทอร์เน็ตขัดข้อง

การย้ายมา Flutter เริ่มเมื่อวันที่ 23 มิถุนายน พ.ศ. 2569 โดยในวันเดียวกันมีทั้งโครงสร้างโครงงาน ชั้นข้อมูลพร้อมการทดสอบ และหน้าจอ 11 หน้าที่มีในขณะนั้น ต่อมามีการปรับการแสดงผลบนอุปกรณ์ขนาดต่าง ๆ และย้ายการจัดการสถานะจาก Riverpod ไปเป็น flutter_bloc เมื่อวันที่ 14 กรกฎาคม พ.ศ. 2569 ผลลัพธ์คือแอป Flutter แบบ offline-first ที่ร้านใช้งานจริงอยู่ และถูกเก็บรักษาไว้เป็นสแนปช็อตอ้างอิงบนสาขา `POC_sample_offline_first` สแนปช็อตข้อมูลจริงของร้านถูกนำเข้าร้านสาธิตบนเซิร์ฟเวอร์เมื่อ 17 กันยายน พ.ศ. 2569 (#185) แต่ยังไม่มีการเก็บความเห็นหรือประเมินผลจากผู้ใช้อย่างเป็นทางการ

1.1.4 เหตุผลที่ต้องมีระบบเซิร์ฟเวอร์แบบหลายร้านค้า

แม้แอป offline-first จะแก้ปัญหาการทำงานหน้าร้านได้ แต่โครงงานในรายวิชา 240-401 ถูกกำหนดโจทย์ให้ขยายเป็นระบบที่มีเซิร์ฟเวอร์กลางและรองรับร้านอะไหล่หลายร้านที่เป็นเจ้าของคนละราย (multi-tenant) ซึ่งเจ้าของโครงงานยืนยันโมเดลนี้เมื่อวันที่ 3 กันยายน พ.ศ. 2569 [7] ในรายงานนี้ **เจ้าของโครงงาน** (product owner) หมายถึง Chavatik Thorarit (NuimanLP) ในฐานะหัวหน้าทีม — โปรดยืนยันชื่อและบทบาท ผู้ตัดสินขอบเขต ลำดับงาน และเกณฑ์ตรวจรับ ข้อความภาษาไทยบนหน้าจอรับรองโดย เจ้าของโครงงานหรือเจ้าของร้าน — โปรดระบุ และคำว่า **เจ้าของร้าน** หมายถึงเจ้าของร้านศรีสุรัตน์อะไหล่ยนต์ ส่วนร้านที่ใช้ระบบเรียกว่า "ผู้เช่า" (tenant) ในเชิง

เทคนิค และ "ร้านค้า" เมื่อกล่าวถึงงานของร้าน ปัจจุบันยังไม่มีร้านที่สองที่ใช้งานจริง การแยกผู้เช่าจึงพิสูจน์ด้วยร้านสาธิตหลายร้านในชุดทดสอบ (`cross-tenant-read.e2e-spec.ts`)

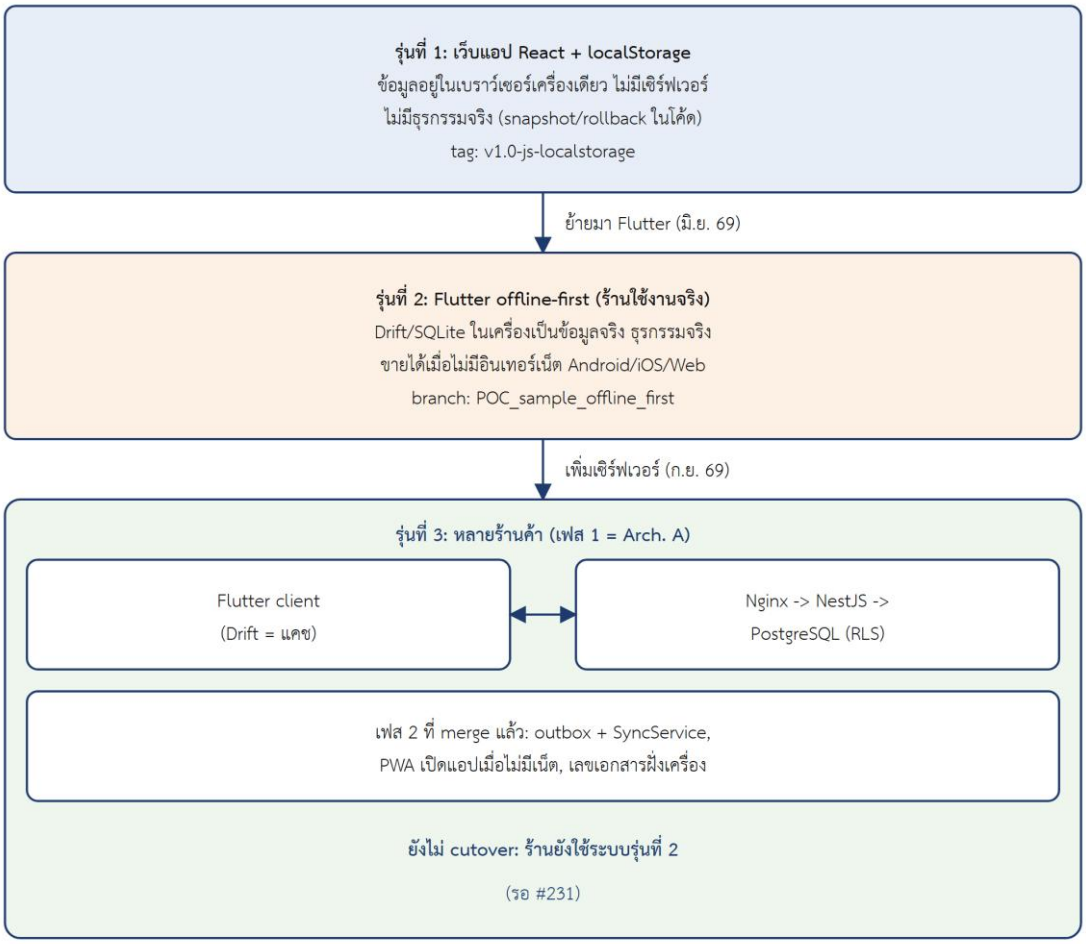
แรงผลักดันของการมีเซิร์ฟเวอร์มี 4 ประการ [3]

1. โจทย์เปลี่ยนเป็นบริการแบบ SaaS (Software as a Service) ที่ให้บริการหลายร้านที่ไม่เกี่ยวข้องกัน ข้อมูลจึงต้องอยู่ศูนย์กลางและต้องแยกขาดจากกันอย่างเด็ดขาด
2. รายวิชากำหนดชุดเทคโนโลยีไว้ ได้แก่ NestJS, PostgreSQL, Redis, BullMQ และ Nginx [3][7]
3. ข้อมูลที่อยู่ในเครื่องเดียวสูญหายพร้อมเครื่อง จึงต้องมีที่เก็บกลางที่สำรองข้อมูลได้
4. ร้านหนึ่งอาจมีหลายเครื่อง เช่น เครื่องขายหน้าร้าน (`pos`) และเครื่องหลังร้าน (`backoffice`) ซึ่งต้องเห็นสต็อกชุดเดียวกัน (บันทึกการตัดสินใจเชิงสถาปัตยกรรม (Architecture Decision Record: ADR) หมายเลข 0004 [8])

การเลือกสถาปัตยกรรมเริ่มจากคำถามแกนว่า "สต็อกตัวจริงอยู่ที่เซิร์ฟเวอร์หรือที่เครื่อง" เอกสารออกแบบเปรียบเทียบสามทางเลือก คือ A (online-first: ทุกการเขียนต้องผ่านเซิร์ฟเวอร์) B (offline-first: เครื่องเป็นที่เก็บหลักแล้วซิงก์ภายหลัง) และ C (ไฮบริด: เริ่มแบบ A แล้วเพิ่มการทำงานออฟไลน์แบบจำกัดภายหลัง) ผลการตัดสินคือเลือก C โดยกำหนดให้เฟส 1 ของ C เท่ากับ A ทุกประการ [7] เหตุผลคือเฟส 1 ตรงกับสิ่งที่รายวิชากำหนดและไม่ต้องรอกลไกซิงก์ ขณะที่ความสามารถ "ขายได้เมื่ออินเทอร์เน็ตขัดข้อง" ไม่ถูกทิ้ง แต่กลับมาในเฟส 2 ด้วยกลไกคิวคำสั่งในเครื่องที่เรียกว่ากล่องขาออก (outbox) ราคาที่ต้องจ่ายคือต้องปรับชั้น repository ของ Flutter ให้เรียกส่วนต่อประสานโปรแกรมประยุกต์ (Application Programming Interface: API) และต้องดูแลเส้นทางโค้ดทั้งออนไลน์และออฟไลน์ ทั้งนี้ ADR-0010 กำหนดให้ Drift ยังคงอยู่เป็นแคชแบบ write-through แทนที่จะถูกลบทิ้ง [9]

สำหรับการแยกข้อมูลของแต่ละร้าน โครงงานเลือกใช้ฐานข้อมูลและสคีมาเดียวร่วมกัน โดยทุกตารางมีคอลัมน์ `tenant_id` และเปิดใช้การรักษาความปลอดภัยระดับแถว (Row-Level Security: RLS) ของ PostgreSQL [10] เพราะต้นทุนต่อร้านต่ำที่สุด ส่วนข้อเสนอให้เปลี่ยนฐานข้อมูลเป็น CouchDB ถูกปฏิเสธเมื่อวันที่ 8 กันยายน พ.ศ. 2569 (ADR-0012 [11]) เหตุผลของทั้งสองเรื่องอธิบายไว้ในบทที่ 2 และกลไกที่สร้างจริงอยู่ในบทที่ 3

ข้อควรย้ำคือ การมีเซิร์ฟเวอร์ไม่ได้หมายความว่าร้านจริงย้ายไปใช้ระบบใหม่แล้ว เฟส 1 กำหนดชัดว่า "ไม่มี cutover" ร้านยังคงใช้แอป Drift เดิม ส่วนเซิร์ฟเวอร์พัฒนาและทดสอบกับผู้เช่าสาธิต (demo tenant) [7] ดังสรุปในรูปที่ 1.1



รูปที่ 1.1 วิวัฒนาการของระบบ POS ร้านศรีสุรัตน์จากรุ่นเว็บ localStorage สู่ระบบหลายร้านค้า

1.2 วัตถุประสงค์ของโครงงาน

โครงงานมีวัตถุประสงค์ดังต่อไปนี้

1. เพื่อพัฒนาแอปพลิเคชัน POS สำหรับร้านอะไหล่รถยนต์ด้วย Flutter ที่ทำงานแบบ offline-first บนเว็บเป็นหลัก (โค้ดชุดเดียวรองรับ Android และ iOS ตามความสามารถของเฟรมเวิร์ก แต่ยังไม่ได้สร้างและทดสอบบนอุปกรณ์จริง) โดยคงกติกาทางธุรกิจของระบบเดิมไว้ครบถ้วน ได้แก่ การขาย การคืนสินค้า การรับสินค้าเข้าและคิดต้นทุนเฉลี่ยถ่วงน้ำหนัก ใบเสนอราคา เครดิตช่าง การเปิด–ปิดกะ และรายงานยอดขาย กำไร และสินค้าขายดี
2. เพื่อพัฒนาระบบเซิร์ฟเวอร์ด้วย NestJS, PostgreSQL, Redis, BullMQ และ Nginx ที่รองรับหลายร้านค้า โดยให้ฐานข้อมูล PostgreSQL เป็นแหล่งความจริง และย้ายกติกาธุรกรรมที่เกี่ยวกับเงินและสต็อกไปบังคับใช้ฝั่งเซิร์ฟเวอร์

3. เพื่อออกแบบและพิสูจน์การแยกข้อมูลระหว่างร้านค้าด้วย `tenant_id` และ RLS พร้อมการทดสอบที่แสดงว่าการอ่านข้ามร้านได้ผลลัพธ์ศูนย์แถว
4. เพื่อให้การเขียนข้อมูลที่เกี่ยวกับเงินและสต็อกปลอดภัยเมื่อมีคำขอซ้ำหรือพร้อมกัน ด้วยกลไก idempotency key และการล็อกแถวตามลำดับที่กำหนด
5. เพื่อพัฒนากลไกการทำงานเมื่อไม่มีอินเทอร์เน็ต (เฟส 2) ได้แก่ คิวคำสั่งในเครื่อง การซิงก์ การออกเลขเอกสารฝั่งเครื่อง และการผูกอุปกรณ์กับร้าน ตามสเปกของเฟส 2 [12]
6. เพื่อสร้างกระบวนการ CI/CD (Continuous Integration / Continuous Delivery) ที่ครอบคลุมการทดสอบอัตโนมัติ การสแกนความปลอดภัย การสร้างอิมเมจ การนำระบบขึ้นใช้งาน (deploy) บนเครื่องเสมือน (Virtual Machine: VM) ของคณะ และการเฝ้าสังเกตระบบ [13]
7. เพื่อบันทึกการตัดสินใจเชิงสถาปัตยกรรมเป็น ADR พร้อมเหตุผล เพื่อให้ผู้ร่วมงานและผู้ตรวจสอบตามย้อนได้ว่าเหตุใดจึงเลือกแต่ละแนวทาง

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตที่อยู่ในโครงงาน

ขอบเขตของโครงงานแบ่งตามเฟสของสถาปัตยกรรมแบบไฮบริด ดังสรุปในตารางที่ 1.2

ตารางที่ 1.2 ขอบเขตของโครงงานจำแนกตามเฟส

เฟส	สิ่งที่อยู่ในขอบเขต	สถานะ ณ วันที่ 1 ตุลาคม พ.ศ. 2569
ฐาน Flutter	แอป offline-first (Drift/SQLite) หน้าจอ 12 หน้าในแถบนำทางหลัก กติกาธุรกรรมที่พอร์ตจาก `db.js`	แล้วเสร็จ ร้านใช้งานจริงบนเว็บ (ยังไม่ทดสอบบน Android/iOS)
เฟส 1	เซิร์ฟเวอร์ NestJS, PostgreSQL พร้อม RLS, ชั้นธุรกรรมและ idempotency, การจัดการร้านค้าโดยผู้ดูแลแพลตฟอร์ม, ชั้นเขียนข้อมูลผ่าน API ของ Flutter, CI/CD ระดับ 1–3	แล้วเสร็จ เว้นการวัดภาระด้วย k6
เฟส 2	เว็บแอปแบบก้าวหน้า (Progressive Web App: PWA), outbox, `SyncService`, เลขใบเสร็จ (RC) และใบลดหนี้ (CN) ฝั่งเครื่อง, รหัสส่วนตัว (Personal Identification Number: PIN) แบบออฟไลน์, การดึงข้อมูล, หลายกะต่อวัน	ตั๋วงานส่วนใหญ่ผ่านการรวมโค้ดแล้ว ตั๋วงานหลักของลำดับเริ่มต้นเฟส 2 ปิดแล้ว 7 จาก 8 ตั๋ว (เหลือ #231)
การปฏิบัติการ	นำระบบขึ้นใช้งาน (deploy) บน VM `mob04` ด้วย self-hosted runner และ Ansible, Prometheus และ Grafana, etcd	deploy จริงครั้งแรกเมื่อ 30 กันยายน พ.ศ. 2569 การสาธิตครบวงจรยังไม่ได้รัน

1.3.2 ขอบเขตที่อยู่นอกโครงงาน

- **การย้ายร้านจริงไปใช้ระบบใหม่ (cutover)** ถูกเลื่อนไปเฟสถัดไปตามการตัดสินใจของเจ้าของโครงงานเมื่อวันที่ 15 กันยายน พ.ศ. 2569 (#231 ยังเปิดอยู่)
- **ใบกำกับภาษีเต็มรูป** ไม่อยู่ในขอบเขตของเวอร์ชันแรก

- **ฮาร์ดแวร์เฉพาะทาง** ได้แก่ เครื่องพิมพ์ใบเสร็จความร้อน ลิ้นชักเก็บเงิน และการสแกนบาร์โค้ด ซึ่งต้องทดสอบที่ร้านจริง ปัจจุบันการสแกนใช้การป้อนด้วยมือ (ต่อมาเพิ่มการสแกนด้วยกล้องแล้ว ดูหัวข้อสถานะล่าสุด 2026-10-12)
- **การสำรองข้อมูลออกนอก VM (#363, #288)** ถูกพักไว้จนกว่าจะเสร็จการสาธิตตามการตัดสินใจเมื่อวันที่ 22 กันยายน พ.ศ. 2569 ดังนั้นในขณะนี้ยังไม่มีข้อมูลสำรองที่ออกจาก VM
- **การให้ร้านสมัครใช้เองแบบ self-service** ไม่ทำในเฟส 1 ผู้ดูแลแพลตฟอร์มเป็นผู้สร้างร้าน (ADR-0001 [14])
- **การย้ายขึ้นผู้ให้บริการคลาวด์ภายนอก** ถูกปฏิเสธโดยเจ้าของโครงงานเมื่อวันที่ 15 กันยายน พ.ศ. 2569

1.3.3 สิ่งที่รายงานนี้ยังไม่ถือว่าแล้วเสร็จ

งานที่ยังเปิดอยู่ ได้แก่ การวัดภาระด้วย k6 (#380 วัดจริงครั้งแรกแล้วเมื่อ 5 ตุลาคม พ.ศ. 2569 แต่ยังไม่ได้รับผล) และการสาธิตครบวงจรบน `mob04` (#344) ส่วนประเด็นการจัดการอุปกรณ์ (#476) ปิดแล้วเมื่อ 3 ตุลาคม พ.ศ. 2569 สำหรับเกณฑ์ตรวจรับของเฟส 1 (Definition of Done: DoD) 17 ข้อ ผ่านแล้ว 16 ข้อ เหลือข้อ k6 หนึ่งข้อ [7] รายละเอียดอยู่ในตารางที่ 4.12 ของบทที่ 4

1.4 แผนการดำเนินงาน

1.4.1 ลำดับเหตุการณ์ที่เกิดขึ้นจริง

ลำดับการดำเนินงานจริงสรุปจากประวัติ `git log` และบันทึกส่งมอบงานของโครงงาน ปริมาณงานกระจุกตัวในเดือนกันยายน พ.ศ. 2569 กล่าวคือ จากคอมมิตทั้งหมด 1,058 รายการ มี 962 รายการเกิดในเดือนกันยายน เทียบกับ 18 รายการในเดือนมิถุนายน 17 รายการในเดือนกรกฎาคม 2 รายการในเดือนสิงหาคม และ 59 รายการในวันที่ 1 ตุลาคม ช่วงแรก (มิถุนายน–กรกฎาคม) เป็นการพัฒนาแอป Flutter ช่วงปลายเดือนสิงหาคมเป็นการจัดทำเอกสารออกแบบซิร์ฟเวอร์ ส่วนเดือนกันยายนเป็นการพัฒนาเซิร์ฟเวอร์ CI/CD และกลไกเฟส 2 แบบคู่ขนานโดยสมาชิกสามคน เหตุการณ์สำคัญได้แก่ การบันทึก ADR-0001 ถึง 0009 และกำหนดให้สาขา `main` เป็นแนวพัฒนาหลายร้านค้า (4 กันยายน) การปิดงานเฟส 1 ส่วนใหญ่และเขียนสเปกเฟส 2 (15–17 กันยายน) และการ deploy ขึ้น VM `mob04` ครั้งแรก (30 กันยายน)

1.4.2 ตารางแผนงานแบบแกนต์

ตารางที่ 1.3 แสดงงานตามช่วงเดือนของปีการศึกษา สัญลักษณ์ ■ หมายถึงช่วงที่ดำเนินการแล้วตามหลักฐาน `git log` ส่วน □ หมายถึงแผนในอนาคต ซึ่งเป็นแผนที่คณะผู้จัดทำเสนอให้อาจารย์ที่ปรึกษาพิจารณา แผนของเฟส 1 จัดลำดับตามความพึ่งพาของงาน [7]

ตารางที่ 1.3 แผนการดำเนินงานของโครงงาน (■ = ดำเนินการแล้ว, □ = แผนในอนาคต)

งาน	มิ.ย. 69	ก.ค. 69	ส.ค. 69	ก.ย. 69	ต.ค. 69	พ.ย. 69	ธ.ค. 69	ม.ค. 70	ก.พ. 70	มี.ค. 70
1. พอร์ตแอป Flutter offline-first (ขั้น 0–6) และปรับหน้าจอ	■	■								
2. ปรับปรุงโค้ดและย้ายเป็น flutter_bloc	■	■								
3. ออกแบบ backend และบันทึก ADR			■	■						
4. เฟส 1: เซิร์ฟเวอร์ ฐานข้อมูล RLS ธุรกรรม				■						
5. ชั้นเขียนข้อมูลผ่าน API ของ Flutter				■						
6. CI/CD ระดับ 1–3				■						
7. เฟส 2: outbox, SyncService, PWA, เลขเอกสาร				■	□					
8. deploy ขึ้น VM mob04 และการเฝ้าสังเกตระบบ				■	□					
9. สาธิตครบวงจร (#344)					□					
10. วัดภาระ k6 สามเครื่อง (#380)					□	□				
11. สำรองข้อมูลออกนอก VM (#363, #288)						□	□			
12. นำร่องกับร้านจริงและ cutover (#231)							□	□		
13. ฮาร์ดแวร์เฉพาะร้าน ทดสอบบน Android/iOS และเอกสารภาษี								□	□	
14. ทดสอบ ปรับปรุง และจัดทำรายงานฉบับสมบูรณ์									□	□

แถวที่ 7 และ 8 มีทั้ง ■ และ □ เพราะงานเริ่มแล้วแต่ยังไม่ปิด แถวที่ 9–11 เรียงตามที่เจ้าของโครงงานกำหนดว่าการสาธิตมาก่อน และงานสำรองข้อมูลออกนอก VM ต้องรอหลังการสาธิต [15] ส่วนแถว 12–14 ยังไม่ระบุวันที่แน่นอนเพราะต้องรอผลการสาธิตและการเข้าถึงร้านจริง

1.4.3 โครงสร้างทีมและความรับผิดชอบ

ทีมประกอบด้วยสมาชิกสามคน ตามกติกาของรายวิชาที่ประกาศเมื่อวันที่ 5 กันยายน พ.ศ. 2569 สมาชิกทุกคนต้องมีงานทั้งในส่วน frontend, backend และ CI/CD โดยแต่ละคนรับผิดชอบเลน (lane) หนึ่งเลนเป็นหลัก ดังตารางที่ 1.4 ในเฟส 2 เจ้าของโครงงานอนุมัติการแบ่งใหม่เมื่อวันที่ 16 กันยายน พ.ศ. 2569 เป็น 35 ตั๋วงาน (issue บน GitHub) แบ่งเป็นเลน A 3 ตั๋ว เลน B 15 ตั๋ว

และเลน C 17 ตั๋ว [15] หลักการแบ่งเลนอธิบายไว้ในหัวข้อวิธีการทำงานของทีมในบทที่ 3 ตารางที่ 1.4 แสดงการแบ่งงานตามแผน ส่วนผลงานจริงของสมาชิกแต่ละคนนับจาก PR ที่รวมแล้วในตารางที่ 4.14 ของบทที่ 4

ตารางที่ 1.4 ความรับผิดชอบของสมาชิกแต่ละเลนตามแผน

เลน	สมาชิก (GitHub)	ความรับผิดชอบในเฟส 1	ความรับผิดชอบในเฟส 2
A (`team/1`)	Chavatik Thorarit (NuimanLP)	เส้นทางเงิน: Docker Compose, idempotency, เลขเอกสาร, การขาย การคืนและการยกเลิกบิล, เครดิตช่าง, กะและรายงานปิดร้าน; CI: สร้างอิมเมจ (#40); ฝั่งแอป: หน้าขาย คืน ลิ้นชัก (เขียนผ่าน API)	ข้อตกลงระหว่างสองฝั่ง (`SyncFacade` และไฟล์ตัวอย่าง `/sync/push`), ข้อความภาษาไทยของเฟส 2, รายการอนุญาตผู้ดูแลแพลตฟอร์ม (3 ตั๋ว)
B (`team/2`)	LomerAlloys	สคีมาและ migration, การสร้างร้านค้า, สินค้า ลูกค้าและช่าง, ใบสั่งซื้อ, ใบเสนอราคา, รายงานฝั่งเซิร์ฟเวอร์; CI: path filter (#39); ฝั่งแอป: หน้าอ่านข้อมูลและ Drift schema v3	เครื่องยนต์ฝั่งเครื่องทั้งหมด: PWA, outbox, SyncService, เลข RC/CN, PIN ออฟไลน์, การดึงข้อมูล และการเปลี่ยนสคีมา Drift (15 ตั๋ว)
C (`team/3`)	Pattarapon Kitcharoen (PattaraponKitcharoen)	ยืนยันตัวตนด้วย JSON Web Token (JWT), บทบาทอุปกรณ์, แคช Redis, การจำกัดอัตรา, คิว BullMQ, การส่งออกข้อมูลร้าน, k6, CI ฝั่งเซิร์ฟเวอร์และด่านความปลอดภัย; ฝั่งแอป: เข้าสู่ระบบและ device token	เซิร์ฟเวอร์ หน้าจอใหม่ที่ไม่ผูกกับเครื่องยนต์ซิงก์ และงานปฏิบัติการ ได้แก่ VM, runner, การเฝ้าสังเกตระบบ (17 ตั๋ว)

1.5 ประโยชน์ที่คาดว่าจะได้รับ

1. **ร้านค้า** ได้ระบบที่ตัดสต็อกจริง คิดต้นทุนเฉลี่ยถ่วงน้ำหนักเมื่อรับสินค้าแต่ละล็อต คุมเครดิตช่าง และออกรายงานยอดขายและกำไร ซึ่งออกแบบเพื่อตอบปัญหา P1–P8 (ยังไม่ได้ประเมินผลกับผู้ใช้จริง ดูตารางที่ 4.3 ในบทที่ 4) โดยยังขายได้เมื่ออินเทอร์เน็ตขัดข้อง
2. **เจ้าของร้านหลายราย** ได้ข้อมูลที่แยกขาดจากกันด้วย RLS ข้อมูลอยู่ที่ศูนย์กลางและสำรองรายวันบนเครื่องเซิร์ฟเวอร์ (การสำรองออกนอกเครื่องยังเป็นงานค้าง #363)
3. **คณะผู้จัดทำ** ได้ประสบการณ์การออกแบบระบบกระจายตัวที่มีธุรกรรมเกี่ยวกับเงิน การจัดการสิทธิ์หลายผู้เช่า การซิงก์ข้อมูลแบบออฟไลน์ และการสร้างไปป์ไลน์ CI/CD ครบวงจร
4. **ผู้ที่สนใจต่อยอด** ได้ชุดเอกสารและบันทึกการตัดสินใจ (ADR) ที่ตามย้อนเหตุผลของการออกแบบได้ และได้ชุดทดสอบที่ใช้เป็นหลักฐานของพฤติกรรมของระบบ

1.6 ลำดับการเรียบเรียงรายงาน

บทที่ 1 อธิบายที่มา วัตถุประสงค์ ขอบเขต และแผนการดำเนินงาน บทที่ 2 อธิบายทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง บทที่ 3 อธิบายการออกแบบและวิธีสร้างระบบ บทที่ 4 รายงานผลที่ได้พร้อมหลักฐาน และบทที่ 5 สรุปผลการดำเนินงาน ปัญหา และงานต่อไป ทุกตัวเลขและสถานะงานในรายงานตรวจสอบกับที่เก็บโค้ด ประวัติ `git` และบันทึกส่งมอบงานของโครงงาน หากรายการใดยังไม่แล้วเสร็จจะระบุไว้อย่างชัดเจน

บทที่ 2
ทฤษฎีและความรู้พื้นฐาน

2.1 บทนำ

บทนี้รวบรวมทฤษฎี แนวคิด และเทคโนโลยีที่โครงงานอาศัยเป็นฐาน โดยจัดลำดับจากโดเมนธุรกิจ (ระบบขายหน้าร้าน) ไปสู่สถาปัตยกรรมฝั่งไคลเอนต์ (offline-first, Flutter, Drift) ต่อด้วยฝั่งเซิร์ฟเวอร์ (ธุรกรรม การแยกข้อมูลผู้เช่า การยืนยันตัวตน) และปิดท้ายด้วยโครงสร้างพื้นฐานและกระบวนการส่งมอบ บทนี้ไม่ทบทวนเทคโนโลยีทั้งหมดอย่างครอบคลุม แต่เลือกเฉพาะแนวคิดที่มีผลต่อการตัดสินใจเชิงออกแบบ ส่วนวิธีที่โครงงานนำแนวคิดเหล่านี้ไปสร้างจริงอธิบายไว้ในบทที่ 3

2.2 แนวคิดของระบบขายหน้าร้านและโดเมนอะไหล่รถยนต์

ระบบขายหน้าร้าน (Point of Sale: POS) คือระบบที่บันทึกการขายสินค้า ณ จุดชำระเงิน พร้อมปรับปรุงสต็อก ยอดลูกค้า และสมุดบัญชีเงินสดของร้านให้สอดคล้องกัน ร้านอะไหล่รถยนต์มีลักษณะเฉพาะ เช่น สินค้าจำนวนมากที่ค้นด้วยเลขอะไหล่ (part number) ลูกค้าประจำที่เป็นช่างซึ่งซื้อเป็นเครดิต และการเสนอราคาก่อนขาย แนวคิดหลักที่ระบบต้องรักษาให้ถูกต้องมีดังนี้

2.2.1 การขายและค่าคงที่ของสต็อก

การขายหนึ่งครั้งประกอบด้วยหัวบิลและรายการสินค้า และต้องทำหลายอย่างพร้อมกันหรือไม่ทำเลย ได้แก่ ตรวจว่าสต็อกพอ ตัดสต็อก บันทึกบิลและเลขที่ใบเสร็จ สะสมคะแนนสะสม (แต้ม) ของลูกค้า และปรับยอดของช่างเมื่อชำระด้วยเครดิต ค่าคงที่ (invariant) ที่สำคัญที่สุดคือสต็อกห้ามติดลบจากการขาย หากสต็อกไม่พอต้องปฏิเสธบิลทั้งใบ ลักษณะ "ทำทั้งหมดหรือไม่ทำเลย" นี้ตรงกับคุณสมบัติความเป็นอะตอม (atomicity) ของธุรกรรมฐานข้อมูลที่กล่าวในหัวข้อการควบคุมภาวะพร้อมกัน

2.2.2 การคืนสินค้าและใบลดหนี้

การคืนสินค้าสร้างเอกสารใบลดหนี้ (credit note: CN) ที่อ้างอิงบิลเดิม หลักสำคัญคือห้ามคืนเกินจำนวนที่ขายหักด้วยจำนวนที่เคยคืนไปแล้ว และต้องย้อนส่วนลด แต้ม และยอดสะสมของช่างตามสัดส่วนของสินค้าที่คืน

2.2.3 การรับสินค้าเข้าและต้นทุนถัวเฉลี่ยถ่วงน้ำหนัก

ใบสั่งซื้อ (Purchase Order: PO) เมื่อรับสินค้าจะเพิ่มสต็อกและปรับต้นทุนต่อหน่วยด้วยวิธีถัวเฉลี่ยถ่วงน้ำหนัก (weighted-average cost) ซึ่งคำนวณจากสูตร

ต้นทุนใหม่ = (จำนวนเดิม × ต้นทุนเดิม + จำนวนที่รับ × ต้นทุนที่รับ) / (จำนวนเดิม + จำนวนที่รับ)

วิธีนี้ทำให้ต้นทุนสะท้อนราคาที่ซื้อในหลายรอบโดยไม่ต้องติดตามล็อตของสินค้าทีละชิ้น อย่างไรก็ตาม หากรายงานกำไรคำนวณจากต้นทุนปัจจุบัน กำไรของบิลเก่าจะเปลี่ยนไปเมื่อต้นทุนเปลี่ยน ระบบจึงควรบันทึกต้นทุน ณ เวลาขาย (cost at sale) ไว้กับบรรทัดบิล

2.2.4 กะ ลิ้นชักเงินสด และเครดิตช่าง

กะ (shift) คือช่วงเวลาที่พนักงานรับผิดชอบลิ้นชักเงินสดชุดหนึ่ง เริ่มด้วยเงินสดตั้งต้น มีรายการนำเงินเข้า-ออก และปิดกะด้วยการนับเงินเทียบกับยอดที่ระบบคาดหวัง ซึ่งต้องคำนวณด้วยวิธีเดียวทุกจุด มิฉะนั้นหน้าจอสองแห่งจะแสดงยอดคาดหวังไม่ตรงกัน ส่วนเครดิตช่างคือบัญชีลูกหนี้ของช่างแต่ละราย ประกอบด้วยยอดค้าง วงเงิน และประวัติการชำระ การขายเกินวงเงินควรต้องได้รับการยืนยันจากผู้ใช้อย่างชัดแจ้ง

2.2.5 เลขที่เอกสาร

เลขที่ใบเสร็จ (RC) และใบลดหนี้ (CN) ต้องไม่ซ้ำและเรียงต่อเนื่อง เพื่อให้ตรวจสอบทางบัญชีได้ เมื่อหลายเครื่องออกเลขได้ รูปแบบเลขต้องรวมหมายเลขเครื่องและงวดเวลา เช่น `RC01-2569-08-0042` ตาม ADR-0007 ของโครงงาน การให้เครื่องที่ออฟไลน์ออกเลขเองโดยไม่ชนกับเซิร์ฟเวอร์จึงเป็นโจทย์เชิงระบบกระจาย

2.2.6 เหตุที่ไม่ใช้ระบบ POS สำเร็จรูป

คณะผู้จัดทำยังไม่ได้สำรวจเปรียบเทียบผลิตภัณฑ์ POS สำเร็จรูปอย่างเป็นระบบ จึงไม่อ้างคุณสมบัติของผลิตภัณฑ์รายใด เหตุผลที่พัฒนาเองมีสองข้อ ข้อแรก กติกาเฉพาะของร้าน เช่น เครดิตช่างพร้อมวงเงิน ราคาช่าง การย้อนส่วนลดและแต้มตามสัดส่วนเมื่อคืนสินค้า และข้อความภาษาไทยที่พนักงานคุ้นเคย อยู่ในไฟล์ `pos/db.js` ซึ่งโครงงานต้องคงพฤติกรรมไว้ทุกข้อ ข้อที่สอง รายวิชากำหนดให้สร้างเซิร์ฟเวอร์หลายผู้เช่าด้วยชุดเทคโนโลยีที่กำหนด ซึ่งการซื้อระบบสำเร็จรูปไม่ตอบโจทย์นี้ การเปรียบเทียบกับผลิตภัณฑ์ในตลาดด้านการขายออฟไลน์ เครดิตช่าง ใบเสร็จภาษาไทย และค่าใช้จ่าย เป็นงานที่ควรเพิ่มในรายงานฉบับสมบูรณ์

2.3 สถาปัตยกรรมออฟไลน์เป็นหลัก (offline-first)

2.3.1 แนวคิด local-first และ offline-first

ระบบ offline-first ออกแบบให้แอปทำงานได้เต็มที่กับข้อมูลในเครื่องเป็นหลัก แล้วซิงโครไนซ์กับเซิร์ฟเวอร์เมื่อมีเครือข่าย ต่างจากแอปแบบ online-first ที่ทุกการกระทำต้องรอเซิร์ฟเวอร์ Kleppmann และคณะ [16] เสนอแนวคิด local-first software พร้อมอุดมคติเจ็ดข้อ เช่น การตอบสนองเร็วเพราะไม่ต้องรอเครือข่าย การทำงานได้ขณะออฟไลน์ และผู้ใช้เป็นเจ้าของข้อมูลของตน บทความชี้ว่าความท้าทายหลักคือการรวมการแก้ไขจากหลายอุปกรณ์ ซึ่งงานของคณะผู้เขียนแก้ด้วยโครงสร้างข้อมูลแบบ CRDT (Conflict-free Replicated Data Type) สำหรับร้านค้าปลีก เหตุผลเชิงปฏิบัติคืออินเทอร์เน็ตของร้านอาจขัดข้อง แต่เคาน์เตอร์ต้องขายต่อได้

2.3.2 เหตุที่ไม่ใช้ CRDT หรือการทำสำเนา (replicate) ฐานข้อมูลโดยตรง

CRDT เหมาะกับข้อมูลที่ผสานได้โดยไม่ขัดแย้ง เช่น เอกสารร่วมแก้ไข แต่ข้อมูลการเงินและสต็อกมีค่าคงที่ที่ต้องรักษา เช่น สต็อกห้ามติดลบและเลขที่ใบเสร็จห้ามซ้ำ การผสานอัตโนมัติอาจทำให้ขายเกินสต็อกโดยไม่มีใครถูกปฏิเสธ ADR-0012 ของโครงงานพิจารณาข้อเสนอให้ใช้ CouchDB ที่ replicate ได้ในตัวแทน PostgreSQL แล้วมีมติไม่ใช้ เพราะการสลับฐานข้อมูลเพียงอย่างเดียวทำให้เสียธุรกรรมข้ามตาราง การล็อกแถว คีย์นอก และ RLS โดยไม่ได้สิ่งทดแทน และการป้องกันการขายเกินสต็อกที่แท้จริงมาจากการจำกัดให้มีเครื่องที่เขียนออฟไลน์ได้ (writer) เพียงเครื่องเดียวต่อร้าน ไม่ใช่จากชนิดของฐานข้อมูล ผลลัพธ์คือสถาปัตยกรรมที่เซิร์ฟเวอร์เป็นแหล่งความจริง ส่วนฐานข้อมูลในเครื่องเป็นแคชและเปลือกสำหรับใช้งานออฟไลน์ (offline shell) ที่บันทึกคำสั่งไว้ส่งทีหลัง

2.3.3 รูปแบบกล่องขาออก (outbox pattern)

เมื่อแอปเขียนข้อมูลขณะออฟไลน์ ต้องบันทึกทั้งผลลัพธ์ในเครื่องและ "ความตั้งใจที่จะส่งเซิร์ฟเวอร์" ในธุรกรรมเดียวกันของฐานข้อมูลในเครื่อง แล้วให้ตัวส่งอ่านคิวนั้นไปส่งภายหลัง รูปแบบนี้เรียกว่า outbox pattern ข้อสำคัญคือการบันทึกผลกับการบันทึกคำสั่งในคิวต้องอะตอมิกด้วยกัน มิฉะนั้นอาจเกิดกรณีที่สต็อกถูกตัดแล้วแต่ไม่มีคำสั่งให้ส่ง หรือมีคำสั่งแต่ผลในเครื่องไม่ตรงกัน แต่ละแถวของคิวมีสถานะ เช่น รอส่ง ติดขัด หรือถูกปฏิเสธ และฝั่งรับต้องรองรับการได้รับคำสั่งเดิมซ้ำ ซึ่งเชื่อมโยงกับแนวคิด idempotency ในหัวข้อถัดไป

2.3.4 ความสอดคล้องในที่สุด

ระบบกระจายที่ยอมให้สำเนาต่างกันชั่วคราวเรียกว่ามีความสอดคล้องในที่สุด (eventual consistency) คือหากไม่มีการเขียนใหม่ สำเนาทั้งหมดจะลู่เข้าสู่ค่าเดียวกัน [17] ในระบบแบบนี้ยอดสต็อกและยอดลูกค้าในเครื่องขณะออฟไลน์เป็นค่าประมาณจากข้อมูลล่าสุดที่ดึงมาบวกด้วยผลของคำสั่งที่ค้างในคิว เมื่อซิงก์สำเร็จจึงถูกแทนด้วยค่าจากเซิร์ฟเวอร์ ข้อควรระวังคือการดึงข้อมูลจากเซิร์ฟเวอร์ต้องไม่ทับค่าในเครื่องที่ยังมีคำสั่งค้างอ้างอิงอยู่ มิฉะนั้นยอดจะกระโดดกลับก่อนคำสั่งถูกส่ง

2.3.5 ความล้มเหลวระหว่างทางและผลลัพธ์ที่ไม่ทราบแน่ชัด

การเรียกผ่านเครือข่ายล้มได้สามรูปแบบที่ต่างกัน คือ ส่งไม่ถึง ส่งถึงแต่คำตอบไม่กลับ และเซิร์ฟเวอร์ตอบข้อผิดพลาด รูปแบบที่สองอันตรายที่สุด เพราะผู้ส่งไม่ทราบว่าการเขียนสำเร็จหรือไม่ หากผู้ส่งตีความว่าล้มเหลวแล้วเขียนซ้ำหรือเขียนลงเครื่องแทน อาจตัดสต็อกสองครั้ง ไคลเอนต์จึงต้องแยกคำตอบที่เป็น "คำตัดสิน" ของเซิร์ฟเวอร์ (เช่น รหัส 4xx ที่บอกว่าคำขอผิด) ออกจากคำตอบที่บอกเพียงว่าเซิร์ฟเวอร์มีปัญหา (5xx) หรือไม่มีคำตอบเลย

2.4 เทคโนโลยีฝั่งไคลเอนต์

2.4.1 Flutter และ Dart

Flutter เป็นชุดเครื่องมือสร้างส่วนติดต่อผู้ใช้ของ Google ที่คอมไพล์รหัสเดียวไปยังแอป Android, iOS และเว็บ [4] เขียนด้วยภาษา Dart ซึ่งเป็นภาษาแบบมีชนิดข้อมูล รองรับการทำงานแบบอะซิงโครนัสด้วย `Future` และ `Stream` ส่วนติดต่อผู้ใช้ประกอบจากวิดเจ็ต (widget) ที่ไม่เปลี่ยนค่า และเฟรมเวิร์กสร้างต้นไม้วิดเจ็ตใหม่เมื่อสถานะเปลี่ยน โครงงานเลือก Flutter เพราะต้องการฐานรหัสเดียวสำหรับเคาน์เตอร์ที่อาจเป็นแท็บเล็ตหรือเว็บเบราว์เซอร์

2.4.2 SQLite, Drift และการรันบนเว็บ

SQLite เป็นฐานข้อมูลเชิงสัมพันธ์แบบฝังตัวในไฟล์เดียว รองรับธุรกรรมแบบ ACID (ความเป็นอะตอม ความสอดคล้อง การแยกจากกัน และความคงทน ดูหัวข้อคุณสมบัติ ACID ในบทนี้) [6] Drift เป็นไลบรารีแมปวัตถุกับฐานข้อมูล (Object-Relational Mapping: ORM) แบบมีชนิดข้อมูลสำหรับ Dart ที่สร้างรหัสจากการประกาศตาราง และเรียกใช้ SQLite ได้ทั้งบนอุปกรณ์เคลื่อนที่และเว็บ [5] บนเว็บ Drift ใช้ SQLite ที่คอมไพล์เป็น WebAssembly ทำงานร่วมกับ web worker และเลือกที่เก็บตามความสามารถของเบราว์เซอร์ เช่น Origin Private File System (OPFS) หรือ IndexedDB

2.4.3 การจัดการสถานะด้วย BLoC และการนำทางด้วย go_router

การจัดการสถานะ (state management) คือการกำหนดว่าข้อมูลที่หน้าจอใช้เก็บที่ใดและเปลี่ยนแปลงอย่างไร รูปแบบ BLoC (Business Logic Component) แยกตรรกะธุรกิจออกจากวิดเจ็ต โดยรับเหตุการณ์และปล่อยสถานะเป็นสตรีม Cubit เป็นรูปแบบย่อยที่เรียกเมธอดแล้วปล่อยสถานะโดยตรง ไลบรารี `flutter_bloc` มี `BlocProvider` และ `RepositoryProvider` สำหรับฉีดการพึ่งพา (dependency injection) ลงต้นไม้วิดเจ็ต [18] การฉีด repository ผ่าน provider ทำให้สลับ implementation ได้โดยหน้าจอไม่ต้องแก้ ส่วน `go_router` เป็นไลบรารีการนำทางแบบประกาศ (declarative routing) ที่แมป URL ไปยังหน้าจอ และมี `ShellRoute` ที่ห่อหน้าจอย่อยด้วยโครงร่างร่วม [19]

2.5 การสื่อสารไคลเอนต์-เซิร์ฟเวอร์

REST (Representational State Transfer) เป็นรูปแบบสถาปัตยกรรมที่ใช้ทรัพยากรที่ระบุด้วย URL และเมธอด HTTP เช่น GET, POST, PATCH ตามความหมายของแต่ละเมธอด โดยเซิร์ฟเวอร์ไม่เก็บสถานะของเซสชันระหว่างคำขอ ทำให้เพิ่มจำนวนอินสแตนซ์ของเซิร์ฟเวอร์ได้ตรงไปตรงมา การห่อผลลัพธ์ด้วยโครงสร้างคงที่ (envelope) ที่มีรหัสข้อผิดพลาดแบบข้อความ ช่วยให้ไคลเอนต์ตัดสินได้ว่าข้อผิดพลาดใดเป็นคำตัดสินและแปลเป็นข้อความสำหรับผู้ใช้ได้ ส่วนจำนวนเงินควรส่งเป็นสตริงทศนิยมเพื่อเลี่ยงความคลาดเคลื่อนของเลขทศนิยมแบบลอยตัว

แคชแบบเขียนผ่าน (write-through cache) คือรูปแบบที่การเขียนทุกครั้งไปยังแหล่งความจริงก่อน แล้วจึงนำผลที่แหล่งความจริงตอบกลับมาปรับแคช แคชจึงไม่มีข้อมูลที่แหล่งความจริงไม่ได้ยอมรับ รูปแบบนี้คือแนวทางที่ ADR-0010 ของโครงงานใช้กับฐานข้อมูล Drift ในเครื่อง

2.6 ธุรกรรมฐานข้อมูลและการควบคุมภาวะพร้อมกัน

2.6.1 คุณสมบัติ ACID และระดับการแยกจากกัน

ธุรกรรม (transaction) คือหน่วยงานที่ฐานข้อมูลรับประกันคุณสมบัติ ACID ได้แก่ ความเป็นอะตอม (Atomicity) คือทำครบหรือไม่ทำเลย ความสอดคล้อง (Consistency) คือข้อจำกัดของข้อมูลยังคงอยู่หลังธุรกรรม การแยกจากกัน (Isolation) คือธุรกรรมที่ทำพร้อมกันไม่เห็นผลครึ่งทางของกันและกัน และความคงทน (Durability) คือเมื่อยืนยัน (commit) แล้วข้อมูลไม่สูญหาย [20] มาตรฐาน SQL นิยามระดับการแยกจากกันตามความผิดปกติที่ยอมให้เกิด เช่น dirty read, non-repeatable read และ phantom read PostgreSQL ใช้ระดับ Read Committed เป็นค่าเริ่มต้น ซึ่งแต่ละคำสั่งเห็นข้อมูลที่ยืนยันแล้วในขณะเริ่มคำสั่งนั้น [20] ในระดับนี้ การอ่านค่าแล้วคำนวณแล้วเขียนกลับ

(read-modify-write) เช่นอ่านสต็อกแล้วลบแล้วเขียน อาจเสียการปรับปรุงของธุรกรรมอื่นที่แทรกกลาง (lost update) เว้นแต่จะล็อกแถวไว้ก่อน

2.6.2 การล็อกระดับแถว เดดล็อก และลำดับการล็อก

PostgreSQL ใช้ MVCC (Multi-Version Concurrency Control) ทำให้ผู้อ่านไม่บล็อกผู้เขียน แต่ผู้เขียนที่แก้แถวเดียวกันต้องรอกัน เมื่อต้องการกันธุรกรรมอื่นแก้แถวระหว่างที่ตัวเองตรวจเงื่อนไขอยู่ ใช้ `SELECT … FOR UPDATE` เพื่อล็อกแถวแบบเขียน หรือ `FOR SHARE` เพื่อล็อกแบบแบ่งปัน [20] เดดล็อก (deadlock) เกิดเมื่อธุรกรรมสองรายการถือล็อกที่อีกฝ่ายต้องการและรอกันเป็นวงกลม PostgreSQL ตรวจพบแล้วยกเลิกธุรกรรมหนึ่งรายการ [20] วิธีป้องกันที่เชื่อถือได้คือกำหนดลำดับการล็อกให้ตรงกันทุกเส้นทางโค้ด ถ้าทุกธุรกรรมล็อกทรัพยากรตามลำดับเดียวกัน วงกลมจะไม่เกิด

2.6.3 ธุรกรรมกับกลุ่มการเชื่อมต่อ

การเชื่อมต่อฐานข้อมูลเป็นทรัพยากรจำกัดที่จัดสรรผ่านกลุ่มการเชื่อมต่อ (connection pool) หากคำขอหนึ่งถือการเชื่อมต่อแรกไว้แล้วขอการเชื่อมต่อที่สองจาก pool เดียวกัน เมื่อจำนวนคำขอพร้อมกันเท่ากับขนาด pool ทุกคำขอจะรอกันเองโดยไม่มีใครได้ไปต่อ (pool starvation) นอกจากนี้ธุรกรรมที่ยาวเกินจำเป็นจะถือทั้งการเชื่อมต่อและล็อกไว้นาน งานที่ช้าแต่ไม่ต้องอยู่ในธุรกรรม เช่น การแฮชรหัสผ่าน จึงควรทำนอกธุรกรรม

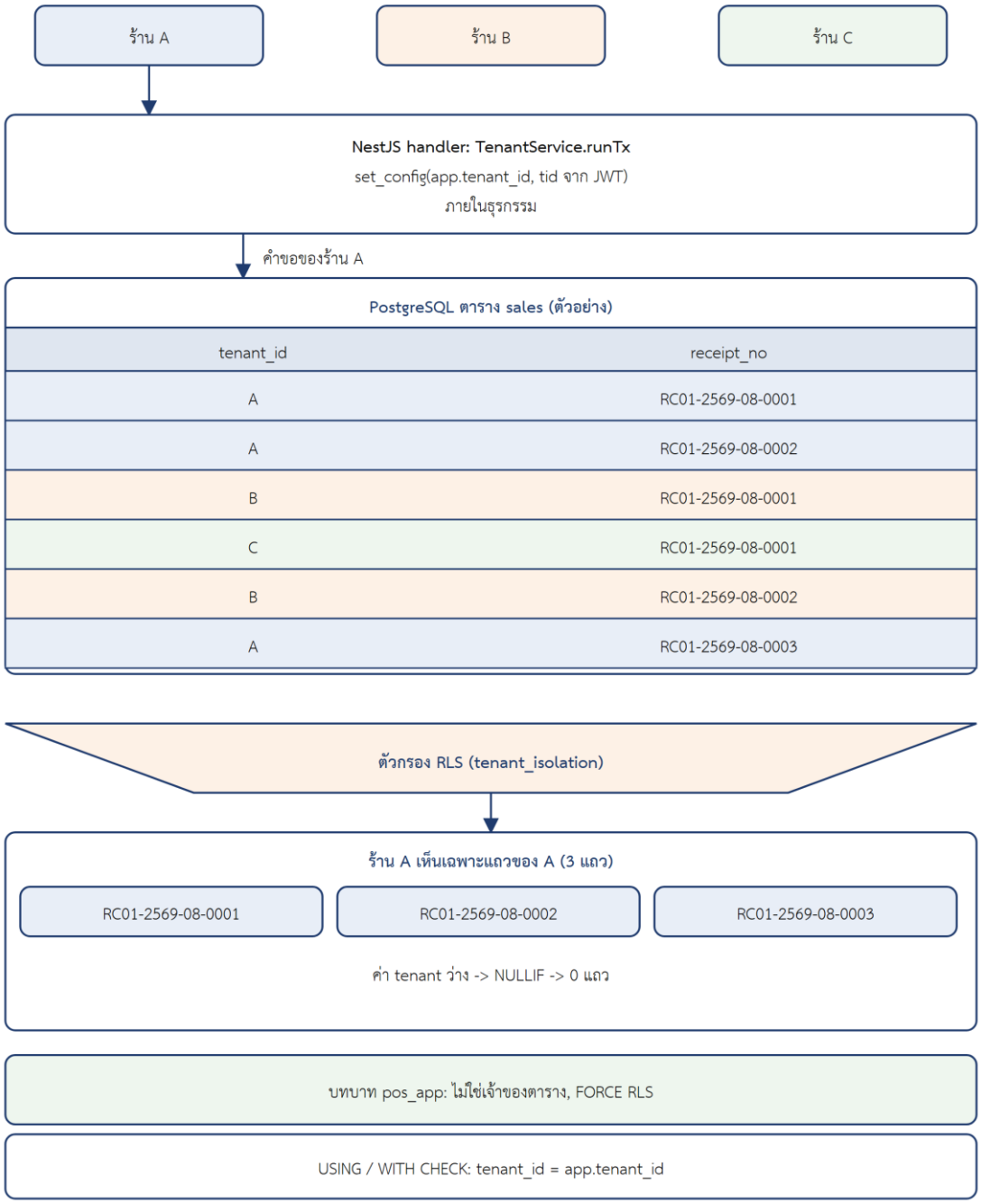
2.7 การแยกข้อมูลผู้เช่าหลายราย (multi-tenancy)

ระบบหลายผู้เช่า (multi-tenant) ให้องค์กรหลายราย (ผู้เช่า) ใช้ระบบเดียวกันโดยข้อมูลต้องไม่ปะปนกัน มีแนวทางหลักสามแบบที่ต่างกันที่ระดับการแยกและต้นทุนการดูแล ดังตารางที่ 2.1

ตารางที่ 2.1 เปรียบเทียบแบบจำลองการแยกผู้เช่าในฐานข้อมูล

แบบจำลอง	การแยกข้อมูล	ต้นทุนการดูแล	ข้อสังเกตต่อโครงงาน
ฐานข้อมูลแยกต่อผู้เช่า	แยกสูงสุดระดับฐานข้อมูล	สูง: ต้องจัดการหลายฐานข้อมูล ย้าย schema หลายชุด	ไม่เหมาะกับทรัพยากรของเครื่องสาธิตเครื่องเดียว
Schema แยกต่อผู้เช่า	แยกระดับ schema ในฐานข้อมูลเดียว	ปานกลาง: migration ต้องทำซ้ำทุก schema	จำนวนออบเจ็กต์โตตามจำนวนผู้เช่า
ตารางร่วมกับคอลัมน์ `tenant_id`	แยกระดับแถว ต้องบังคับด้วยกติกา	ต่ำ: schema เดียว migration ครั้งเดียว	โครงงานเลือกแบบนี้ร่วมกับ Row-Level Security

ความเสี่ยงของแบบที่สามคือ หากโค้ดลืมใส่เงื่อนไข `tenant_id` ในคำสั่งใดคำสั่งหนึ่ง ข้อมูลของผู้เช่าอื่นจะรั่วไหล จึงต้องมีกลไกที่ไม่พึ่งความระมัดระวังของนักพัฒนา PostgreSQL มีคุณสมบัติ Row-Level Security (RLS) ที่ให้กำหนดนโยบาย (policy) ต่อตาราง ซึ่งฐานข้อมูลจะเติมเป็นเงื่อนไขกรองแถวให้ทุกคำสั่งโดยอัตโนมัติ [10] นโยบายมีสองส่วน คือ `USING` ที่กรองแถวที่มองเห็นหรือแก้ได้ และ `WITH CHECK` ที่ตรวจแถวที่กำลังจะเขียน โดยทั่วไปนโยบายเทียบ `tenant_id` กับตัวแปรเซสชันที่แอปตั้งไว้ในธุรกรรม ดังรูปที่ 2.1 ข้อควรระวังคือเจ้าของตารางและผู้ใช้ระดับ superuser ได้รับการยกเว้นจาก RLS โดยปริยาย แอปจึงต้องเชื่อมต่อด้วยบทบาทที่ไม่ใช่เจ้าของตาราง หรือบังคับด้วย `FORCE ROW LEVEL SECURITY`



รูปที่ 2.1 แนวคิด Row-Level Security: คำขอของผู้เช่าหนึ่งเห็นเฉพาะแถวของผู้เช่านั้น

ผู้ดูแลแพลตฟอร์มที่สร้างและดูแลผู้เช่าหลายรายต้องข้ามขอบเขตผู้เช่าได้ แนวทางที่ปลอดภัยคือแยก "ระนาบผู้ดูแล" (admin plane) ออกจากระนาบผู้เช่าทั้งในระดับบัญชี โทเค็น และเส้นทางเครือข่าย เพื่อไม่ให้ความผิดพลาดของฝั่งหนึ่งเปิดทางเข้าอีกฝั่ง

2.8 การทำซ้ำคำขออย่างปลอดภัยและรหัสกำกับการเขียน (idempotency)

การดำเนินการเป็น idempotent เมื่อทำซ้ำกี่ครั้งก็ให้ผลต่อสถานะเท่ากับทำครั้งเดียว ใน HTTP เมธอด GET, HEAD, PUT, DELETE ถูกนิยามให้ idempotent แต่ POST ซึ่งใช้สร้างบิลไม่เป็น [21] เมื่อไคลเอนต์ส่ง POST แล้วเครือข่ายหมดเวลา ไคลเอนต์ไม่ทราบว่าเซิร์ฟเวอร์ทำสำเร็จหรือไม่ การส่งซ้ำอาจทำให้ขายซ้ำสองบิล แนวทางที่กำลังถูกกำหนดเป็นมาตรฐานคือให้ไคลเอนต์สร้างรหัสสุ่มหนึ่งค่าต่อหนึ่งเจตนา แนบในส่วนหัว `Idempotency-Key` [22] เซิร์ฟเวอร์บันทึกคีย์พร้อมค่าสรุปของคำขอ (request fingerprint) และผลลัพธ์ หากได้รับรหัสเดิมกับคำขอเดิมจะตอบผลเดิมโดยไม่ทำซ้ำ หากรหัสเดิมมากับคำขอที่ต่างไปจะปฏิเสธ กลไกนี้ได้ผลก็ต่อเมื่อไคลเอนต์ใช้รหัสเดิมทุกครั้งที่ลองส่งเจตนาเดิมซ้ำ และการจดรหัสอยู่ในธุรกรรมเดียวกับการเขียน

2.9 การยืนยันตัวตนและการรักษาความปลอดภัย

2.9.1 JSON Web Token และอายุของโทเค็น

JWT ตามมาตรฐาน RFC 7519 [23] เป็นรูปแบบโทเค็นแบบกะทัดรัดที่ประกอบด้วยส่วนหัว เพย์โหลดที่มี claim เช่น `iss`, `sub`, `exp`, `aud` และลายเซ็น ผู้รับตรวจลายเซ็นด้วยกุญแจโดยไม่ต้องถามฐานข้อมูล การเซ็นด้วยกุญแจอสมมาตร เช่น RS256 ทำให้ผู้ตรวจถือเพียงกุญแจสาธารณะ ระบบทั่วไปแยกโทเค็นเข้าถึง (access token) อายุสั้นเพื่อจำกัดความเสียหายเมื่อรั่ว ออกจากโทเค็นต่ออายุ (refresh token) ที่อายุยาวกว่าและใช้ขอ access token ใหม่ การเพิกถอนทำได้ที่จังหวะต่ออายุ ซึ่งตรวจฐานข้อมูลอีกครั้ง บนเว็บ โทเค็นที่เก็บใน `localStorage` อ่านได้จากสคริปต์ใด ๆ ในหน้า จึงเสี่ยงต่อการโจมตีแบบ Cross-Site Scripting (XSS)

2.9.2 การแฮชรหัสผ่านด้วย Argon2

รหัสผ่านต้องไม่เก็บเป็นข้อความตรง แต่เก็บค่าแฮชจากฟังก์ชันที่ออกแบบให้ช้าและใช้หน่วยความจำมาก เพื่อต้านการเดาแบบเหมารวม Argon2 ผู้ชนะการแข่งขัน Password Hashing Competition ถูกกำหนดใน RFC 9106 [24] มีพารามิเตอร์จำนวนหน่วยความจำ จำนวนรอบ และระดับขนาน ข้อควรระวังสำหรับข้อความภาษาไทยคือสตริงเดียวกันอาจเข้ารหัสยูนิโคดได้หลายรูป การทำให้เป็นรูปมาตรฐาน (normalization) ก่อนแฮชจึงมีผลต่อการตรวจรหัสผ่าน

2.9.3 การผูกอุปกรณ์และการจำกัดอัตราคำขอ

ระบบที่ต้องจำกัดสิทธิ์ตามเครื่อง เช่น ให้มีเครื่องขายเพียงเครื่องเดียวต่อร้าน มักใช้รหัสลงทะเบียนแบบใช้ครั้งเดียวที่ผู้มีสิทธิ์ออกให้ แล้วนำไปแลกเป็นโทเค็นของอุปกรณ์ ทำให้ตัวตนของ

เครื่องมาจากเซิร์ฟเวอร์ ไม่ใช่จากค่าที่ไคลเอนต์ส่งเอง ส่วนการจำกัดอัตราคำขอ (rate limiting) ช่วยลดการเดารหัสผ่านและการใช้ทรัพยากรเกินควร มักทำหลายชั้น เช่น ต่อ IP ที่พร็อกซีหน้าระบบ และต่อผู้เช่าในแอปพลิเคชัน

2.10 เทคโนโลยีฝั่งเซิร์ฟเวอร์

NestJS เป็นเฟรมเวิร์กฝั่งเซิร์ฟเวอร์บน Node.js ที่เขียนด้วย TypeScript จัดโครงสร้างเป็นโมดูล คอนโทรลเลอร์ และผู้ให้บริการ (provider) พร้อมระบบฉีดการพึ่งพา (dependency injection) และกลไก guard, interceptor และ middleware [25] โครงสร้างที่บังคับให้แยกชั้นชัดเจนเหมาะกับทีมสามคนที่แบ่งงานตามโดเมน PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์โอเพนซอร์สที่รองรับ ACID, MVCC, RLS, ข้อจำกัดเชิงซ้อน และดัชนีบางส่วน [20]

Redis เป็นที่เก็บข้อมูลแบบคีย์–ค่า (key-value store) ในหน่วยความจำ ใช้เป็นแคช ตัวนับ และคิว [26] BullMQ เป็นไลบรารีคิวงานบน Redis ที่มีการลองซ้ำและตารางเวลา [27] เหมาะกับงานเบื้องหลังที่ไม่ควรทำในคำขอ HTTP เช่น การส่งออกข้อมูล ข้อควรสังเกตคือแคชกับคิวต้องการนโยบายหน่วยความจำที่ตรงข้ามกัน แคชทิ้งข้อมูลได้เมื่อเต็ม แต่คิวห้ามทิ้งงานที่ยังไม่ทำ

พร็อกซีย้อนกลับ (reverse proxy) รับคำขอจากภายนอกแล้วส่งต่อไปยังเซิร์ฟเวอร์ภายใน ทำหน้าที่ยุติ Transport Layer Security (TLS) จำกัดอัตรา และซ่อนโครงสร้างภายใน Nginx ทำหน้าที่นี้ได้ด้วยคำสั่ง `proxy_pass` และ `limit_req_zone` [28] เมื่อมีพร็อกซี แอปจะอ่าน IP ผู้เรียกจากส่วนหัว `X-Forwarded-For` จำนวนชั้นของพร็อกซีจึงต้องแน่นอน หากมีพร็อกซีหรือ Content Delivery Network (CDN) ซ้อนเพิ่ม ผู้ใช้ทุกคนอาจถูกมองเป็น IP เดียว

2.11 คอนเทนเนอร์และการจัดวาง

คอนเทนเนอร์เป็นการแยกโปรเซสด้วยฟีเจอร์ของเคอร์เนล (namespace และ cgroup) ที่บรรจุแอปพร้อมไลบรารีที่ต้องใช้เป็นอิมเมจ ทำให้รันได้เหมือนกันทุกเครื่อง [29] Docker Compose ประกาศชุดบริการหลายตัว เครือข่าย และเพดานทรัพยากรในไฟล์เดียว [30] การตรึงอิมเมจด้วย digest แทนป้ายชื่อทำให้สร้างสภาพแวดล้อมเดิมซ้ำได้ เพราะป้ายชื่ออย่าง `latest` อาจชี้ไปยังอิมเมจคนละตัวในเวลาต่างกัน

2.12 การบูรณาการและการส่งมอบอย่างต่อเนื่อง (CI/CD)

การบูรณาการต่อเนื่อง (Continuous Integration: CI) คือการรวมโค้ดเข้าสาขาหลักบ่อยครั้งพร้อมสร้างและทดสอบอัตโนมัติ เพื่อพบข้อผิดพลาดแต่เนิ่น การส่งมอบต่อเนื่อง

(Continuous Delivery: CD) ต่อยอดโดยทำให้ผลงานที่ผ่านการทดสอบพร้อมนำขึ้นใช้งาน (deploy) ได้ตลอดเวลา ไปป์ไลน์ที่ดีให้ผลลัพธ์ที่ทำซ้ำได้และมีประตูตรวจคุณภาพก่อนถึงสภาพแวดล้อมจริง

GitHub Actions เป็นบริการรันเวิร์กโฟลว์ที่นิยามเป็น YAML ประกอบด้วยงาน (job) ที่ทำงานบนตัวรัน (runner) ทั้งแบบที่ GitHub จัดให้และแบบติดตั้งเอง (self-hosted) [31] สาขาหลักสามารถบังคับให้ต้องผ่านการตรวจสอบสถานะ (status check) ที่กำหนดก่อนรวมโค้ด ซึ่งมีเงื่อนไขว่าการตรวจสอบนั้นต้องรายงานผลทุกครั้ง GitHub Container Registry (GHCR) เป็นที่เก็บอิมเมจคอนเทนเนอร์ [32] Trivy เป็นเครื่องมือสแกนช่องโหว่ของอิมเมจและการพึ่งพา [33] และ gitleaks ตรวจหาความลับที่เผลอคอมมิต เช่น คีย์ API [34] เมื่อพบความลับจริงในประวัติ วิธีที่ถูกต้องคือหมุนเปลี่ยนความลับนั้น ไม่ใช่ใส่ไว้ในรายการยกเว้น

Ansible เป็นเครื่องมือจัดการคอนฟิกที่สั่งงานเครื่องปลายทางผ่าน Secure Shell (SSH) ด้วย playbook และโมดูลส่วนใหญ่ออกแบบให้ idempotent [35] เมื่อเครื่องปลายทางอยู่ในเครือข่ายปิดที่ตัวรันของ GitHub เข้าไม่ถึง ต้องใช้ตัวรันแบบติดตั้งเองภายในเครือข่ายนั้น และควรมีประตูอนุมัติโดยผู้ตรวจที่กำหนดก่อนงานติดตั้งเริ่ม

2.13 การสังเกตการณ์ระบบ (observability)

การสังเกตการณ์คือความสามารถในการอนุมานสถานะภายในของระบบจากข้อมูลที่ระบบปล่อยออกมา โดยทั่วไปประกอบด้วยตัวชี้วัด (metrics) บันทึก (logs) และร่องรอย (traces) Prometheus เก็บอนุกรมเวลาด้วยการดึง (scrape) จาก endpoint ที่แสดงตัวชี้วัดในรูปแบบข้อความ และมีภาษาสืบค้น PromQL [36] Grafana แสดงผลเป็นแดชบอร์ด [37] ป้ายกำกับ (label) ของตัวชี้วัดควรมีค่าที่เป็นไปได้จำนวนจำกัด เช่น ใช้รูปแบบเส้นทางแทนเส้นทางจริง มิฉะนั้นจำนวนอนุกรม (cardinality) จะพุ่งสูง

ตัวชี้วัดระดับบริการ (Service Level Indicator: SLI) คือการวัดเชิงปริมาณของพฤติกรรมบริการ เช่น สัดส่วนคำขอที่สำเร็จ และเปอร์เซ็นไทล์ที่ 95 (p95) ของเวลาตอบ การเลือกว่าคำขอใดถูกนับมีความสำคัญเท่ากับการนับ เพราะคำขอที่ระบบสร้างเอง เช่น การตรวจสุขภาพ อาจทำให้อัตราสำเร็จดูดีเกินจริง ส่วนการทดสอบโหลดใช้เครื่องมือเช่น k6 ที่เขียนสถานการณ์เป็นสคริปต์ JavaScript เพื่อวัดพฤติกรรมของระบบภายใต้ปริมาณคำขอที่กำหนด

2.14 การบันทึกการตัดสินใจเชิงสถาปัตยกรรม (ADR)

Nygard [38] เสนอให้บันทึกการตัดสินใจทางสถาปัตยกรรมเป็นเอกสารสั้นหนึ่งฉบับต่อหนึ่งการตัดสินใจ ประกอบด้วยบริบท การตัดสินใจ สถานะ และผลที่ตามมา เหตุผลคือโค้ดบอกว่าระบบทำอะไร แต่ไม่บอกว่าทำไมจึงเลือกทางนั้นและทางใดถูกปฏิเสธ เมื่อสมาชิกเปลี่ยนหรือเวลาผ่านไป

ความรู้นี้จะหายถ้าไม่จด ADR ที่ปฏิเสธข้อเสนอพร้อมเหตุผลมีประโยชน์เท่ากับ ADR ที่ยอมรับ เพราะป้องกันการถกเถียงเดิมซ้ำ

2.15 สรุป

บทนี้วางแนวคิดที่ใช้ในบทถัดไป ได้แก่ กติกาโดเมน POS ที่ต้องอะตอมิก offline-first ด้วยกล่องขาออกและเครื่องเขียนเครื่องเดียว ลำดับการล็อก RLS รหัสกำกับการเขียน การยืนยันตัวตน และ CI/CD พร้อมการสังเกตการณ์

บทที่ 3
รายละเอียดการดำเนินงาน

3.1 ภาพรวมของโครงงาน

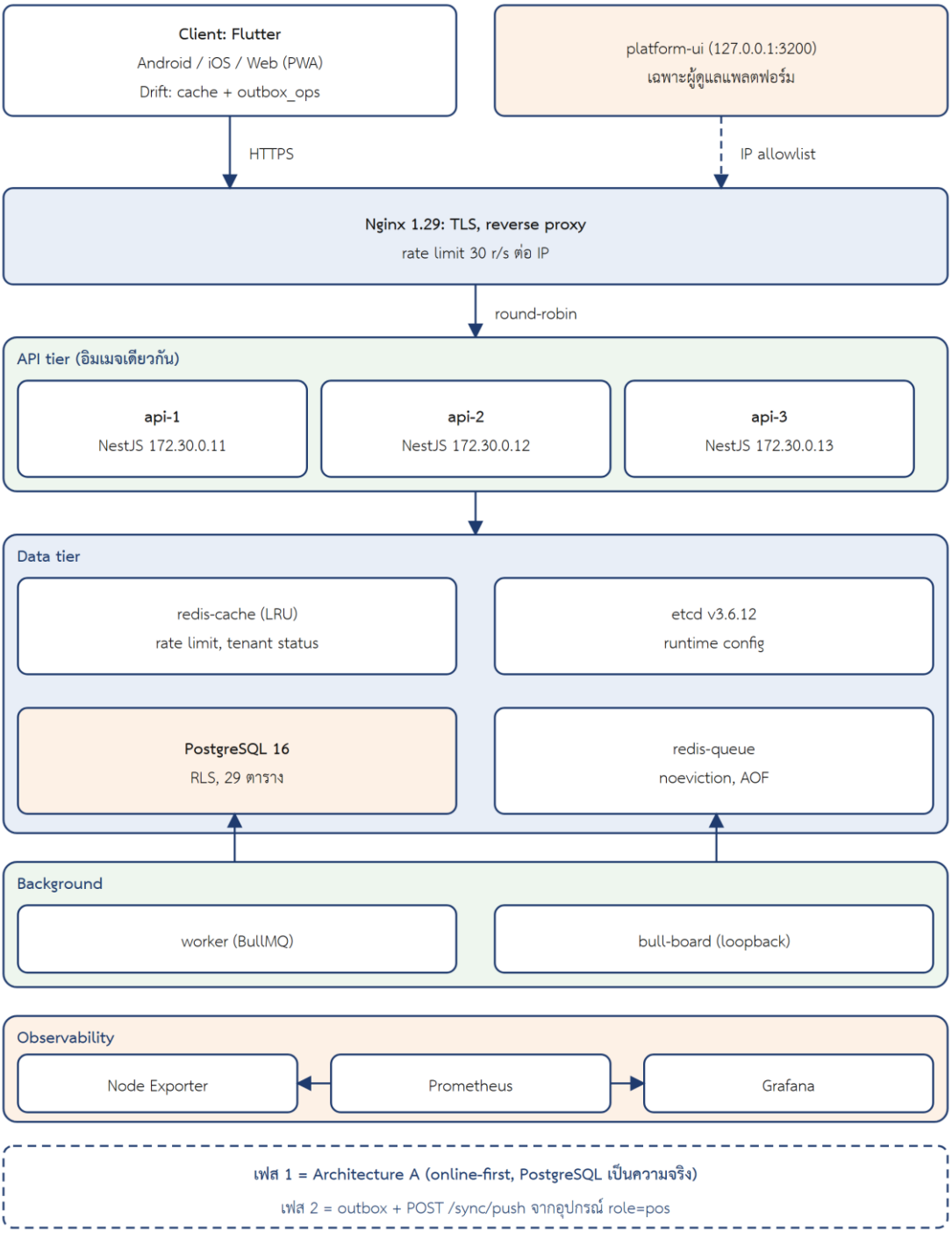
บทนี้อธิบายการออกแบบและการสร้างระบบ Srisurart Autopart POS ตามสถานะของโค้ดบนสาขา `main` ณ วันที่ 1 ตุลาคม พ.ศ. 2569 ได้แก่ สถาปัตยกรรม ส่วนประกอบ ฝั่งไคลเอนต์ ฝั่งเซิร์ฟเวอร์ การซิงก์ข้อมูลในเฟส 2 ความปลอดภัย การนำระบบขึ้นใช้งาน (deploy) เทคโนโลยีที่ใช้ และวิธีการทำงานของทีม การตัดสินใจที่สำคัญอ้างอิงบันทึกการตัดสินใจเชิงสถาปัตยกรรม (Architecture Decision Record: ADR) ในโฟลเดอร์ `docs/Backend_design/adr/` เพื่อบอกเหตุผลของการเลือก ส่วนผลที่ได้และหลักฐานรายงานไว้ในบทที่ 4

3.1.1 หลักการของสถาปัตยกรรม

ตามที่กล่าวในบทที่ 1 โครงงานเลือก Architecture C (ไฮบริด) ร่วมกับโมเดลผู้เช่าแบบ T1 (ฐานข้อมูลและสคีมาเดียว คอลัมน์ `tenant_id` และ Row-Level Security) โดยเฟส 1 คือ Architecture A หรือ online-first modular monolith ที่ให้ PostgreSQL เป็นแหล่งความจริงเพียงแหล่งเดียว และเฟส 2 เพิ่มความสามารถทำงานออฟไลน์กลับเข้ามา เอกสาร `03_ARCHITECTURE.md` ระบุว่า หากเวลาไม่พอ ให้ทำ A กับ T1 ให้สมบูรณ์ดีกว่าทำส่วนเสริมไม่ครบ หลักการนี้กำหนดลำดับงานของทีม คือเฟส 1 ต้องทำงานได้จริงและทดสอบได้ก่อน ทั้งไคลเอนต์ เซิร์ฟเวอร์ และไปป์ไลน์อยู่ในที่เก็บโค้ดเดียว (monorepo) ตาม ADR-0011 เพื่อให้การแก้ API หนึ่งครั้งแก้ได้ทั้งสองฝั่งในคอมมิตเดียว

3.1.2 สถาปัตยกรรมโดยรวม

ไคลเอนต์ Flutter (สร้างและทดสอบบนเว็บ; เวิร์กโฟลว์สร้างไฟล์ติดตั้ง Android เพิ่มเมื่อ 3 ตุลาคม พ.ศ. 2569 และสร้างไฟล์ติดตั้งได้แล้ว แต่ยังไม่มีบันทึกการทดสอบบนอุปกรณ์จริง) สื่อสารกับเซิร์ฟเวอร์ผ่าน HTTPS ทุกคำขอเข้าสู่ Nginx ซึ่งเป็น reverse proxy ตัวเดียวหน้าระบบ แล้วกระจายไปยังอินสแตนซ์ NestJS [25] สามตัว (`api-1` ถึง `api-3`) ที่ใช้อิมเมจเดียวกัน ข้อมูลธุรกิจเก็บใน PostgreSQL ส่วน Redis แบ่งเป็นสองอินสแตนซ์ตามหน้าที่ งานเบื้องหลังประมวลผลโดยโปรเซส `worker` แยกต่างหาก ค่าตั้งที่ปรับขณะรันได้เก็บใน etcd และการเฝ้าสังเกตระบบใช้ Prometheus กับ Grafana ดังรูปที่ 3.1 ตารางที่ 3.1 สรุปความต่างระหว่างเฟส 1 และเฟส 2 ในมุมที่กระทบการออกแบบ ตามเอกสาร `03_ARCHITECTURE.md` และ `08_PHASE2_SPEC.md`



รูปที่ 3.1 สถาปัตยกรรมโดยรวมของระบบ Srisurart Autopart POS

ตารางที่ 3.1 เปรียบเทียบเฟส 1 (Architecture A) กับเฟส 2 (offline shell)

หัวข้อ	เฟส 1	เฟส 2
แหล่งความจริงของข้อมูล	PostgreSQL	PostgreSQL (ไม่เปลี่ยน)
บทบาทของ Drift ในเครื่อง	แคชแบบ write-through ของคำตอบจากเซิร์ฟเวอร์	แคช และเป็นที่เก็บคิว `outbox_ops` ตอนออฟไลน์
การเขียนข้อมูลเมื่อไม่มีเครือข่าย	ไม่รองรับ ยกเว้นคิวรับชำระเครดิตช่าง (#24) และบิลด์ offline-first เดิม	เครื่อง `pos` เข้าคิวได้ตามรายการคำสั่ง (operation: op) ที่กำหนด
การออกเลขใบเสร็จ RC และใบลดหนี้ CN	เซิร์ฟเวอร์ออกให้	เครื่อง `pos` ออกเองทั้งออนไลน์และออฟไลน์
การส่งข้อมูลกลับ	เรียก endpoint ออนไลน์โดยตรง	`POST /sync/push` ด้วย device token
สถานะของไคลเอนต์	ออนไลน์ล้วน	Online / Degraded / Syncing

3.2 ส่วนประกอบของโครงงาน

ที่เก็บโค้ดแบ่งเป็นส่วนหลักดังตารางที่ 3.2 นอกจากนี้ `docs/` เก็บบันทึกการส่งมอบงาน (`handoff_log`) และคู่มือผู้ใช้ภาษาไทย

ตารางที่ 3.2 ส่วนประกอบหลักของที่เก็บโค้ด

ส่วนประกอบ	ตำแหน่ง	หน้าที่
ไคลเอนต์ Flutter	`frontend/`	แอปหน้าร้าน (สร้างและทดสอบบนเว็บ; มีเวิร์กโฟลว์สร้างไฟล์ติดตั้ง Android ที่สร้างไฟล์ได้แล้ว; Android/iOS ยังไม่มีบันทึกการทดสอบบนอุปกรณ์จริง) ประกอบด้วยชั้น data, domain และ presentation
เซิร์ฟเวอร์ NestJS	`server/`	API, worker, Bull Board, migrations และเครื่องมือบรรทัดคำสั่งของแพลตฟอร์ม
ชุด deploy	`deploy/`	Ansible playbook, ไฟล์ Compose ส่วนเสริมสำหรับ VM, การตั้งค่า Prometheus และ Grafana, สคริปต์สำรองข้อมูลและ runner
ไปป์ไลน์	`.github/workflows/`	`flutter.yml`, `server.yml` และ `deploy.yml`
เอกสารออกแบบ	`docs/Backend_design/`	เอกสารหมายเลข 00 ถึง 09 และ ADR ที่เป็นบันทึกการตัดสินใจที่มีผลผูกพัน

บนเครื่องเสมือน ระบบถูกจัดเป็นคอนเทนเนอร์ภายใต้ Docker Compose [30] ตามตารางที่ 3.3 ค่าเพดานหน่วยความจำ (`mem_limit`) มาจากไฟล์ `server/docker-compose.yml` ซึ่งตั้งไว้เพื่อให้ระบบทั้งชุดอยู่ในทรัพยากรของเครื่องเสมือนระดับเล็กได้

ตารางที่ 3.3 คอนเทนเนอร์หลักและเพดานหน่วยความจำ

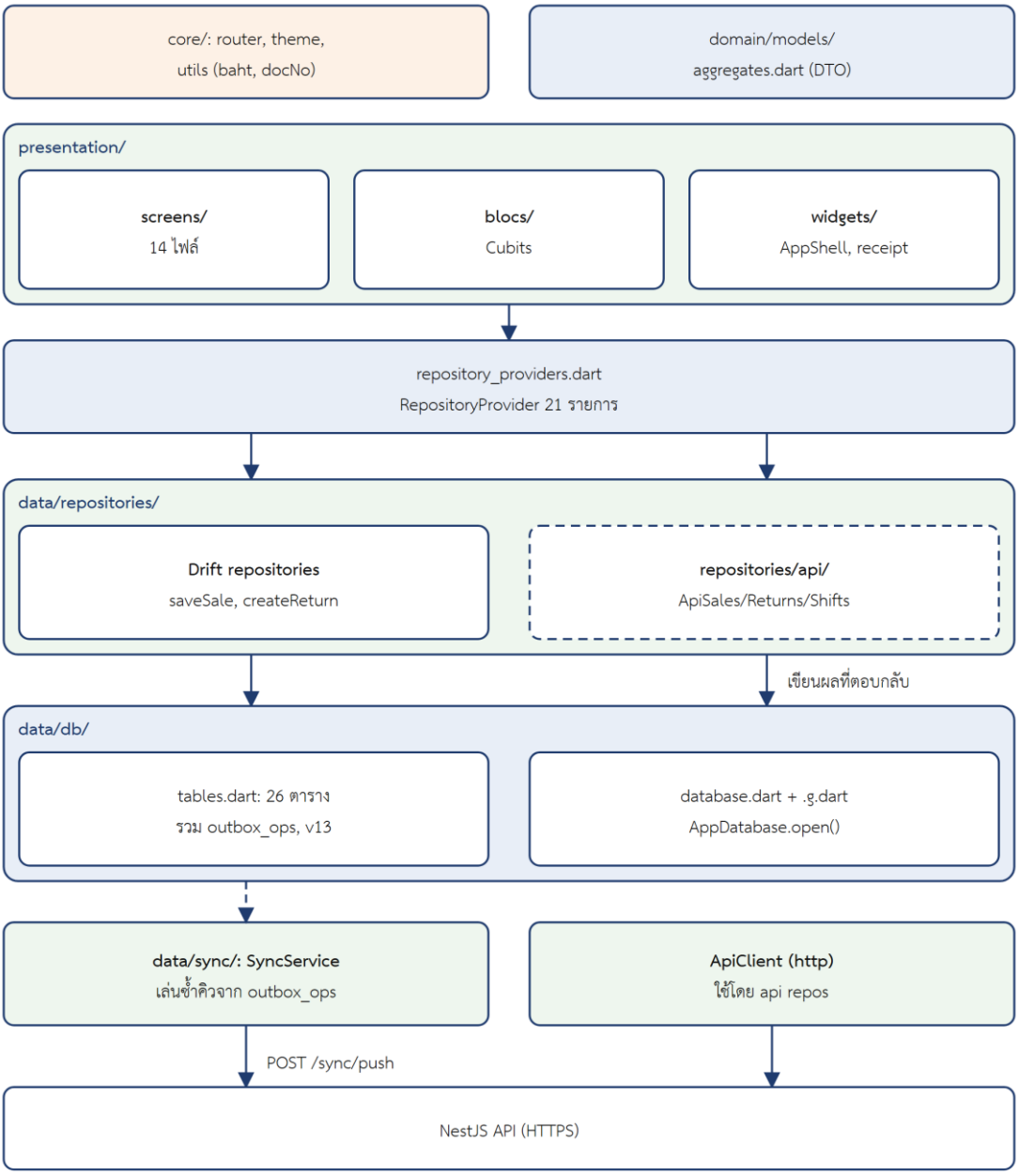
บริการ	อิมเมจ	เพดาน RAM	บทบาท
`nginx`	`nginx:1.29-alpine`	64 MB	reverse proxy, TLS (ใบรับรองจาก CA ส่วนตัวของโครงงาน), rate limit ต่อ IP, ปิดเส้นทาง `/metrics` ต่อภายนอก
`api-1`, `api-2`, `api-3`	อิมเมจของ server	384 MB ต่อตัว	NestJS API หลัง Nginx
`worker`	อิมเมจของ server	256 MB	ประมวลผลงานเบื้องหลัง เช่น export ข้อมูลรายร้าน
`bull-board`	อิมเมจของ server	128 MB	หน้าจอดูคิวงาน ผูกกับ loopback และต้องใช้ basic auth
`migrate`	อิมเมจของ server	128 MB	งานครั้งเดียวที่รัน migrations ก่อนที่ api ใด ๆ จะเริ่ม
`postgres`	`postgres:16-alpine`	1024 MB	ฐานข้อมูลหลัก ตั้ง `max_connections=100`
`redis-cache`	`redis:7-alpine`	256 MB (Redis `maxmemory` 192 MB)	`allkeys-lru`, ไม่เขียนดิสก์
`redis-queue`	`redis:7-alpine`	256 MB (Redis `maxmemory` 192 MB)	`noeviction`, `appendonly yes` สำหรับ BullMQ
`etcd`	`etcd v3.6.12`	256 MB	ค่าตั้งแบบไดนามิก (เช่น `log_level`) ไม่ใช่ข้อมูลธุรกิจ
`platform-ui`	`nginx:1.29-alpine`	32 MB	หน้าจอผู้ดูแลแพลตฟอร์ม เปิดบน `127.0.0.1:3200` เท่านั้น
Prometheus, Grafana, Node Exporter	v2.55.1, 11.2.0, v1.8.2	512, 256 และ 64 MB (ไฟล์ `monitoring.yml`)	เก็บและแสดงตัวชี้วัด

เหตุผลที่แยก Redis เป็นสองอินสแตนซ์คือนโยบายหน่วยความจำต้องตรงกันข้าม [39] ข้อมูลใน `redis-cache` ทิ้งได้เมื่อเต็ม (ตัวนับ rate limit หายแล้วนับใหม่ได้) แต่ข้อมูลใน `redis-queue` คือคิวงานที่ยังไม่ทำ จึงใช้ `noeviction` พร้อมเขียนล็อกลงดิสก์

3.3 รายละเอียดฝั่งไคลเอนต์

3.3.1 โครงสร้างแบบสามชั้น

ไคลเอนต์ Flutter [4] อยู่ในโฟลเดอร์ `frontend/lib/` และแบ่งตามหลัก data ⟶ domain ⟶ presentation ดังรูปที่ 3.2 ชั้น `core/` เก็บสิ่งที่ใช้ร่วมกัน ได้แก่ตัวกำหนดเส้นทาง `go_router` ธีมพร้อมฟอนต์ Sarabun และ Barlow และฟังก์ชันช่วยเรื่องเงินและเลขเอกสาร (`baht`, `round2`, `newId`, `docNo`, `csvSafe`) กติกาที่ใช้ทั้งโปรเจกต์คือหน้าจอต้องเรียกใช้ repository ผ่าน provider เท่านั้น ห้ามเข้าถึงฐานข้อมูลโดยตรง การจัดการสถานะใช้ `flutter_bloc` โดยมี Cubit สำหรับโหมดธีม ขนาดฟอนต์ ใบเสนอราคาที่รอ ตะกร้าสินค้า และสถานะการเข้าสู่ระบบ ส่วน repository และบริการทั้งหมดลงทะเบียนผ่านต้นไม้ `RepositoryProvider` 21 รายการ (repository 17 ตัว และบริการ `ApiClient`, `BootstrapService`, `DocCounterSeeder`, `SyncFacade`)



รูปที่ 3.2 โครงสร้างชั้นของไคลเอนต์ Flutter และเส้นทางไปยังเซิร์ฟเวอร์

3.3.2 ฐานข้อมูลภายในเครื่อง (Drift)

ฐานข้อมูลภายในเครื่องใช้ Drift [5] กำหนดไว้ใน `data/db/tables.dart` มี 26 ตาราง และ `schemaVersion` เท่ากับ 13 แบ่งเป็นสามกลุ่ม กลุ่มแรก 20 ตารางพอร์ตมาจากที่เก็บข้อมูล `sa_*` ของแอปเดิม เช่น `Products`, `Customers`, `Mechanics`, `Sales`, `SaleItems`, `Returns`, `Shifts` และ `DocCounters` กลุ่มที่สองเป็นคิวและตารางสนับสนุนการซิงก์ ได้แก่ `PendingCreditPayments` (#24) และตารางของเฟส 2

`DocCounterSeeds`, `OutboxOps`, `SyncCursors` และ `OpEffects` กลุ่มที่สามคือ `AppMeta` สำหรับค่าเมตาดาต้า เช่น รหัสผู้เช่าที่แคชนี้เป็นเจ้าของ การยกระดับสคีมาที่เกี่ยวข้องกับรายงานนี้ ได้แก่ เวอร์ชัน 7 ที่ลบคอลัมน์ `offlineOk` เวอร์ชัน 12 ที่เพิ่มดัชนี และเวอร์ชัน 13 ที่เพิ่มตาราง `op_effects` เพื่อเก็บผลต่างที่คำสั่งออฟไลน์ใช้จริง

ไฟล์ `database.g.dart` ที่ Drift สร้างอัตโนมัติถูกคอมมิตไว้ในที่เก็บโค้ด เพราะเครื่องมือ `build_runner` ทำงานบนพาธที่มีอักขระนอกชุด ASCII ไม่ได้ (รายละเอียดในบทที่ 5) และ CI มีงาน `codegen-check` ตรวจว่าไฟล์ที่สร้างตรงกับของที่คอมมิตไว้ สำหรับเว็บ ไฟล์ `web/sqlite3.wasm` และ `web/drift_worker.js` ต้องมีเวอร์ชันตรงกับแพ็กเกจ `sqlite3` และ `drift` ที่ล็อกไว้ CI จึงทำให้บิลด์ล้มเหลวเมื่อเวอร์ชันไม่ตรง เพราะความไม่ตรงกันเปลี่ยนพฤติกรรมของแกน SQLite หรือการยกเลิกงานของ worker

3.3.3 Repository ที่พอร์ตมาจาก db.js และกติกาที่คงไว้

ชั้น `data/repositories/` มี repository หนึ่งตัวต่อโดเมน กฎธุรกิจถอดมาจากไฟล์ `pos/db.js` ของแอปเดิม แต่เปลี่ยนกลไกจาก snapshot/rollback ใน JavaScript มาเป็น `transaction(() async {...})` ของ Drift ซึ่งหากเกิดข้อผิดพลาดกลางทางจะยกเลิกทั้งชุด ตารางที่ 3.4 สรุปกติกาสำคัญ

ตารางที่ 3.4 กติกาธุรกิจของ repository ที่ทำธุรกรรม

กระบวนการ	กติกาที่คงไว้
`saveSale`	ตรวจสต็อกล่วงหน้า (ข้อความผิดพลาดภาษาไทย สต็อกไม่พอ...); ตัดสต็อกแบบเข้มงวด ไม่ปัดค่าติดลบเป็นศูนย์; แต้มที่ได้ = `floor(total/10)`; อัปเดตยอดสะสมและแต้มลูกค้า รวมทั้งสถิติช่าง เมื่อชำระด้วย เครดิตช่าง
`createReturn`	กันคืนเกินจำนวน (คืนได้ไม่เกินจำนวนที่ขายลบจำนวนที่คืนไปแล้ว); ย้อนส่วนลด แต้ม และยอดช่างตามสัดส่วน; ลดยอดเครดิตเฉพาะกรณี หักจากเครดิต; ยกเลิกบิลแม่อัตโนมัติเมื่อคืนครบทั้งบิล
`receivePO`	ต้นทุนถัวเฉลี่ยถ่วงน้ำหนัก `round2((oldQty·oldCost + newQty·newCost)/total)`
`openShift`	เปิดได้หลายกะต่อวัน (แตกต่างจาก db.js โดยตั้งใจ); ถ้า `id` มีอยู่แล้วคืนกะเดิม มิฉะนั้นจัดเก็บ (archive) กะที่ยังเปิดอยู่ก่อนหน้าก่อนเปิดกะใหม่; บันทึกรายการลิ้นชักไม่ได้หลังปิดกะ
ใบเสนอราคา และบิลที่พัก	ไม่แตะสต็อกเลย
`adjustStock`	ปรับสต็อกด้วยมือ และปัดค่าติดลบเป็น 0 (ต่างจากการขาย)

การแยกให้การขาย "ตัดสต็อกแบบเข้มงวด" แต่การปรับด้วยมือ "ปัดที่ศูนย์" เป็นการแยกความหมายโดยตั้งใจ เพราะการขายที่ทำให้สต็อกติดลบหมายถึงข้อมูลคลาดเคลื่อนที่ควรยกเลิกทั้งธุรกรรม (rollback) เพื่อให้เห็นชัด ส่วนการปรับด้วยมือเป็นการแก้ตัวเลขโดยผู้ใช้ นอกจากนี้ฟังก์ชัน `planReturn()` และ `refundedQtyOf()` ในไฟล์ `return_plan.dart` เป็นกฎที่ไม่มีผลข้างเคียง (pure function) ที่ทั้ง `ReturnsRepository` และ `ApiReturnsRepository` ใช้ร่วมกัน และยอดเงินสดที่คาดหวังในลิ้นชักคำนวณจากฟังก์ชันเดียว `ShiftsRepository.drawerCash` ซึ่งนับตามกะ (เจ้าของโครงงานตัดสินเมื่อ 3 ตุลาคม พ.ศ. 2569, PR #584) ทั้งในหน้าลิ้นชักและรายงานปิดร้าน ส่วนการเลือกว่ากะใดเป็นกะปัจจุบันใช้ฟังก์ชันเดียว `currentShiftOf` (PR #592) ซึ่งถือว่ากะที่ยังเปิดอยู่เป็นกะปัจจุบันแม้ข้ามเที่ยงคืน เพื่อไม่ให้มีกติกาเดียวกันสองชุดที่เบี่ยงจากกัน ส่วน `SnapshotRepository` มี `exportSnapshot()` และ `importLegacyBackup()` ที่นำเข้าไฟล์สำรองของแอป JavaScript เดิมแบบอะตอมมิก ซึ่งเป็นเส้นทางย้ายข้อมูลของร้านจริง

3.3.4 Repository แบบ API (USE_API_WRITES)

เมื่อคอมไพล์ด้วย `--dart-define=USE_API_WRITES=true` ต้นไม้ provider จะสลับไปใช้ repository ชุดที่เรียกเซิร์ฟเวอร์ เช่น `ApiSalesRepository`, `ApiReturnsRepository` และ `ApiShiftsRepository` ตามที่ ADR-0010 กำหนดว่า `ApiRepository` คือ implementation ใหม่ของ interface เดิมที่เขียนผลที่เซิร์ฟเวอร์ตอบลง Drift (write-through) เหตุผลคือ interface เดิมอยู่ใต้หน้าจอทั้งหมด การสลับ implementation จึงไม่ต้องแก้หน้าจอ และสคีมาของ Drift กับของ PostgreSQL วิวัฒนาการแยกกันได้ กติกาของชั้นนี้มีห้าข้อ

1. ค่า `id` ของบิลและ `Idempotency-Key` สร้างครั้งเดียวต่อตะกร้า ไม่ใช่ต่อการเรียก โดย `PendingWrites` เก็บความพยายามที่ยังไม่ทราบผล มิฉะนั้นการลองใหม่จะได้รหัสใหม่และทำให้บิลซ้ำ
2. เฉพาะรหัส 4xx เท่านั้นที่เป็นคำตัดสิน ส่วน 5xx, 429 หรือคำตอบที่หายไปหมายถึงไม่ทราบว่าการเขียนสำเร็จหรือไม่ ห้ามถอยไปเขียนในเครื่อง ยกเว้นความล้มเหลวของการส่งผ่านเครือข่ายจริง (transport failure) ไฟล์ `api_repository_contract_test.dart` บังคับข้อนี้กับโค้ดทั้งโฟลเดอร์ และเมื่อวันที่ 27 กันยายน พ.ศ. 2569 เจ้าของโครงงานตัดสินเพิ่มว่า 5xx หรือ 429 ไม่เข้าคิวออฟไลน์ แต่เก็บความพยายามไว้พร้อมแสดงข้อผิดพลาด
3. `ApiRepository` ห้ามเรียกบริการธุรกรรมของ Drift เพราะจะตัดสต็อกซ้ำสองรอบ
4. `ApiException` ห้ามหลุดถึงหน้าจอ ต้องแปลงเป็นข้อความภาษาไทยผ่าน `rethrowThai` หรือ `rethrowServerRefusal`
5. ความยินยอม เช่น `overrideCreditLimit` ต้องส่งต่อจากกล่องโต้ตอบที่ผู้ใช้เห็นจริง ไม่ใช่สรุปเองจากการตรวจในเครื่องที่อาจล้าสมัย

3.3.5 หน้าจอ

ไคลเอนต์มีไฟล์หน้าจอ 14 ไฟล์ใน `presentation/screens/` โดย 12 หน้าอยู่ในเมนู AppShell อีกสองหน้าคือ `/devices` และ `/login` ดังตารางที่ 3.5 สตริงภาษาไทยในหน้าจอยึดหลัก "พฤติกรรมเหมือนเดิม" คือคัดลอกจาก `db.js` หรือไฟล์ `.jsx` ของแอปเดิมโดยไม่แปลใหม่ ส่วนสตริงใหม่ของเฟส 2 ถูกเก็บไว้ใน `02_API_SCREENS.md` ให้เจ้าของโครงงานรับรองก่อน

ตารางที่ 3.5 หน้าจอหลักของไคลเอนต์

ไฟล์	หน้าที่
checkout_screen.dart	ขายสินค้า คำนวณส่วนลดและแต้ม รับชำระ ใบเสร็จ พักบิล ราคาช่าง
products_screen.dart	จัดการสินค้า สต็อก หมวดหมู่ ซัพพลายเออร์ ป้ายราคา
purchase_orders_screen.dart	ใบสั่งซื้อและการรับสินค้า
vehicle_search_screen.dart	ค้นหาอะไหล่จากรุ่นรถ
customers_screen.dart, mechanics_screen.dart	ลูกค้า แต้มสะสม และช่างพร้อมเครดิตและการรับชำระ
returns_screen.dart	คืนสินค้า ออกใบลดหนี้ และประวัติการคืน
quotes_screen.dart	ใบเสนอราคาและ PDF A4
reports_screen.dart	รายงานสรุป รายงานปิดร้าน สต็อกต่ำ
cash_drawer_screen.dart	เปิดปิดกะ รายการเข้าออกลิ้นชัก
owner_review_screen.dart	รายการที่ซิงก์ไม่ผ่านหรือต้องให้เจ้าของร้านตรวจ
devices_screen.dart	ผูกและถอดถอนเครื่อง ออกรหัสผูกเครื่อง
settings_screen.dart, login_screen.dart	ข้อมูลร้าน การตั้งค่า และการตั้ง PIN ออฟไลน์; การเข้าสู่ระบบ

3.3.6 การแยกข้อมูลแคชตามผู้เช่า

บนบิลด์ API ฐานข้อมูล Drift ต้องไม่ปะปนข้อมูลของร้านต่างกัน ตาราง `AppMeta` เก็บ `tenant_id` ของแคชปัจจุบัน และ `TenantCacheGuard` ตรวจทุกครั้งที่เข้าสู่ระบบ หากพบว่าเปลี่ยนร้านจะล้างแคช แต่จะปฏิเสธการเปลี่ยนร้านเมื่อยังมีงานที่ส่งไม่ครบ ส่วนตัวนับรุ่นของแคช (`cacheGeneration`) และรุ่นของเซสชันใน `ApiClient` ป้องกันไม่ให้คำตอบที่มาช้าของร้านหรือผู้ใช้ก่อนหน้าไปเขียนทับข้อมูลของคนถัดไป

3.4 รายละเอียดฝั่งเซิร์ฟเวอร์

3.4.1 โมดูลของ NestJS

เซิร์ฟเวอร์อยู่ใน `server/src/` เขียนด้วย TypeScript บน NestJS โดยแต่ละโดเมนเป็นโมดูลแยกตามโฟลเดอร์ ดังตารางที่ 3.6 โค้ดมีสามจุดเริ่มต้นจากอิมเมจเดียวกัน คือ `main.ts` (API), `worker.ts` (ตัวประมวลผลคิว) และ `bull-board.ts` (หน้าจอดูคิว)

ตารางที่ 3.6 โมดูลหลักของเซิร์ฟเวอร์

โมดูล	หน้าที่
`auth`, `devices`, `people`	การเข้าสู่ระบบ โทเค็นต่ออายุ รหัสผ่านชั่วคราว การ enrol เครื่อง และผู้ใช้
`sales`, `returns`, `shifts`, `parked-sales`, `quotes`	งานขายหน้าร้านและธุรกรรมเงิน
`products`, `purchasing`, `customers`, `mechanics`	ข้อมูลหลักและสต็อก
`documents`	ตัวนับเลขเอกสาร (`doc_counters`)
`idempotency`	การกันคำสั่งเขียนซ้ำด้วย `Idempotency-Key`
`sync`	`POST /sync/push` และ `POST /sync/discards`
`reports`, `review-items`, `settings`	รายงาน รายการรอตรวจ และการตั้งค่า รวมถึง `GET /bootstrap`
`platform`	ระนาบผู้ดูแลแพลตฟอร์ม: สร้างและระงับร้าน นำเข้าข้อมูล
`backup`, `queue`	export ข้อมูลรายร้านแบบงานเบื้องหลัง คิวและตัวตั้งเวลา
`metrics`, `health`, `rate-limit`, `audit`, `config`, `infra`, `common`, `db`	สังเกตการณ์ จำกัดอัตรา บันทึกตรวจสอบ ค่าตั้งขณะรัน และโครงสร้างพื้นฐาน

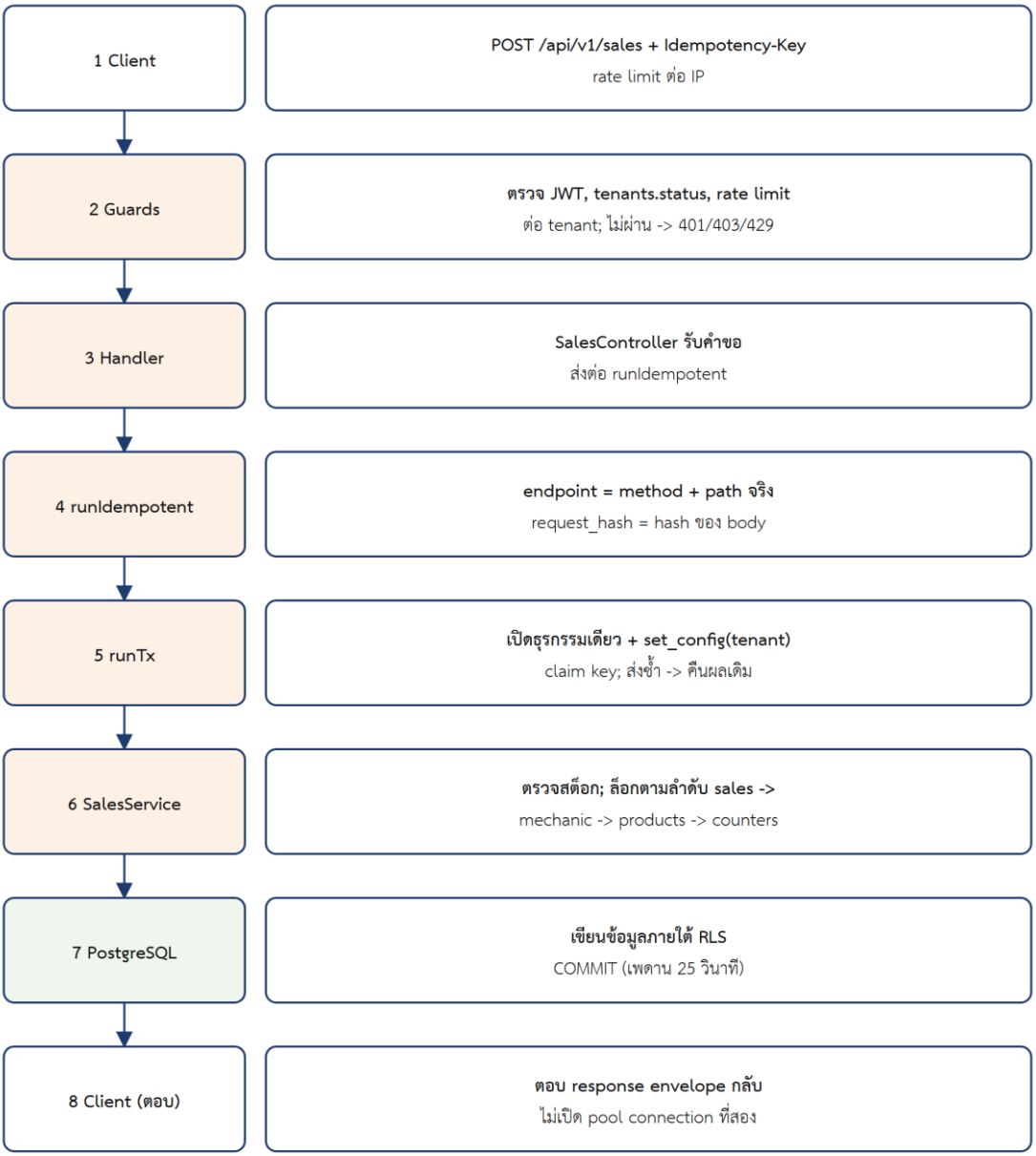
3.4.2 การแยกข้อมูลผู้เช่าด้วย RLS และ TenantService.runTx

ทุกตารางธุรกิจมีคอลัมน์ `tenant_id` และเปิด Row-Level Security แบบ `ENABLE` พร้อม `FORCE` [10] ตามแนวคิดในรูปที่ 2.1 แอปเชื่อมต่อในฐานะบทบาท `pos_app` ที่ไม่ใช่ superuser และไม่ใช่เจ้าของตาราง นโยบาย `tenant_isolation` อ่านค่า `current_setting('app.tenant_id', true)` ที่ตั้งด้วย `set_config` ภายในธุรกรรมของคำขอ และห่อด้วย `NULLIF(…, '')` เพื่อให้ค่าว่างให้ผลเป็นศูนย์แถวแทนข้อผิดพลาด ดังนั้นแม้โค้ดแอปลืมใส่เงื่อนไข `WHERE tenant_id` ฐานข้อมูลก็ยังคืนเฉพาะแถวของร้านนั้น

หัวใจของการบังคับใช้คือ `TenantService.runTx(fn)` ซึ่งไม่รับรหัสผู้เช่า (tenant id) เป็นอาร์กิวเมนต์ จึงปลอดภัยโดยโครงสร้าง เพราะผู้เช่ามาจาก JWT ที่ guard ตรวจแล้วเท่านั้น ภาคผนวกของ ADR-0003 แบ่งหน้าที่ว่า guard เป็นผู้ตัดสินสถานะของร้าน ส่วน `runTx` ซึ่งอยู่ใน handler เป็นผู้เปิดธุรกรรมและตั้งค่าตัวแปรเซสชัน แทนแบบเดิมที่ middleware เปิดธุรกรรมให้ทุกคำขอ กติกาที่ผูกพันกับส่วนนี้มีสี่ข้อ

1. ธุรกรรมอยู่ใน handler ไม่ใช่ middleware, guard หรือ interceptor
2. ไม่มีส่วนใดขอการเชื่อมต่อที่สองจาก pool ในคำขอเดียว (รวมถึง `Promise.all` ของ `runTx` หลายตัว) เพราะเคยทำให้ connection pool เต็มจนระบบหยุดตอบสนองมาแล้ว (บทที่ 5)
3. งานที่ช้าแต่ไม่ต้องอยู่ในธุรกรรมทำก่อนเข้าธุรกรรม เช่น การตรวจ PIN ผู้จัดการด้วย argon2 สำหรับการยกเลิกบิลในเฟส 1 (เฟส 2 ตัด PIN ออกและให้ระบุเหตุผลแทน)
4. มีตัวคุมเพดานเวลา 25 วินาทีที่ย้อนธุรกรรมก่อน `COMMIT` พร้อมตั้ง `statement_timeout=25s` ให้ `pos_app`

ไฟล์ทดสอบเชิงสถาปัตยกรรมสามไฟล์คือ `tenant-door.spec.ts`, `tenant-wrapper.spec.ts` และ `idempotency-routes.spec.ts` บังคับข้อแรกและข้อที่สองในระดับโครงสร้างโค้ด ส่วนเพดานเวลาทดสอบใน `tx-ceiling.e2e-spec.ts` ลำดับการประมวลผลคำขอเขียนหนึ่งคำขอแสดงดังรูปที่ 3.3



รูปที่ 3.3 ลำดับการประมวลผลคำขอเขียนหนึ่งคำขอผ่านชั้นการแยกผู้เช่าและ idempotency

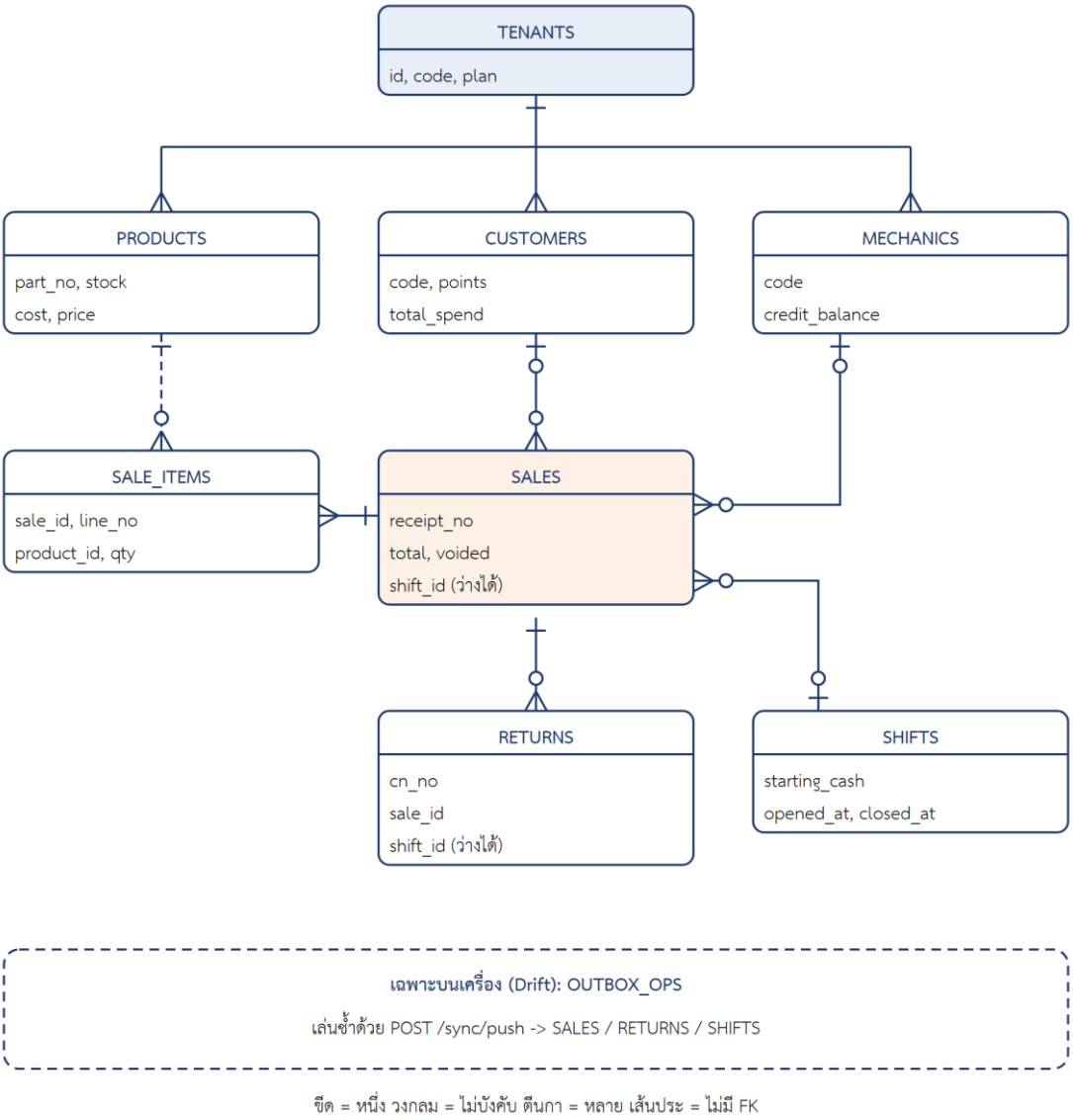
3.4.3 Idempotency และลำดับการล็อก

ระบบบังคับ `Idempotency-Key` กับคำสั่งเขียน ตาราง `idempotency_keys` เก็บคีย์และค่าสรุปของคำขอ (fingerprint) โดย fingerprint คือ method บวกพาธจริงของคำขอ (ไม่ใช่ pattern ของ route) บวกแฮชของเนื้อหา เพื่อไม่ให้คีย์เดียวถูกใช้กับสองบิลที่ต่างกันแล้วชนกัน คำขอซ้ำที่คีย์ตรงกันได้คำตอบเดิม คีย์เดิมกับเนื้อหาต่างกันถูกปฏิเสธ และคีย์ที่คำขอแรกยังประมวลผลอยู่ได้ `503 IDEMPOTENCY_KEY_IN_FLIGHT` การบันทึกคีย์อยู่ในธุรกรรมเดียวกับการเขียน และตัวนับการตอบซ้ำเพิ่มค่าเฉพาะเมื่อธุรกรรมยืนยันแล้ว

เพื่อกันเดดล็อก ทีมกำหนดลำดับล็อกเดียวสำหรับงานด้านการเงินและสต็อก คือ `sales` → ช่าง → `products` → `doc_counters` → ลูกค้า โดยการยกเลิกบิลและการคืนสินค้าอ่าน `shifts` แบบ `FOR SHARE` แทรกระหว่างล็อกบิลกับล็อกช่าง ส่วนการรับใบสั่งซื้อล็อกแถว PO ก่อนแล้วล็อกสินค้าเรียงตาม id และคำนวณต้นทุนถัวเฉลี่ยแบบปัดสตางค์ครึ่งขึ้น (ต่างจาก `round2` แบบทศนิยมลอยตัวฝั่ง Dart โดยตั้งใจ) โค้ดใหม่ที่เขียนเกินหนึ่งตารางในกลุ่มนี้ต้องคงลำดับเดิม

3.4.4 ฐานข้อมูล PostgreSQL

สคีมาเกิดจาก migrations เท่านั้น ไฟล์ `server/src/db/data-source.ts` ลงทะเบียน migration 20 ไฟล์ ซึ่งสร้างตารางรวม 29 ตาราง (27 ตารางจาก `InitialSchema` บวก `import_jobs` และ `owner_review_items`) เอกสาร `01_DATABASE.md` ระบุว่าไฟล์ migration เป็นแหล่งความจริงของสคีมา ส่วนภาษานิยามข้อมูล (Data Definition Language: DDL) ในเอกสารเป็นเพียงตัวอย่างประกอบ คีย์หลักของตารางธุรกิจเป็นคีย์ผสม `(tenant_id, id)` และคีย์นอกอ้างอิงด้วยคู่นั้น เช่น `sales` อ้างอิง `customers (tenant_id, customer_id)` ทำให้บิลของร้านหนึ่งชี้ไปลูกค้าของอีกร้านไม่ได้ ตัวเลขเงินใช้ `NUMERIC(12,2)` และส่งผ่านเครือข่ายเป็นสตริง เช่น `"1234.50"` ส่วนคอลัมน์ `id` เป็นข้อความที่ไคลเอนต์สร้างได้ ซึ่งจำเป็นต่อการออกบิลออฟไลน์ ตารางแกนกลางแสดงในรูปที่ 3.4 และตารางที่ 3.7



รูปที่ 3.4 แผนภาพ ER ของตารางแกนกลางในเซิร์ฟเวอร์ และตาราง outbox_ops ที่อยู่ในเครื่อง

ตารางที่ 3.7 ตารางแกนกลางและข้อกำหนดสำคัญ

ตาราง	คีย์ / ข้อกำหนดสำคัญ	หมายเหตุ
tenants	id UUID; code UNIQUE; plan ∈ basic, demo, loadtest; status ∈ active, suspended, closed	ฐานของผู้เช่า ไม่มี tenant_id เพราะเป็นตัวเอง
products	PK (tenant_id, id); ดัชนีไม่ซ้ำ uq_products_partno_ci บนเลขชิ้นส่วนแบบไม่แยกตัวพิมพ์	ความไม่ซ้ำเป็นข้อจำกัดของฐานข้อมูล ไม่ใช่การตรวจในแอป
sales	PK (tenant_id, id); UNIQUE (tenant_id, receipt_no); voided; shift_id	ผูกลูกค้าและช่างด้วยคีย์ผสม
sale_items	PK (tenant_id, sale_id, line_no); cost_at_sale	บิลเก่าค่า NULL ห้าม backfill (ADR-0008)
returns	PK (tenant_id, id); UNIQUE (tenant_id, cn_no); อ้างอิง sales	ใบลดหนี้
customers, mechanics	PK (tenant_id, id); UNIQUE (tenant_id, code)	มีดัชนีสำหรับการดึงข้อมูลแบบ keyset
shifts	PK (tenant_id, id); uq_shift_active หนึ่งกะ active ต่ออุปกรณ์	หลายกะต่อวันได้
devices	UNIQUE (tenant_id, device_no); one_pos_per_tenant	เครื่อง pos ได้เพียงเครื่องเดียวต่อร้าน
doc_counters	คีย์ (tenant_id, device_id, doc_type, period)	เก็บ high-water mark ของเลขเอกสาร
idempotency_keys	PK (tenant_id, key); endpoint (method + พาธจริง) และ request_hash	ใช้ใน runIdempotent

นอกจากนี้ `audit_log` เป็นแบบเพิ่มอย่างเดียว (append-only) สำหรับ `pos_app` และต้นทุน ณ วันขาย (`cost_at_sale`) ถูกเก็บในบรรทัดขายตาม ADR-0008 เพื่อให้กำไรในรายงานไม่เปลี่ยนเมื่อราคาซื้อเปลี่ยน

3.4.5 จุดปลายทาง API หลัก

เซิร์ฟเวอร์เปิดจุดปลายทางภายใต้ `/api/v1` ตารางที่ 3.8 สรุปกลุ่มหลักที่ตรวจจากตัวควบคุม (`*.controller.ts`) โดยตรง เอกสาร `02_API_SCREENS.md` เป็นสัญญาฉบับเต็ม และระบุทบาทอุปกรณ์ที่เรียกได้ในแต่ละแถว คำตอบมีรูปแบบซองมาตรฐาน (`status`, `data`, `meta`) ข้อผิดพลาดมีรหัสที่เครื่องอ่านได้ ซึ่งไคลเอนต์แปลงเป็นข้อความไทยผ่าน `server_error_resolver.dart`

ตารางที่ 3.8 กลุ่มจุดปลายทางหลักของ API

กลุ่ม	จุดปลายทาง	หน้าที่
ยืนยันตัวตน	`POST /auth/token`, `/auth/refresh`, `/auth/change-password`, `/auth/device`; `GET /auth/me`	เข้าสู่ระบบ ต่ออายุ เปลี่ยนรหัสผ่าน และแลก enrolCode เป็น device token
การขาย	`POST /sales`, `GET /sales`, `GET /sales/:id`, `GET /sales/:id/refunded-qty`, `POST /sales/:id/void`	ขาย ดูบิล และยกเลิกบิล
การคืน	`POST /returns`, `GET /returns`	ใบลดหนี้
กะและลิ้นชัก	`GET /shifts/current`, `/shifts/history`; `POST /shifts/open`, `/shifts/close`, `/shifts/current/entries`	เปิด ปิดกะ และบันทึกเงินเข้าออก
สินค้า	`GET/POST/PATCH/DELETE /products`, `POST /products/:id/adjust-stock`, `GET /products/:id/suppliers`	จัดการสินค้า ปรับสต็อก (ดึงด้วย `updatedSince` และ `afterId`)
จัดซื้อ	`/purchase-orders` พร้อม `POST /:id/receive`, `POST /:id/cancel`	ใบสั่งซื้อ รับสินค้า

กลุ่ม	จุดปลายทาง	หน้าที่
หมวดหมู่และซัพพลายเออร์	`/categories`, `/suppliers`, `GET /movements`	ข้อมูลประกอบสินค้าและประวัติความเคลื่อนไหวสต็อก
ลูกค้าและช่าง	`/customers`, `/mechanics`, `POST /mechanics/:id/credit-payments`	ข้อมูลหลักและการรับชำระเครดิต
บิลที่พัก	`GET/POST/DELETE /parked-sales`	พักตะกร้า
ใบเสนอราคา	`/quotes` พร้อม `/:id/duplicate`, `/:id/convert`, `POST /quotes/purge`	ใบเสนอราคาและการแปลงเป็นบิล
รายงาน	`GET /reports/summary`, `/closing`, `/top-products`, `/by-category`, `/stock-value`, `/low-stock`, `/product-sales`	รายงาน
อุปกรณ์	`GET/POST /devices`, `POST /devices/:id/retire`; `GET /doc-counters`	enrol ถอดถอน และอ่านตัวนับเลข
ซิงก์	`POST /sync/push`, `POST /sync/discards`	ส่งคิวจากเครื่อง pos และทิ้งรายการที่ติด
รายการตรวจ	`GET /review-items`, `POST /review-items/:id/reviewed`	หน้า "รอเจ้าของร้านตรวจ"
การตั้งค่า	`GET/PATCH /settings`, `GET /bootstrap`	ตั้งค่าร้านและข้อมูลเริ่มต้น
สำรองข้อมูล	`POST /backup/export`, `GET /backup/jobs/:id`, `.../download`	export รายร้านแบบงานเบื้องหลัง
ผู้ดูแลแพลตฟอร์ม	`POST /platform/auth/token`; `/platform/tenants` (สร้าง ระงับ ดู ออกรหัสผ่านชั่วคราว นำเข้า)	ระนาบแยกต่างหาก
สุขภาพและตัวชี้วัด	`GET /health/live`, `/health/ready`, `/metrics`	ตรวจสถานะและ Prometheus

3.4.6 การยืนยันตัวตนและการผูกเครื่อง

ตาม ADR-0009 โทเค็นเข้าถึงมีอายุ 15 นาที และโทเค็นต่ออายุหมดอายุเวลา 04:00 น. ของทุกวัน ไม่ใช่ 24 ชั่วโมงนับจากการเข้าสู่ระบบ เพราะร้านเปิดปิดกะทุกวันอยู่แล้ว เครื่องหน้าเคาน์เตอร์มีคนเดินผ่านจึงไม่ควรค้างเซสชันหลายวัน และการเพิกถอนทำได้โดยไม่ต้องมีรายการห้ามใช้ใน Redis เพราะเมื่อปิด `users.is_active` การต่ออายุครั้งถัดไปจะไม่ผ่าน เวลา 04:00 อยู่นอกเวลาทำการ พนักงานจึงไม่ถูกบังคับออกจากระบบขณะลูกค้ายืนรอ ภาคผนวกของ ADR-0009 กำหนดลายเซ็น RS256 [23] และบนเว็บให้เก็บโทเค็นเข้าถึงในหน่วยความจำเท่านั้น ส่วนโทเค็นต่ออายุและโทเค็นอุปกรณ์เก็บใน IndexedDB โดยห้ามมี `localStorage` เป็นทางสำรอง

ADR-0004 แบ่งเครื่องตามบทบาท คือ `pos` ได้เพียงเครื่องเดียวต่อร้าน (แตะลิ้นชัก ออกเลขเอกสาร และเขียนออฟไลน์ได้) และ `backoffice` ได้หลายเครื่อง (สต็อกและรายงาน ออนไลน์เท่านั้น) เส้นแบ่งคือ "แตะลิ้นชักหรือไม่" ไม่ใช่ "แตะสต็อกหรือไม่" เจ้าของร้านออก `enrolCode` ผ่าน `POST /devices` แล้วเครื่องนำรหัสไปแลกเป็น device token ที่ `POST /auth/device` และดัชนีไม่ซ้ำ `one_pos_per_tenant` บังคับกติกาเครื่อง `pos` เดียวที่ระดับฐานข้อมูล ในเฟส 2 เจ้าของโครงงานกำหนดเพิ่มว่าร้านมีบทบาทผู้ใช้ `owner` เพียงบทบาทเดียว และการถอดถอนเครื่อง การ enrol และการ export ต้องมี device token ที่ enrol แล้ว รหัสผ่านอย่างเดียวไม่พอ

การตรวจรหัสผ่านใช้ argon2 โดยลองรูปแบบยูนิโคด Normalization Form C (NFC) ก่อนแล้วลองสตริงดิบเป็นทางสำรอง เพราะ NFC จัดเรียงสระและวรรณยุกต์ภาษาไทยใหม่ ขณะที่แฮชที่สร้างไว้ก่อนหน้าเป็นแฮชของสตริงดิบ หากตัดทางสำรองออก เจ้าของร้านที่ตั้งรหัสไว้ก่อนจะเข้าระบบไม่ได้ ความยาวรหัสผ่านขั้นต่ำกำหนดไว้ที่เดียวใน `src/common/password.ts` และเมื่อสร้างร้านใหม่ ระบบออกรหัสผ่านชั่วคราวอายุ 7 วันให้เจ้าของร้าน และเมื่อเข้าสู่ระบบด้วยรหัสนี้ต้องตั้งรหัสใหม่ภายใน 10 นาที

3.4.7 ระนาบผู้ดูแลแพลตฟอร์ม

ADR-0001 กำหนดให้ผู้ดูแลแพลตฟอร์มสร้างร้านใหม่ผ่าน `POST /platform/tenants` ในธุรกรรมเดียว ได้ผู้เช่า เจ้าของ การตั้งค่า และข้อมูลเริ่มต้นพร้อมกัน ADR-0002 แยกผู้ดูแลเป็นตาราง `platform_admins` ต่างหาก ใช้ JWT ที่มีกลุ่มผู้รับ (`aud`) ต่างกัน ใช้การเชื่อมต่อฐานข้อมูลที่ข้าม RLS แยกต่างหาก และบันทึกตรวจสอบทุกครั้ง เหตุผลคือถ้าบัญชีผู้ดูแลอยู่ในตาราง `users` เดียวกับร้าน ขอบเขตของ RLS จะพร่าเลือน Nginx อนุญาตเส้นทาง `/api/v1/`

`platform/` เฉพาะ loopback และ IP ที่ระบุใน `PLATFORM_ADMIN_IPS` และหน้าจอเว็บ `platform-ui` ผูกกับ `127.0.0.1:3200` ซึ่งเข้าถึงผ่านอุโมงค์ SSH เท่านั้น

3.4.8 การจำกัดอัตรา งานเบื้องหลัง และ etcd

ADR-0006 บังคับการจำกัดอัตราสองชั้น ชั้นนอกคือ Nginx ใช้ `limit_req_zone` ต่อ IP (30 คำขอต่อวินาที burst 60) ป้องกันการท่วมจากภายนอก ชั้นในคือ guard ของ NestJS กับ Redis ต่อ `tenant_id` กันไม่ให้ร้านเดียวกินทรัพยากรจนร้านอื่นช้า เหตุผลที่ไม่ทำต่อร้านที่ Nginx คือ Nginx รุ่นฟรีอ่าน JWT ไม่ได้ และการให้ไคลเอนต์ส่งรหัสร้านในส่วนหัวมาเองฝ่าฝืนกติกาว่าผู้เช่าต้องมาจาก JWT เท่านั้น การเข้าสู่ระบบนับความพยายามด้วยคำสั่ง Lua แบบอะตอมิกบน Redis ก่อนทราบผล เพื่อให้ทุกการปฏิเสธถูกนับ หาก Redis ล่ม ตัวนับทำงานแบบ fail-open เพราะร้านหยุดขายไม่ได้ แต่การตรวจสถานะร้านไม่ fail-open ร้านที่ถูกระงับจึงยังถูกปฏิเสธทันที

งานเบื้องหลังใช้ BullMQ [27] บน `redis-queue` โดย `TenantJobRunner` ใช้เพดานเวลาคอมมิตเดียวกับ `runTx` และมีคิวงานล้มเหลว (dead-letter queue) การ export ข้อมูลรายร้านเขียนไฟล์ลงโวลุ่มที่ใช้ร่วมกันระหว่าง `worker` กับ `api` แทนการฝังข้อมูลทั้งชุดลงในงานของคิว เพราะ `redis-queue` จำกัดหน่วยความจำไว้ที่ 192 MB แบบ `noeviction` ส่วน etcd เก็บเฉพาะค่าตั้งขณะรันที่ปรับได้โดยไม่ต้อง deploy ใหม่ เช่น `log_level` ตามที่ ADR-0013 กำหนด

3.4.9 การวัดผลและสังเกตการณ์

`GET /metrics` คืนข้อความรูปแบบ Prometheus [36] และ Nginx ตอบ 404 ต่อภายนอกสำหรับเส้นทางนี้ ตัวชี้วัดหลักคือ `http_requests_total` และ `http_request_duration_seconds` ซึ่งชื่อถูกอ้างโดยแผงใน Grafana หากเปลี่ยนชื่อ แผงจะว่างโดยไม่มีข้อผิดพลาด การนับทำใน middleware เพื่อให้คำขอที่ guard ปฏิเสธ (401 และ 429) ถูกนับด้วย และ label `route` เป็น pattern ไม่ใช่พาธจริง เส้นทาง `/metrics`, `/health/live` และ `/health/ready` ถูกตัดออกจาก SLI เพราะการ scrape และ healthcheck สร้างคำขอที่สำเร็จแน่นอนราว 36 ครั้งต่อนาที ซึ่งทำให้วันที่ทุกบิลจริงล้มเหลวยังอ่านอัตราสำเร็จได้ราวร้อยละ 92 (เจ้าของโครงงานอนุมัติเมื่อ 21 กันยายน พ.ศ. 2569) ตัวนับ `pos_idempotency_replay_total` ไม่มี label `tenant_id` เพื่อควบคุมจำนวนอนุกรม

3.5 การซิงก์ข้อมูลเฟส 2

3.5.1 ภาพรวมของการทำงานออฟไลน์

เฟส 2 นำการทำงานออฟไลน์กลับมาโดยยังให้ PostgreSQL เป็นแหล่งความจริง ข้อกำหนดอยู่ใน `08_PHASE2_SPEC.md` ซึ่งเป็นผลของการตัดสินใจของเจ้าของโครงงานหลายรอบ เครื่อง `pos` เพียงเครื่องเดียวต่อร้านเป็นผู้เขียนที่ออฟไลน์ได้ ซึ่งตามที่กล่าวในบทที่ 2 เป็นกติกาที่ป้องกันการขายเกินสต็อกจากสองเครื่องที่เขียนพร้อมกันโดยไม่เห็นกัน ไคลเอนต์มีสามสถานะ (Online, Degraded, Syncing) โดย Degraded เกิดเมื่อ `GET /health/ready` ล้มต่อเนื่อง 3 ครั้ง หรือช้าเกิน 5 วินาที หรือคำสั่งเขียนไม่ได้คำตัดสิน ในสถานะ Degraded การเขียนที่อยู่ในรายการ "เข้าคิว" ตามตารางที่ 3.9 จะเข้า `outbox_ops` ส่วนการเขียนที่เป็น "ออนไลน์เท่านั้น" (เช่น สินค้า ใบสั่งซื้อ ใบเสนอราคา การตั้งค่า การยกเลิกบิลออนไลน์ การปิดกะ และการจัดการเครื่อง) จะถูกปิดปุ่ม การพักบิลเป็นข้อมูลในเครื่องอย่างเดียวและไม่ซิงก์

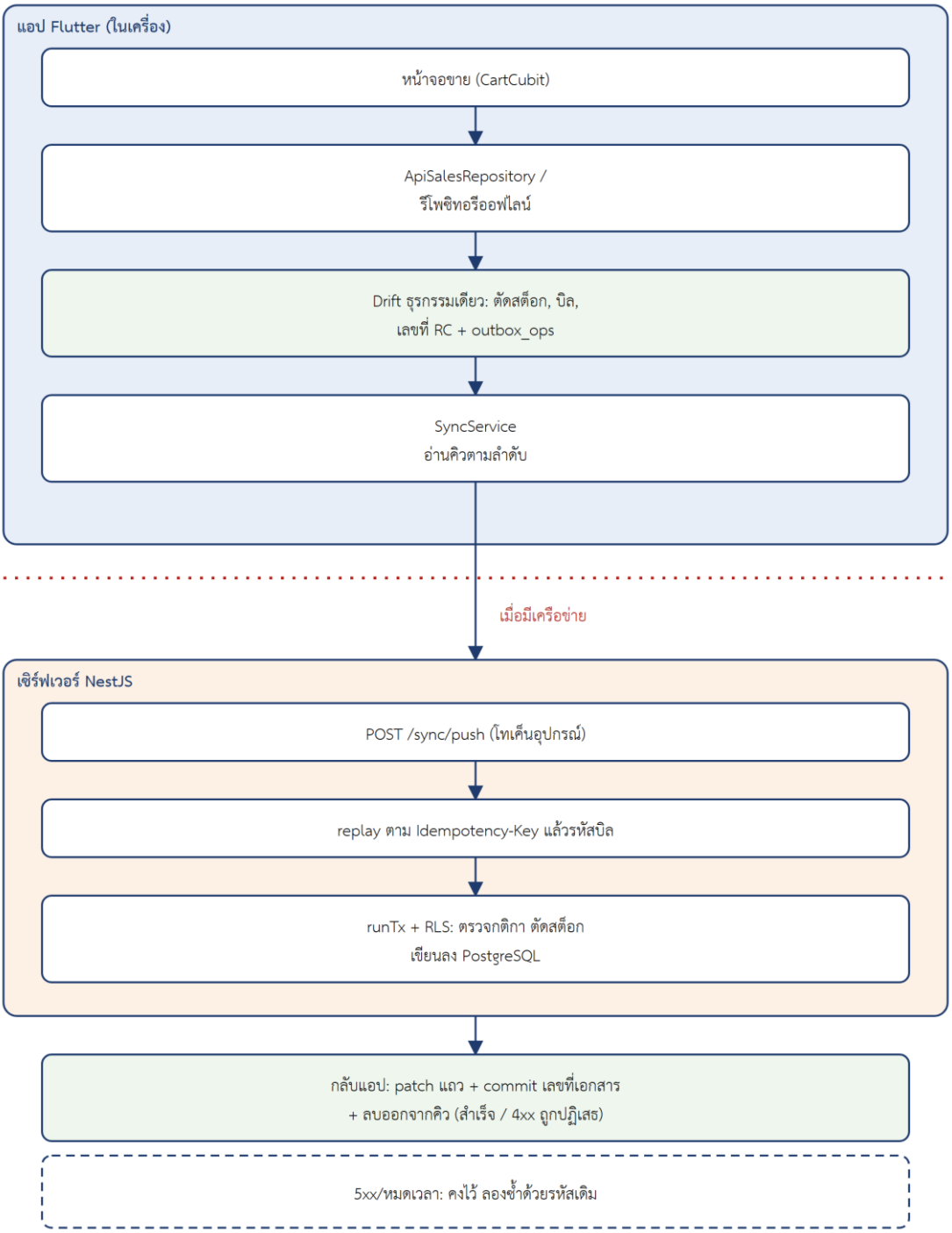
ตารางที่ 3.9 รายการ op ที่เข้าคิวและ endpoint ออนไลน์ที่เป็นคู่กัน

type	endpoint ออนไลน์ที่เทียบเท่า	หมายเหตุ
`shift.open`	`POST /shifts/open`	เปิดกะออฟไลน์ได้
`sale.create`	`POST /sales`	การขายเกินวงเงินเครดิตถูกส่งไปเป็นรายการตรวจ
`return.create`	`POST /returns`	ใบลดหนี้ออกเลข CN ในเครื่อง
`drawer.entry`	`POST /shifts/current/entries`	เงินเข้าออกลิ้นชัก
`credit_payment.create`	`POST /mechanics/:id/credit-payments`	ย้ายมาจากคิวรับชำระเครดิตเดิม
`customer.create`, `customer.update`	`POST` และ `PATCH /customers`	เทียบด้วย id อย่างเดียวสำหรับ create
`sale.void_offline`	ส่งผ่าน push เท่านั้น	ยกเลิกได้เฉพาะบิลที่ขายออฟไลน์

3.5.2 Outbox ในเครื่อง

ตาราง `OutboxOps` ใน Drift เป็นคิวตารางเดียว มีคอลัมน์ `opId` (คีย์หลัก), `idempotencyKey`, `type`, `payload` (เท่ากับเนื้อหาคำขอออนไลน์), `aggregates` (กลุ่มข้อมูลที่คำสั่งนั้นแก้ เช่น บิลหนึ่งใบหรือกะหนึ่งกะ และกลุ่มที่ต้องรอก่อน), `status` (`pending`, `stuck`, `rejected`) และ `attempts` พร้อมรหัสและข้อความผิดพลาดล่าสุด กติกาที่ทำให้

outbox เชื่อถือได้คือ id และ key ถูกสร้างก่อนส่ง และแถวธุรกรรมกับแถว outbox ถูกเขียนในธุรกรรมเดียวกันของ Drift ถ้าแอปถูกปิดกลางการขาย จะมีทั้งบิลและ op หรือไม่มีทั้งคู่ ดังรูปที่ 3.5



รูปที่ 3.5 การไหลของรูปแบบกล่องขาออกในโครงงาน ตั้งแต่การขายออฟไลน์จนถึงการเล่นซ้ำที่เซิร์ฟเวอร์

3.5.3 POST /sync/push

เมื่อเครือข่ายกลับมา `SyncService` ส่ง op เป็นชุดละไม่เกิน 50 รายการด้วยส่วนหัว `X-Device-Token` ฝั่งเซิร์ฟเวอร์ประมวลผลทีละ op ตามลำดับดังนี้

1. เรียก `runIdempotent` ด้วยคีย์และ fingerprint เดียวกับเส้นทางออนไลน์ ถ้าเคยทำแล้วตอบ `applied`
2. ตรวจซ้ำด้วยรหัสที่ไคลเอนต์สร้าง (เช่น `sales.id`) ในธุรกรรมเดียวกัน ถ้า id ซ้ำแต่เนื้อหาไม่ตรง ตอบ `rejected`
3. เรียกบริการเดียวกับ controller ออนไลน์ด้วยลำดับล็อกเดิม ถ้าบริการปฏิเสธ ตอบ `rejected`
4. ทำผลพลอยได้ในธุรกรรมเดียวกัน เช่น การตั้งธงว่าขายออฟไลน์ การปรับวันที่ที่ผิดปกติ และการสร้างรายการตรวจ

ทุก op มี `runTx` ของตัวเองต่อกันทีละตัว เมื่อ op ใดได้ผลที่ไม่แน่นอน (คำขอเดิมยังค้างเกินเพดานเวลา หรือ 5xx) เซิร์ฟเวอร์ตอบ `retry` แล้วหยุดชุดนั้น

ฝั่งไคลเอนต์ส่งทีละคำขอ เมื่อได้ `retry` จะนับ `attempts` และเมื่อครบ 3 ครั้ง op จะเป็น `stuck` และปรากฏในหน้ารายการรอเจ้าของร้านตรวจ op ที่อยู่ใน aggregate เดียวกับ op ที่ติดจะรอต่อเนื่องกันตามลำดับ ส่วน op อื่นยังไปต่อได้

ตัวอย่างเช่น คิว [เปิดกะ S] [ขาย A] [ยกเลิก A] [ขาย B] ถ้าขาย A ติด การยกเลิก A จะรอ แต่ขาย B ส่งต่อได้ ทว่าถ้าการเปิดกะ S ติด ทุก op ในกะ S จะรอ

เจ้าของร้านเลือกส่งใหม่ด้วยคีย์เดิมหรือทิ้งพร้อมหมายเหตุผ่าน `POST /sync/discards` และการทิ้งใช้ตาราง `op_effects` ย้อนผลเฉพาะที่ได้ใช้จริง การปิดกะต้องรอให้คิวว่างทั้งหมด `ApiShiftsRepository.closeShift` จึงส่งคิวที่ส่งได้ก่อน แล้วปฏิเสธด้วย `OUTBOX_NOT_EMPTY` เมื่อยังมีแถวใน `outbox_ops` ไม่ว่าสถานะหรือชนิดใด การตรวจนี้ทำที่ไคลเอนต์เท่านั้นเพราะเซิร์ฟเวอร์มองไม่เห็นคิวในเครื่อง

3.5.4 เลข RC และ CN ที่ออกบนเครื่อง

ADR-0007 กำหนดเลขเอกสารแบบเรียงต่อเครื่องและรีเซ็ตรายเดือน รูปแบบ `RC01-2569-08-0042` คือ prefix ตามชนิดเอกสาร (RC ใบเสร็จ, CN ใบลดหนี้, PO ใบสั่งซื้อ, QT ใบเสนอราคา, CP ใบรับชำระเครดิตช่าง) ตามด้วยหมายเลขเครื่องสองหลัก ปี พ.ศ. เดือน และลำดับสี่หลัก การเติมศูนย์ให้หมายเลขเครื่องทำให้ `RC1-` กับ `RC12-` ไม่อ่านกำกวม ในเฟส 1 เซิร์ฟเวอร์ออกเลขทั้งหมดด้วย `UPDATE doc_counters … RETURNING` ใต้ row lock ในธุรกรรมเดียวกับการขาย ในเฟส 2 เครื่อง `pos` ออก RC และ CN เองทั้งออนไลน์และออฟไลน์ ส่วน PO, QT

และ CP เซิร์ฟเวอร์ยังออกให้เสมอ เพราะเครื่อง `backoffice` ออนไลน์เสมอและไม่ได้ประโยชน์จากการออกเลขเอง

เซิร์ฟเวอร์ตรวจว่า prefix ตรงกับชนิดเอกสาร และหมายเลขเครื่องตรงกับเครื่องจากโทเค็น เก็บ high-water mark ด้วย `INSERT … ON CONFLICT … DO UPDATE SET last_no = GREATEST(…)` และข้อจำกัด UNIQUE บน `receipt_no` และ `cn_no` เป็นด่านสุดท้ายกันเลขซ้ำ เมื่อออฟไลน์ข้ามเดือน เลขเริ่มที่ 0001 ใหม่ตามนาฬิกาเครื่อง ส่วนเครื่องที่ยังไม่เคยเตรียมตัวนับ (เพิ่ง enrol หรือเพิ่งอัปเกรด) ถูกห้ามออกเลขออฟไลน์ด้วยรหัส `OFFLINE_SEED_REQUIRED` กติกาสำคัญอีกข้อคือเลข RC หรือ CN ที่เซิร์ฟเวอร์ออก ทั้งในคำตอบออนไลน์และจากการเล่นซ้ำของ `/sync/push` ต้องถูกบันทึกลงตัวนับในเครื่องผ่าน `DocNumberService.commitServerIssued` มิฉะนั้นเครื่องอาจออกเลขซ้ำกับที่เซิร์ฟเวอร์เคยให้

3.5.5 การดึงข้อมูลแบบ keyset

หลังส่งคิวหมด ไคลเอนต์ดึงข้อมูลใหม่จากเซิร์ฟเวอร์ (push ก่อน pull เสมอ) โดยใช้ `meta.nextCursor` ของเซิร์ฟเวอร์ที่เก็บในตาราง `SyncCursors` ต่อชนิดข้อมูล เหตุผลที่ต้องเป็น keyset แทน offset หรือ timestamp เปล่า คือหลายแถวอาจมี `updated_at` เท่ากันจาก `now()` ของธุรกรรมเดียว และ timestamp ระดับมิลลิวินาทีจะข้ามหรือซ้ำแถวที่ใช้ค่าเดียวกัน cursor จึงมีความละเอียดระดับไมโครวินาทีร่วมกับ `afterId` ส่วนหน้าแรกของแต่ละรอบไคลเอนต์ถอย cursor กลับ 30 วินาทีและไม่ส่ง `afterId` เพื่อรองรับธุรกรรมที่ commit ช้ากว่าที่ cursor ผ่านไปแล้ว ซึ่งปลอดภัยเพราะเพดานเวลา 25 วินาทีทำให้ไม่มีธุรกรรมใดค้างนานกว่านั้น แถวที่ถูกลบเป็นสัญลักษณ์ลบ (tombstone) ให้ไคลเอนต์ซ่อน การดึงข้อมูลไม่ทับสต็อกของสินค้าที่ยังมี op ค้าง และไม่ทับยอดลูกค้าหรือช่างที่ถูกอ้างโดย op ทางการเงินที่ยังไม่ส่ง ตามหลักความสอดคล้องในที่สุดที่กล่าวในบทที่ 2

การขายเกินสต็อกขณะออฟไลน์จึงถูกจำกัดด้วยกลไกสี่ชั้น ชั้นแรกคือมีผู้เขียนออฟไลน์ได้เพียงเครื่อง `pos` เดียวต่อร้าน (`one_pos_per_tenant`) ชั้นที่สองคือการขายออฟไลน์ใน `ApiSalesRepository` ตรวจสต็อกในเครื่องแบบเข้มงวดก่อนบันทึก และปฏิเสธด้วยข้อความ สต็อกไม่พอ เมื่อไม่พอ ชั้นที่สามคือการดึงข้อมูลไม่ทับสต็อกที่ยังมี op ค้าง ตัวเลขในเครื่องจึงสะท้อนการขายที่ยังไม่ส่ง และชั้นสุดท้ายคือเมื่อ `/sync/push` เล่นซ้ำการขาย เซิร์ฟเวอร์ตรวจสต็อกอีกครั้งด้วยบริการเดียวกับเส้นทางออนไลน์ หากสต็อกไม่พอ (เช่น ถูกปรับสต็อกจากเครื่อง `backoffice` ระหว่างที่ออฟไลน์) op นั้นได้ผล `rejected` รหัส `INSUFFICIENT_STOCK` และปรากฏในหน้ารายการรอเจ้าของร้านตรวจ ไม่มีการตัดสต็อกจนติดลบโดยเงียบ

3.6 ความปลอดภัย

มาตรการด้านความปลอดภัยอยู่หลายชั้น ตารางที่ 3.10 รวบรวมไว้พร้อมที่มา ส่วนการสำรองข้อมูลออกนอกเครื่องเสมือนยังไม่ได้ทำ (สถานะรายงานในบทที่ 4)

ตารางที่ 3.10 มาตรการด้านความปลอดภัยและที่มา

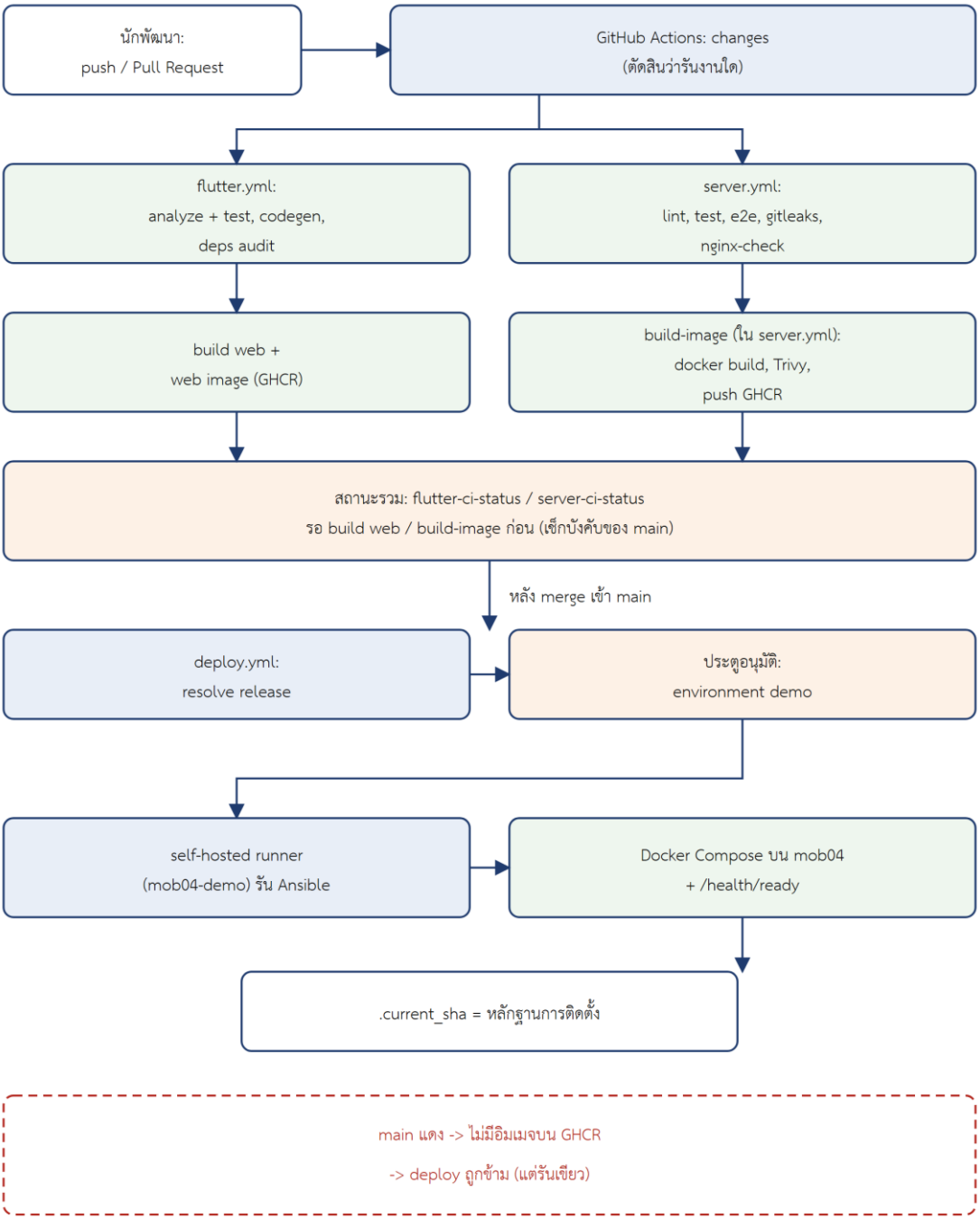
ชั้น	มาตรการ	เหตุผล / ที่มา
การแยกผู้เช่า	RLS แบบ ENABLE และ FORCE บนตารางธุรกิจ; บทบาท `pos_app` ไม่ใช่ superuser	ฐานข้อมูลบังคับแม้โค้ดแอปผิดพลาด (T1, ADR-0003)
การยืนยันตัวตน	JWT RS256; access 15 นาที; refresh หมดอายุ 04:00; รหัสผ่านแฮชด้วย argon2	ADR-0009
โทเค็นบนเว็บ	โทเค็นเข้าถึงในหน่วยความจำ, โทเค็นต่ออายุหรือโทเค็นอุปกรณ์ใน IndexedDB, ห้าม localStorage	ภาคผนวก ADR-0009 (25 ก.ย. 2569)
งานเสี่ยง	การถอดถอน การ enrol และการ export ต้องมี device token ที่ enrol แล้ว	สเปกเฟส 2
ระนาบผู้ดูแล	ตาราง `platform_admins` แยก , `aud` ต่างกัน, การเชื่อมต่อที่ข้าม RLS แยก, Nginx จำกัด IP	ADR-0002
การเขียนซ้ำ	`Idempotency-Key` พร้อม fingerprint ของพาธจริง	กันบิลซ้ำเมื่อคำตอบหาย
การจำกัดอัตรา	Nginx ต่อ IP และ guard ต่อผู้เช่า; เข้าสู่ระบบนับก่อนรู้ผลด้วย Lua `INCR` อะตอมิก	ADR-0006
บันทึกตรวจสอบ	`audit_log` เพิ่มได้อย่างเดียวสำหรับ `pos_app`	migration `AuditLogAppendOnly`
ความลับ	ค่าตัวอย่างสาธารณะทำให้แอปไม่ยอมเริ่มทำงาน เว้นแต่ `ALLOW_DEV_SECRETS=true` ซึ่งห้ามตั้งบนเครื่องจริง	#398, #410

ชั้น	มาตรการ	เหตุผล / ที่มา
อิมเมจและ dependency	อิมเมจตรึงด้วย digest; Trivy สแกนและบล็อกการ push; `pnpm audit`; gitleaks สแกนความลับ	ADR-0013
การเข้ารหัสระหว่างทาง	HTTPS บน `mob04` ด้วยใบรับรองเซิร์ฟเวอร์ที่ออกโดย CA ส่วนตัว (`certgen`); แอปบนอุปกรณ์เชื่อถือ CA นี้ผ่าน `HttpOverrides` โดยไม่ใช้ `badCertificateCallback`	ไม่มีชื่อ DNS จึงขอใบรับรองสาธารณะไม่ได้ (`07_CICD_DEPLOY.md` §5 "TLS", PR #552)
ส่วนหัวและต้นทาง	helmet; Cross-Origin Resource Sharing (CORS) ตาม `CORS_ORIGINS` ที่ต้องไม่ว่าง	#367

3.7 การนำระบบขึ้นใช้งาน (deploy)

3.7.1 โครงสร้างการ deploy และระดับของ CI/CD

สภาพแวดล้อมจริงเพียงแห่งเดียวคือเครื่องเสมือนของคณะชื่อ `mob04` (สภาพแวดล้อม `demo`) ตาม ADR-0013 ซึ่งอยู่ในเครือข่ายมหาวิทยาลัย Docker Compose ประกอบจากสามไฟล์ คือ `server/docker-compose.yml` (ฐาน), `deploy/compose/vm.override.yml` (สลับเป็นอิมเมจจาก GHCR ตามแท็กของคอมมิต) และ `deploy/compose/monitoring.yml` (Prometheus, Grafana, Node Exporter) ไปป์ไลน์ทั้งหมดแสดงดังรูปที่ 3.6



รูปที่ 3.6 ภาพรวมไปป์ไลน์ CI/CD ของโครงงาน ตั้งแต่คำขอรวมถึงการติดตั้งบนเครื่องสาธิต

CI/CD บน GitHub Actions [31] แบ่งเป็นสี่ระดับ ระดับ 1 คือ Flutter CI (`flutter.yml`: `analyze-and-test`, `deps-audit`, `codegen-check`, `build-web`) ระดับ 2 คือ backend CI (`server.yml`: `secrets`, `lint`, `audit`, `unit`, `ci-guards`, `nginx-check`, `integration`, `build-image`) ระดับ 3 คือการสร้างอิมเมจลง GHCR พร้อม Trivy ที่บล็อกเมื่อพบช่องโหว่ระดับสูง และระดับ 4 คือการ deploy ด้วย Ansible (`deploy.yml`)

นอกจากสี่ระดับนี้ยังมีเวิร์กโฟลว์ `android-apk.yml` ที่สั่งรันด้วยมือเท่านั้น (รายละเอียดในหัวข้อถัดไป) ทั้งสองเวิร์กโฟลว์แรกถูกเรียกทุก push และ pull request โดยไม่กรองพาธที่ชั้นนอก แต่มีงาน `changes` ภายในตัดสินว่าจะรันงานใด และลงท้ายด้วยงานสถานะที่รายงานผลเสมอ (`flutter-ci-status`, `server-ci-status`) ซึ่งเป็นเงื่อนไขบังคับของสาขา `main` เพียงสองรายการ เหตุผลคือการตรวจสอบสถานะที่บังคับต้องรายงานผลทุกครั้ง หากกรองพาธที่ชั้นนอก PR ที่แก้เฉพาะ `server/` จะไม่มีสถานะของ Flutter CI ให้ผ่าน push ที่แก้เฉพาะเอกสาร (`*.md` หรือ `docs/**`) ข้ามการทดสอบและการสร้างอิมเมจ เครื่องเสมือนจึงอยู่ที่ SHA ของโค้ดล่าสุดเสมอ

3.7.2 Ansible และ self-hosted runner

playbook `deploy.yml` ของ Ansible [35] ทำตามลำดับ ตรวจ SHA ที่รันอยู่และออกก่อนเมื่อซ้ำ ซิงก์ไฟล์ Compose และ Nginx ดึงอิมเมจจาก GHCR รัน migrations ผ่านคอนเทนเนอร์ `migrate` (สคีมาก่อนโค้ด) รีสตาร์ตแบบหมุนเวียน `api-1` ⟶ `api-2` ⟶ `api-3` พร้อมตรวจสุขภาพ รีสตาร์ต worker และ bull-board อัปเดตไฟล์เว็บแล้วสร้าง Nginx ใหม่ ตรวจ `GET /health/ready` ผ่าน Nginx บันทึก SHA ลง `/opt/pos/.current_sha` และสุดท้ายเริ่มชุดมอนิเตอร์ ซึ่งหากล้มเหลวจะเพียงแจ้งเตือน migration รันเป็นงานครั้งเดียวก่อน api ทุกตัว ไม่ใช่ตอนบูต เพราะอินสแตนซ์สามตัวจะแย่งกัน เมื่อ readiness ล้มเหลว สคริปต์ deploy จะติดตั้งรุ่นก่อนหน้ากลับอัตโนมัติ

เครื่องเสมือนอยู่หลังเครือข่ายมหาวิทยาลัยที่ตัวรันของ GitHub เข้าไม่ถึง ทีมจึงติดตั้ง self-hosted runner บน `mob04` และสภาพแวดล้อม `demo` กำหนดให้ผู้ตรวจที่ระบุอนุมัติก่อนงาน `deploy` จะแตะเครื่อง (ภาคผนวกของ ADR-0013) การ trigger อัตโนมัติทุกครั้งที่ `main` ผ่านยังคงอยู่ แต่งานจะค้างในสถานะรออนุมัติ ซึ่งเป็นกลไกเดียวที่กันการ merge ไม่ให้ไปทับเครื่องระหว่างการสาธิต ผลการ deploy จริงรายงานในบทที่ 4

3.7.3 TLS ด้วย CA ส่วนตัวและการสร้างไฟล์ติดตั้ง Android

แอปบน Android ที่สร้างด้วยบิลด์ API เรียก `https://172.30.58.20` ซึ่ง `HttpClient` ของ Dart จะปฏิเสธเมื่อใบรับรองไม่เชื่อมโยงถึง root ที่เชื่อถือหรือไม่มี IP นั้นใน Subject Alternative Name (SAN) โครงงานยังไม่มีชื่อ DNS จึงขอใบรับรองสาธารณะไม่ได้ ทีมจึงเลือกใช้ Certificate Authority (CA) ส่วนตัว (PR #552) แทนการตั้งให้แอปข้ามการตรวจใบรับรอง คอนเทนเนอร์ครั้งเดียว `certgen` (`server/docker/certgen/certgen.sh`) สร้าง CA ไว้ในวอลุ่ม `certs-ca` เพียงครั้งเดียวและไม่แทนที่อีก เพราะทุกไฟล์ติดตั้งที่สร้างแล้วฝังส่วนสาธารณะของ CA นี้ไว้ จากนั้นออกใบรับรองเซิร์ฟเวอร์ที่มี SAN เป็น `localhost`, `127.0.0.1`

และ `172.30.58.20` ให้ Nginx ใช้ และออกใหม่เมื่อใบเดิมหาย ไม่ได้ลงนามโดย CA นี้ มี SAN ต่างไป หรือใกล้หมดอายุภายใน 30 วัน ส่วนกุญแจของ CA ไม่ออกจากวอลุ่ม ไม่เข้า git และไม่เข้า CI playbook `deploy.yml` รัน `certgen` ด้วย `docker compose run` ก่อนสร้าง Nginx ใหม่ เพื่อให้การออกใบรับรองที่ล้มเหลวทำให้การ deploy ล้มเหลวด้วย (`up -d` ไม่รอคอนเทนเนอร์ครั้งเดียว) และงาน `nginx-check` ของ `server.yml` รันสคริปต์ทดสอบ `deploy/scripts/test/certgen.test.sh` ฝั่งแอป `frontend/lib/core/network/pos_trust_io.dart` อ่านใบรับรอง CA จากแอสเซตแล้วตั้ง `HttpOverrides` ให้ทุก `HttpClient` เชื่อถือ CA นี้ควบคู่กับ root ของระบบ การตรวจชื่อโฮสต์ยังทำงานตามเดิม และถ้าแอสเซตว่างหรือเสียแอปจะไม่หยุดทำงาน เพียงใช้ root ของระบบ

เวิร์กโฟลว์ `.github/workflows/android-apk.yml` (PR #551) สั่งรันด้วยมือเท่านั้น (`workflow_dispatch`) และทำงานเฉพาะบนสาขา `main` สร้างไฟล์ติดตั้ง Android Package Kit (APK) ชนิดเดียว คือบิลด์ API (`USE_API_WRITES=true`, `API_BASE_URL=https://172.30.58.20`) ลงนามด้วยกุญแจถาวรจากความลับ `ANDROID_KEYSTORE_B64` เพื่อให้รุ่นถัดไปติดตั้งทับได้ แล้วเผยแพร่เป็น pre-release บน GitHub ชื่อ `apk-<sha7>` เวิร์กโฟลว์ปฏิเสธการสร้างหากไม่มีความลับดังกล่าว (ไม่ใช้กุญแจชั่วคราว) หรือ release ชื่อเดิมมีอยู่แล้ว ไม่มี APK แบบออฟไลน์ล้วนจาก `main` เพราะค่าเริ่มต้นของบิลด์คือโหมด API และแอปต้องเพิ่มสิทธิ์ `INTERNET` ใน `AndroidManifest.xml` เนื่องจากบิลด์ release ไม่มีสิทธิ์นี้โดยปริยาย

ทั้งสองส่วนถูกนำขึ้นใช้งานในวันเดียวกัน (3 ตุลาคม พ.ศ. 2569) คือ `certgen` สร้าง CA บน `mob04` ในการ deploy คอมมิต `7ea0178` ใบรับรอง CA ถูกคอมมิตลงแอสเซตตามขั้นตอนใน `07_CICD_DEPLOY.md` §5 "TLS" (PR #559) และเวิร์กโฟลว์ APK สร้างไฟล์ติดตั้งได้หลังยกรุ่นปลั๊กอิน `file_picker` (PR #560) หลักฐานและข้อจำกัดอยู่ในบทที่ 4 ข้อควรระวังเชิงออกแบบคือการเปลี่ยน IP ของ VM ต้องแก้ SAN ใน `certgen.sh` และหาก CA ในวอลุ่มสูญหาย APK ทุกตัวที่แจกไปแล้วจะเชื่อมต่อไม่ได้จนกว่าจะคอมมิต CA ใหม่และสร้าง APK ใหม่

3.8 เทคโนโลยีที่ใช้

ตารางที่ 3.11 รวบรวมเทคโนโลยีและเวอร์ชันของไลบรารี โดยเป็นช่วงเวอร์ชันที่ประกาศใน `frontend/pubspec.yaml` และ `server/package.json` ส่วนเวอร์ชันของอิมเมจโครงสร้างพื้นฐานแสดงไว้แล้วในตารางที่ 3.3

ตารางที่ 3.11 เทคโนโลยีที่ใช้ในโครงงาน

ชั้น	เทคโนโลยี	เวอร์ชัน	วัตถุประสงค์
ไคลเอนต์	Flutter / Dart	Flutter 3.44.3 (ใน CI); Dart SDK ^3.12.2	โค้ดชุดเดียวหลายแพลตฟอร์ม (CI สร้างเฉพาะเว็บ)
ไคลเอนต์	Drift + sqlite3	drift ^2.34.0; sqlite3 ^3.4.0 (ทดสอบ)	ฐานข้อมูล SQLite ในเครื่องแบบ type-safe
ไคลเอนต์	flutter_bloc / bloc, go_router, http	^9.1.1 / ^9.2.1, ^17.3.0, ^1.5.0	จัดการสถานะ กำหนดเส้นทาง และเรียก API
ไคลเอนต์	pdf, printing, barcode_widget	^3.13.0, ^5.15.0, ^2.0.4	ใบเสร็จ ใบเสนอราคา A4 ป้ายบาร์โค้ด
ไคลเอนต์	ฟอนต์ Sarabun, BarlowCondensed	ฝังเป็น asset (OFL)	แสดงผลภาษาไทยและสร้าง PDF โดยไม่ต้องใช้เครือข่าย
เซิร์ฟเวอร์	Node.js, pnpm	>=22 (`node:22-alpine`), 10.34.5	รันไทม์และจัดการแพ็กเกจ
เซิร์ฟเวอร์	NestJS, Express, helmet	^12.0.1, ^5.2.1, ^8.3.0	เฟรมเวิร์ก API แบบโมดูล และส่วนหัวความปลอดภัย
เซิร์ฟเวอร์	TypeORM, pg	^1.1.1, ^8.23.0	เชื่อมต่อ PostgreSQL และรัน migrations
เซิร์ฟเวอร์	BullMQ, ioredis	^6.3.4, ^6.0.0	คิวงานเบื้องหลัง และไคลเอนต์ Redis
เซิร์ฟเวอร์	argon2, jsonwebtoken	^0.45.1, ^9.0.3	แฮชรหัสผ่านและออก JWT
เซิร์ฟเวอร์	pino, prom-client	^10.3.1, ^15.1.3	บันทึกแบบมีโครงสร้าง และตัวชี้วัด Prometheus
เซิร์ฟเวอร์	TypeScript, vitest, oxlint	^6.0.2, ^4.1.2, ^1.58.0	ภาษา ทดสอบ และตรวจโค้ด
ทดสอบโหลด	k6	ตามสคริปต์ใน `server/test/k6/`	วัดโหลดและสภาวะแข่งกันบนสต็อก
CI/CD	GitHub Actions, GHCR, Trivy, gitleaks, Ansible	ตามไฟล์ workflow	สร้าง ตรวจ เผยแพร่อิมเมจ และ deploy

3.9 วิธีการทำงานของทีม

3.9.1 การแบ่งงานและการติดตามงาน

ทีมสามคนแบ่งเลนตามตารางที่ 1.4 ในบทที่ 1 และติดตามงานด้วยตั๋วงานบน GitHub (issue) ตั๋วงานหลัก (parent issue) ไม่ถูกลงมือโดยตรง และตั๋วงานที่เป็นการตัดสินใจของเจ้าของโครงงานไม่ถูกตัดสินใน pull request สำหรับเฟส 2 เอกสาร `09_PHASE2_LANES.md` แบ่งตั๋วงาน 35 ตั๋วเป็นสามเลน โดยเลน B ดูแลกลไกฝั่งเครื่องทั้งหมด เลน C ดูแลเซิร์ฟเวอร์ หน้าจอใหม่ และงานปฏิบัติการ ส่วนเลน A มีตั๋วงานปลดล็อกสามตั๋ว หลักสำคัญคือ "การรอกันไม่ข้ามเลน" กล่าวคือความต้องการข้ามเลนต้องกำหนดเป็นสัญญา (contract) เช่น ไฟล์ตัวอย่างคำขอ `/sync/push` และอินเทอร์เฟซ `SyncFacade` ไม่ใช่การรอคิวกัน ตั๋วงานที่ถูกแบ่งเป็นสองส่วน (ไคลเอนต์กับเซิร์ฟเวอร์) ไม่ถือว่าทำงานกับของจริงได้จนกว่าอีกส่วนจะรวมโค้ด แต่ละฝั่งจึงทดสอบกับตัวจำลอง (fake) ของฝั่งตนเอง

3.9.2 ADR และลำดับความสำคัญของเอกสาร

การตัดสินใจเชิงสถาปัตยกรรมบันทึกเป็น ADR หนึ่งไฟล์ต่อหนึ่งเรื่องตามแนวทางของ Nygard [38] ปัจจุบันมี 13 หมายเลข (0001 ถึง 0013) ครอบคลุมการสร้างผู้เช่า ระนาบผู้ดูแล วงจรชีวิตผู้เช่า บทบาทอุปกรณ์ การส่งออกข้อมูล ขีดจำกัดอัตรา เลขที่ใบเสร็จ ต้นทุน ณ เวลาขาย อายุเซสชัน การเขียนผ่านแคช โมโนรีโป CouchDB (ถูกปฏิเสธ) และชุดเครื่องมือ CI/CD กติกาของทีมคือเมื่อเอกสารขัดกับ ADR ให้ยึด ADR และ ADR ที่เปลี่ยนใจภายหลังใช้ภาคผนวกลงวันที่แทนการเขียนทับประวัติ

3.9.3 ขั้นตอนการพัฒนาและการตรวจ

ทุกงานเข้า `main` ผ่าน pull request ที่ต้องผ่านการตรวจสอบสถานะบังคับสองรายการ (ตั้งแต่ 6 ตุลาคม พ.ศ. 2569 PR งานเข้าสาขา `develop` ก่อน แล้ว `develop` เข้า `main` ด้วย merge commit เท่านั้นตาม ruleset `24564072` เพื่อให้ SHA ที่ใช้เป็นเกณฑ์การย้อนรุ่นยังอยู่ในประวัติของ `main` และ `develop` มีการป้องกันสาขาแบบเดียวกับ `main`) การป้องกันสาขาตั้งแต่ 15 กันยายน พ.ศ. 2569 ห้าม force-push และห้ามลบสาขา แต่ไม่ได้กำหนดจำนวนผู้อนุมัติขั้นต่ำ (ตั้งไว้ 0 คน) ที่เก็บโค้ดมีสาขายาวสองสาขาคือ `main` และ `POC_sample_offline_first` (เพิ่ม `develop` เป็นสาขาที่สามเมื่อ 6 ตุลาคม พ.ศ. 2569) ตั๋วงานแต่ละตั๋วเริ่มด้วยการทบทวน

แนวทางจากมุมคนนอก (ตั้งคำถามว่าเจตนาถูกหรือไม่และมีทางที่ง่ายกว่าหรือไม่ แล้วตามรอยเส้นทางโค้ดจริง) แก้โค้ดเท่าที่จำเป็นพร้อมเกณฑ์ที่ตรวจได้ และปิดด้วยการทบทวนโค้ดสองแกน คือ มาตรฐานของที่เก็บโค้ด และความตรงกับสเปกของตั๋วงานต้นทาง บทเรียนจากกระบวนการนี้ถูกบันทึกกลับเป็นกติกาของโครงงาน ซึ่งกรณีสำคัญสรุปไว้ในบทที่ 5

ในทางปฏิบัติ PR ที่รวมแล้ว 330 รายการถูกรวมโดยผู้เขียน PR เองทั้งหมดยกเว้นหนึ่งรายการ และไม่มีรายการใดได้รับการอนุมัติผ่านฟังก์ชันรีวิวของ GitHub จากสมาชิกคนอื่น การตรวจก่อนรวมจึงอาศัยการตรวจสอบสถานะของ CI ที่บังคับ ความเห็นจากบริการรีวิวโค้ดอัตโนมัติ CodeRabbit ซึ่งแสดงความเห็นใน 328 PR และการทบทวนโค้ดด้วยเครื่องมือช่วยพัฒนาที่อธิบายในหัวข้อถัดไป

3.9.4 เครื่องมือช่วยพัฒนาด้วยปัญญาประดิษฐ์

ทีมใช้ Claude Code ซึ่งเป็นผู้ช่วยเขียนโค้ดด้วยปัญญาประดิษฐ์ ในการสร้างโค้ดและชุดทดสอบ การทบทวนโค้ด และการร่างเอกสาร จากคอมมิตบน `main` ทั้งหมด 1,058 รายการ มี 591 รายการ (ร้อยละ 56) ที่มีบรรทัด `Co-Authored-By: Claude` ระบุผู้ร่วมเขียน และหากไม่นับคอมมิตที่เป็นการ merge มี 567 จาก 706 รายการ (ร้อยละ 80) ตัวเลขนี้นับจากข้อความคอมมิต จึงบอกได้เพียงว่าคอมมิตใดมีเครื่องมือมีส่วนร่วม ไม่ได้วัดสัดส่วนของโค้ดที่เครื่องมือเขียน งานที่สมาชิกทีมทำเอง ได้แก่ การกำหนดความต้องการ การตัดสินใจเชิงออกแบบที่บันทึกเป็น ADR การกำหนดและตรวจรับเกณฑ์ของตั๋วงาน การรวม PR และการอนุมัติการ deploy บนสภาพแวดล้อม `demo` ผลลัพธ์ของเครื่องมือถูกตรวจด้วยกลไกเดียวกับโค้ดอื่น คือ CI ที่บังคับ ชุดทดสอบ e2e และการนับเกณฑ์ตรวจรับซ้ำกับไฟล์ทดสอบ

บทที่ 4
ความก้าวหน้าการดำเนินงาน

บทนี้รายงานผลการดำเนินงาน ณ วันที่ 1 ตุลาคม พ.ศ. 2569 โดยส่วนแรกครอบคลุมงานฝั่งไคลเอนต์ Flutter ได้แก่ การย้ายระบบเดิม หน้าจอ ชั้นเขียนข้อมูลผ่าน API ระบบทำงานออฟไลน์ฝั่งอุปกรณ์ และผลการทดสอบ ส่วนที่สองครอบคลุมเซิร์ฟเวอร์ CI/CD การ deploy และเกณฑ์ตรวจรับของเฟส 1 การออกแบบอยู่ในบทที่ 3 บทนี้จึงเน้นสิ่งที่ทำได้จริงและหลักฐาน

ตัวเลข วันที่ และสถานะทุกรายการตรวจสอบย้อนกลับได้จากประวัติ git ตั๋วงาน หรือผลการรันชุดทดสอบ งานที่ยังไม่เสร็จระบุว่ายังไม่เสร็จ คำว่า "ผ่านการรวมโค้ด" (merged) หมายถึงคำขอรวมโค้ด (pull request: PR) ถูกรวมเข้าสาขา `main` แล้ว และ "ปิดตั๋ว" หมายถึงตั๋วงานมีสถานะ closed บน GitHub

4.1 ผลการย้ายระบบเดิมสู่ Flutter

4.1.1 ลำดับความก้าวหน้าของการย้ายระบบ ขั้น 0 ถึง ขั้น 6

การย้ายระบบแบ่งเป็นขั้น 0 ถึง 6 (ไม่ใช่เฟส 1–2 ของเซิร์ฟเวอร์) ผลที่ตรวจสอบได้จากประวัติ git แสดงในตารางที่ 4.1 งานช่วงแรกเกิดขึ้นในวันเดียวคือ 23 มิถุนายน พ.ศ. 2569 เพราะโครงสร้างและข้อกำหนดถูกเขียนไว้ล่วงหน้าในเอกสารสัญญา `CONTRACT.md` ซึ่งระบุตารางลายเซ็นของ repository เส้นทาง และความเป็นเจ้าของหน้าจอย่อย

ตารางที่ 4.1 ลำดับการย้ายระบบเดิมสู่ Flutter (ตรวจสอบจากประวัติ git)

วันที่	งาน	หลักฐาน (commit)
23 มิ.ย. 2569	ขั้น 0: สร้างโครงโปรเจกต์ Flutter	`5fbe8163`
23 มิ.ย. 2569	ขั้น 1: ชั้นข้อมูลด้วย Drift และ repository แบบธุรกรรม พร้อมการทดสอบหน่วย	`ea9a9778`
23 มิ.ย. 2569	ขั้น 3–6: หน้าจอทั้งหมดในรอบแรก ชุดวิดเจ็ตกลาง เมนูนำทาง และชั้นเปิดปิดกะ	`8eac3bfb`
23–24 มิ.ย. 2569	แก้ข้อค้นพบจากการตรวจสอบเชิงวิพากษ์ และเชื่อมฐานข้อมูล Drift บนเว็บ	`453bc3f5`, `621f86c1`, `6daf8d20`
25–26 มิ.ย. 2569	ปรับหน้าจอสำหรับแท็บเล็ตและมือถือ และคู่มือผู้ใช้พร้อมภาพหน้าจอจริง	`11d43adb`, `594d75db`, `bca4dc1a`
2–10 ก.ค. 2569	ตัวปรับขนาดตัวอักษร ฟอนต์ไทยในใบเสนอราคา A4 และการปรับโครงสร้างโค้ดพร้อมการทดสอบ repository	`a110a672`, `19d3191c`, `279b90ac`
14 ก.ค. 2569	ย้ายการจัดการสถานะจาก Riverpod เป็น flutter_bloc	`d8344503`

การตรวจสอบเชิงวิพากษ์ (adversarial scrutiny) หลังขั้น 6 พบข้อบกพร่องหลายประเด็น เช่น การส่งออกข้อมูลบนเว็บ รายงานปิดร้านที่ไม่ปรับตามขนาดหน้าจอ และปัญหาข้อความล้นกรอบ (overflow) บนจอโทรศัพท์ เมื่อแก้แล้วระบบผ่าน `dart analyze` โดยไม่มีข้อผิดพลาดและชุดทดสอบผ่านทั้งหมด ก่อนเริ่มงานเชื่อมต่อกับเซิร์ฟเวอร์

การย้ายไลบรารีจัดการสถานะจาก Riverpod เป็น flutter_bloc ดำเนินการครบ 12 ขั้นตอนตามแผน เนื่องจากชั้น repository ไม่ได้ผูกกับ Riverpod การย้ายจึงกระทบเฉพาะชั้นการแสดงผล ขณะนั้นชุดทดสอบ 122 กรณีผ่านทั้งหมด

4.1.2 การนำเข้าข้อมูลสำรองและฐานข้อมูลบนเว็บ

`importLegacyBackup()` นำเข้าไฟล์ JSON ที่ส่งออกจากระบบ JavaScript เดิมในธุรกรรมเดียว พร้อมแปลงข้อมูลโซนเป็นหมวดหมู่ ร้านจึงย้ายข้อมูลเดิมได้โดยไม่ต้องป้อนข้อมูลใหม่

บนบิลด์ที่ใช้เซิร์ฟเวอร์ยังมี `purgeDemoSeed()` ที่ลบข้อมูลตัวอย่าง (seed data) ซึ่งระบบสร้างไว้ตอนเริ่มต้นออกหนึ่งครั้ง โดยข้ามเมื่อยังมีงานค้างในคิวส่ง และลบเฉพาะแถวตัวอย่างที่ไม่เคยถูกแก้ไขและไม่มีระเบียนใดอ้างอิง

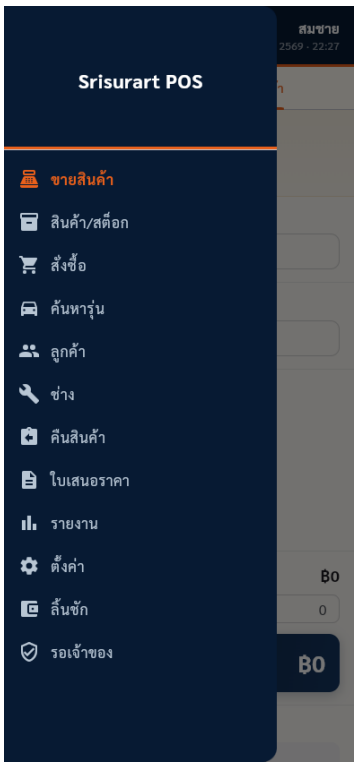
ฐานข้อมูลบน Flutter Web ทำงานผ่าน SQLite ที่คอมไพล์เป็น WebAssembly [5] ปัญหาที่พบคือเบราว์เซอร์ที่ไม่รองรับความสามารถ `dedicatedWorkersInSharedWorkers` เกิดข้อผิดพลาด `LinkError` เมื่อ Drift ถอยไปใช้โหมดสำรอง (#266) ซึ่งแก้ด้วย PR #310 (รวมโค้ด 17 กันยายน พ.ศ. 2569 ปิดตั๋ว 19 กันยายน พ.ศ. 2569)

4.2 หน้าจอของระบบ

ไคลเอนต์มีไฟล์หน้าจอ 14 ไฟล์ตามตารางที่ 3.5 รวมโค้ดหน้าจอประมาณ 20,800 บรรทัด (ผลรวมจาก `wc -l`) ภาพประกอบในหัวข้อนี้มาจากคู่มือผู้ใช้ `docs/tutorial/sri-pos-manual/` ซึ่งถ่ายจากแอปที่ทำงานจริงบนร้านสาธิต บางภาพมีข้อมูลตัวอย่างของนักพัฒนา และบางภาพถ่ายก่อนการแก้ข้อบกพร่องซึ่งระบุไว้ใต้ภาพนั้น

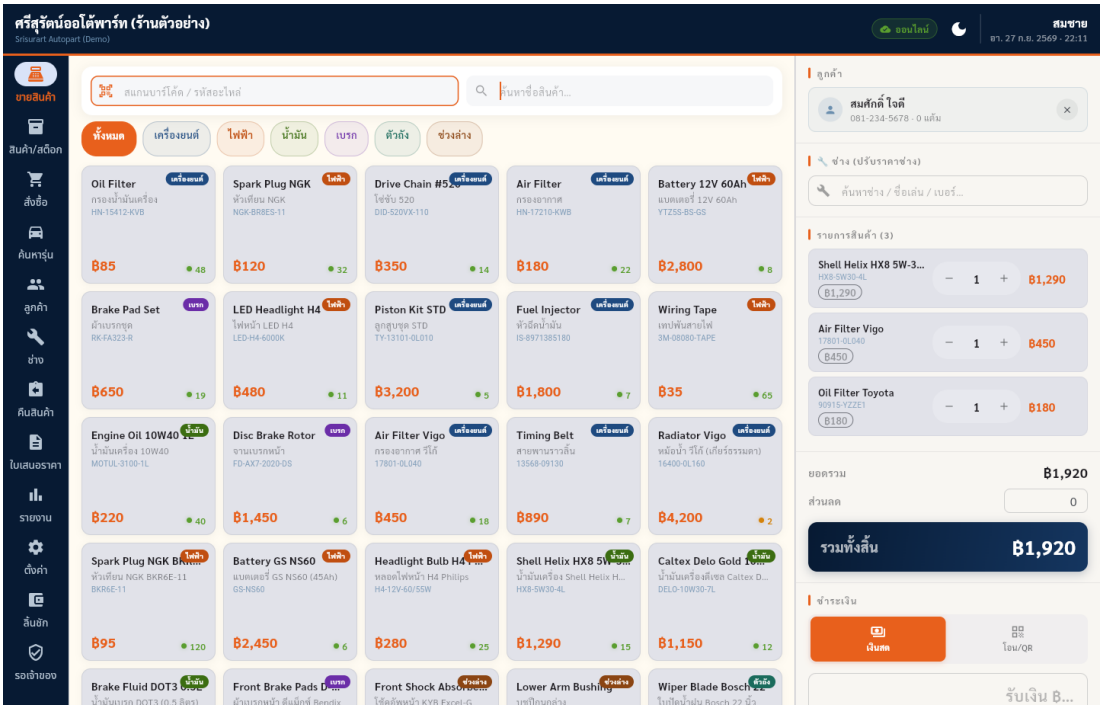
4.2.1 เมนูนำทางและหน้าขายสินค้า

เมนูหลัก 12 หน้าอยู่ด้านซ้ายบนจอกว้าง และซ่อนในปุ่มเมนูบนจอแคบ ดังรูปที่ 4.1 บนโทรศัพท์ หน้าขายสินค้ายังแยกเป็นแท็บ "สินค้า" และ "ตะกร้า" เพื่อให้ใช้ด้วยมือเดียวได้

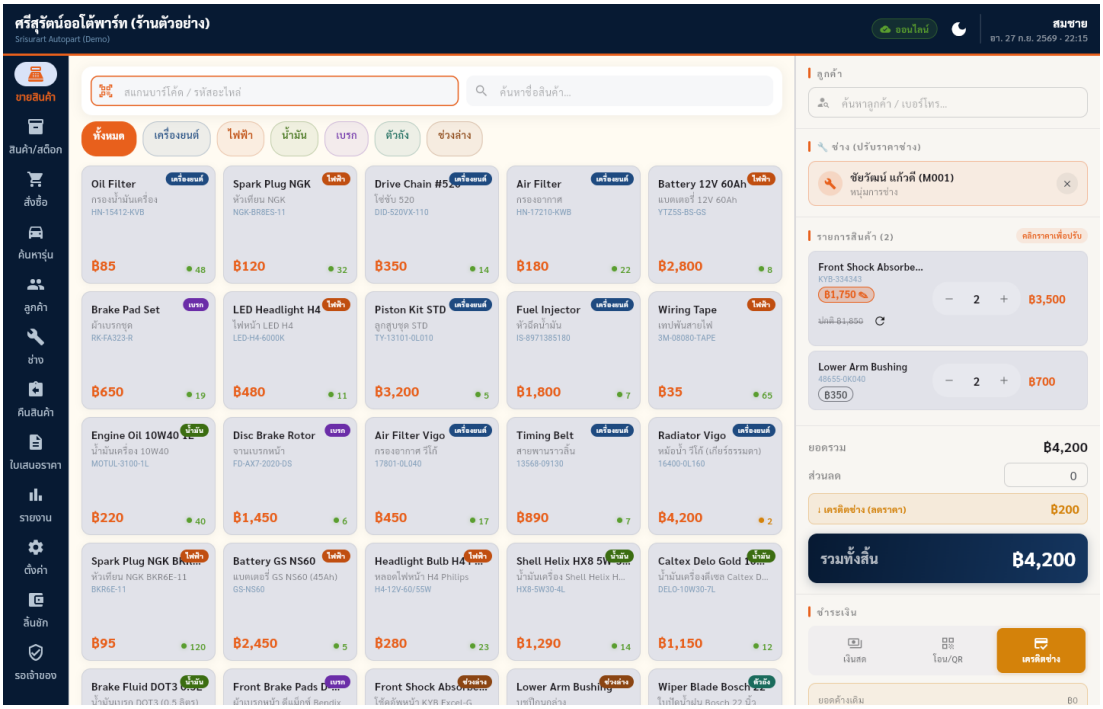


รูปที่ 4.1 เมนูนำทางบนโทรศัพท์ เรียงครบ 12 หน้า

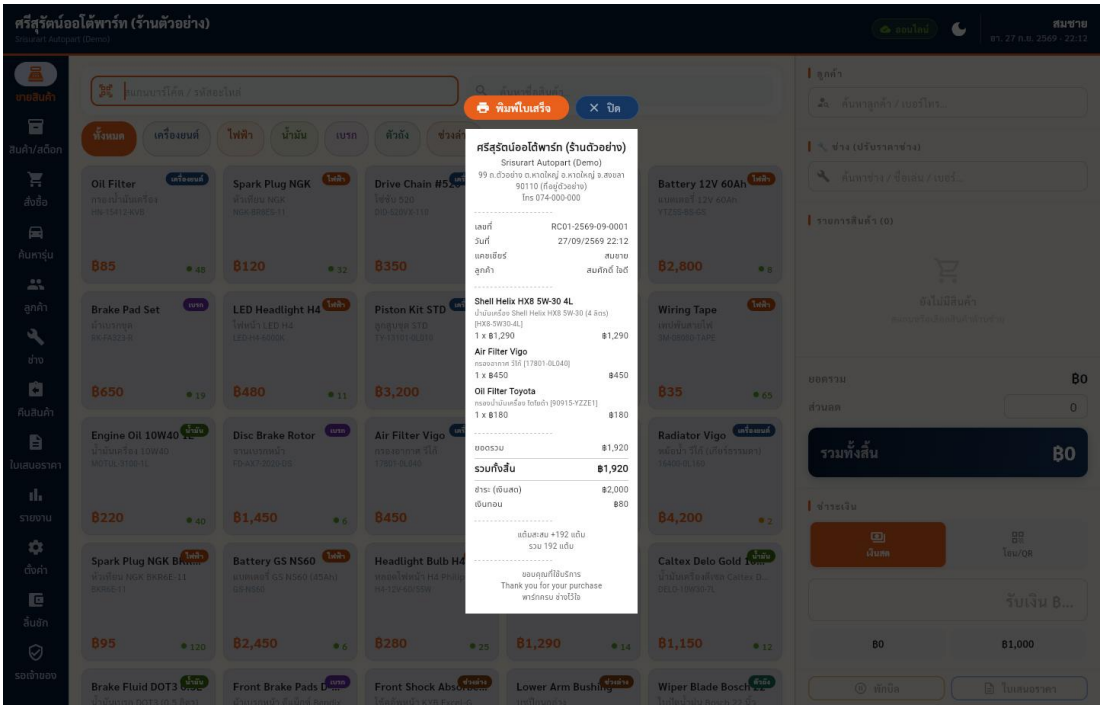
หน้าขายสินค้า (`checkout_screen.dart` ขนาด 2,956 บรรทัด) เป็นหน้าที่พนักงานใช้ทุกวัน แบ่งเป็นรายการสินค้าตรงกลาง ตะกร้าและการชำระเงินด้านขวา และป้ายสถานะการเชื่อมต่อมุมขวาบน ดังรูปที่ 4.2 พนักงานค้นสินค้าด้วยชื่อ รหัส หรือบาร์โค้ด ปรับจำนวน เลือกลูกค้าหรือช่าง และใช้ปุ่มลัดรับเงินสำหรับธนบัตรที่ใช้บ่อย เมื่อเลือกช่าง ระบบเปลี่ยนเป็นราคาช่างและเปิดวิธีชำระ "เครดิตช่าง" ดังรูปที่ 4.3 ใบเสร็จเงินสดแสดงเงินทอนและแต้มสะสม ดังรูปที่ 4.4 ส่วนฟังก์ชันพักบิลให้พนักงานเก็บตะกร้าไว้ชั่วคราวโดยไม่ตัดสต็อก



รูปที่ 4.2 หน้าขายสินค้า แสดงเมนู รายการสินค้า และตะกร้า



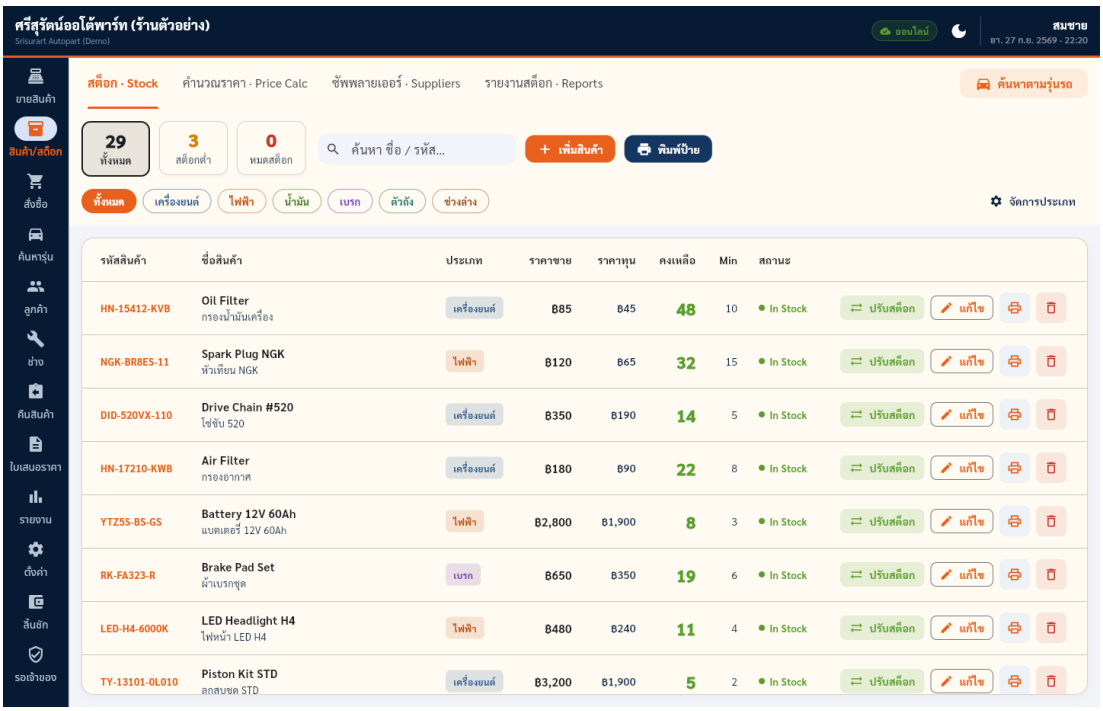
รูปที่ 4.3 การเลือกช่าง ราคาช่าง และการชำระด้วยเครดิตช่าง



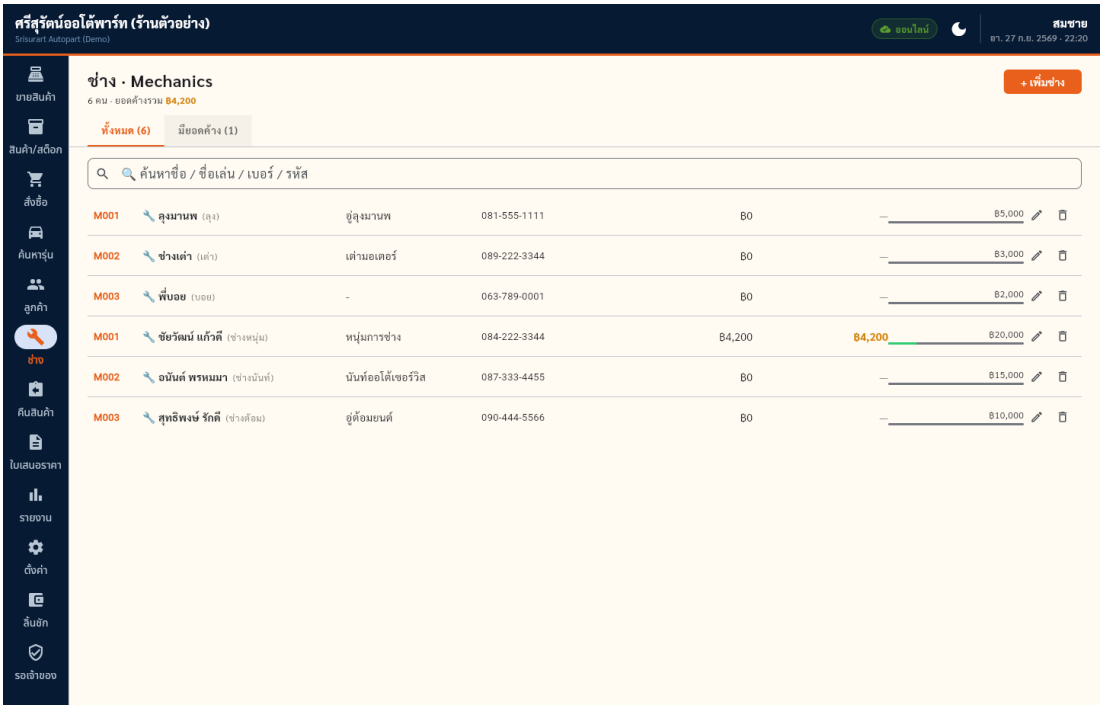
รูปที่ 4.4 ใบเสร็จเงินสด แสดงเงินทอนและแต้มสะสมของลูกค้า

4.2.2 หน้าสินค้า ช่าง และใบเสนอราคา

หน้าสินค้าและสต็อก (`products_screen.dart` ขนาด 3,843 บรรทัด) รวมแท็บสต็อก คำนวณราคา ซัพพลายเออร์ และรายงานสต็อก ดังรูปที่ 4.5 การปรับสต็อกด้วยมือแสดง "สต็อกหลังปรับ" ก่อนบันทึก และมีหน้าต่างพิมพ์ป้ายราคาที่มีบาร์โค้ด Code128 หน้าช่าง (`mechanics_screen.dart`) แสดงส่วนลดราคาช่าง ยอดค้างเครดิตเทียบวงเงิน และการรับชำระหนี้ ดังรูปที่ 4.6 ส่วนหน้าค้นหาตามรุ่นรถให้พนักงานค้นอะไหล่ด้วยชื่อรุ่น เพราะลูกค้ามักถามด้วยรุ่นรถมากกว่ารหัสสินค้า



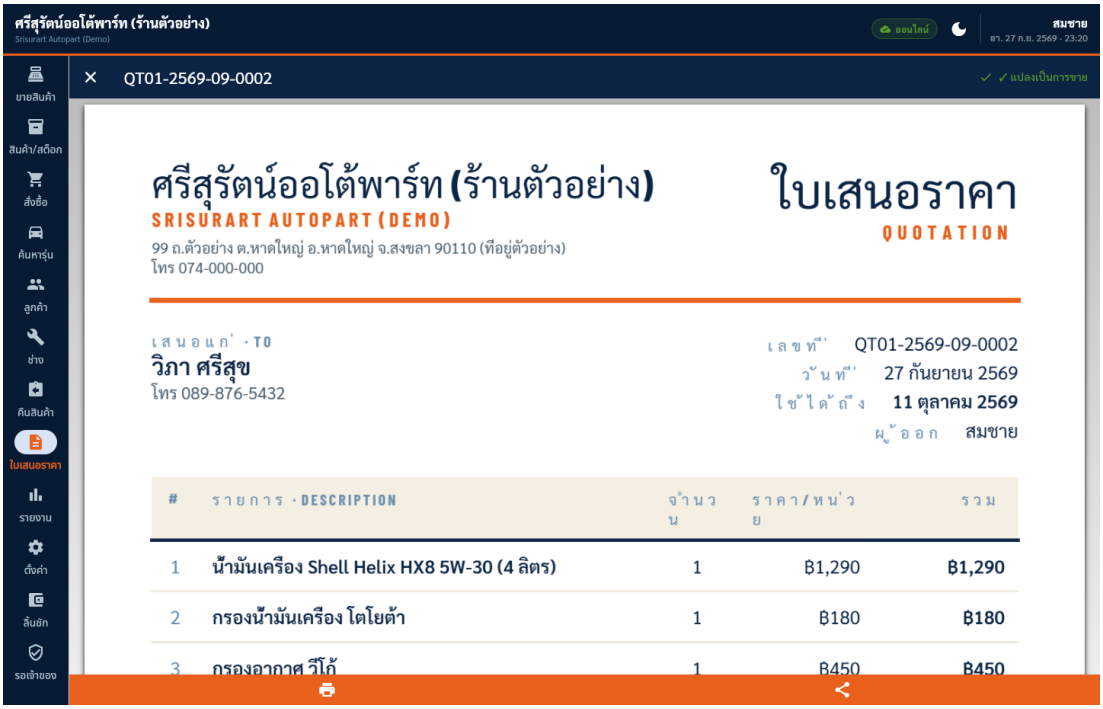
รูปที่ 4.5 หน้าสินค้า/สต็อก แท็บสต็อก



รูปที่ 4.6 หน้าช่าง แสดงยอดค้างและวงเงินเครดิตของช่างแต่ละราย

หน้าใบเสนอราคา (`quotes_screen.dart`) กรองตามสถานะ (ยังใช้ได้ หมดอายุ แปลงแล้ว) และออกเอกสาร A4 ที่มีหัวร้าน รายการสินค้า เงื่อนไข และช่องลายมือชื่อ ดังรูปที่ 4.7 จุดที่

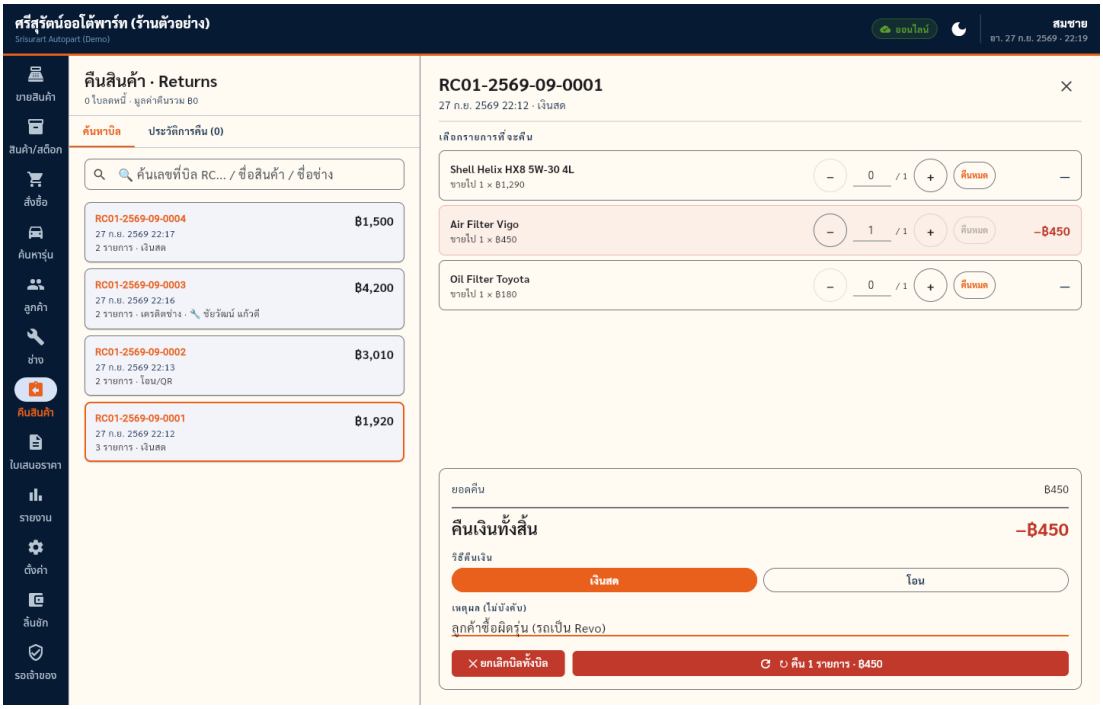
แก้ไขสำคัญคือการไม่ใช้ระยะห่างตัวอักษรกับข้อความไทย เพราะทำให้สระและวรรณยุกต์หลุดจากพยัญชนะ (#479, PR #482) ภาพนี้ถ่ายก่อนการแก้ จึงยังเห็นป้ายกำกับที่วรรณยุกต์หลุด



รูปที่ 4.7 ใบเสนอราคา A4 ส่วนหัวและรายการสินค้า (ถ่ายก่อนแก้ #479)

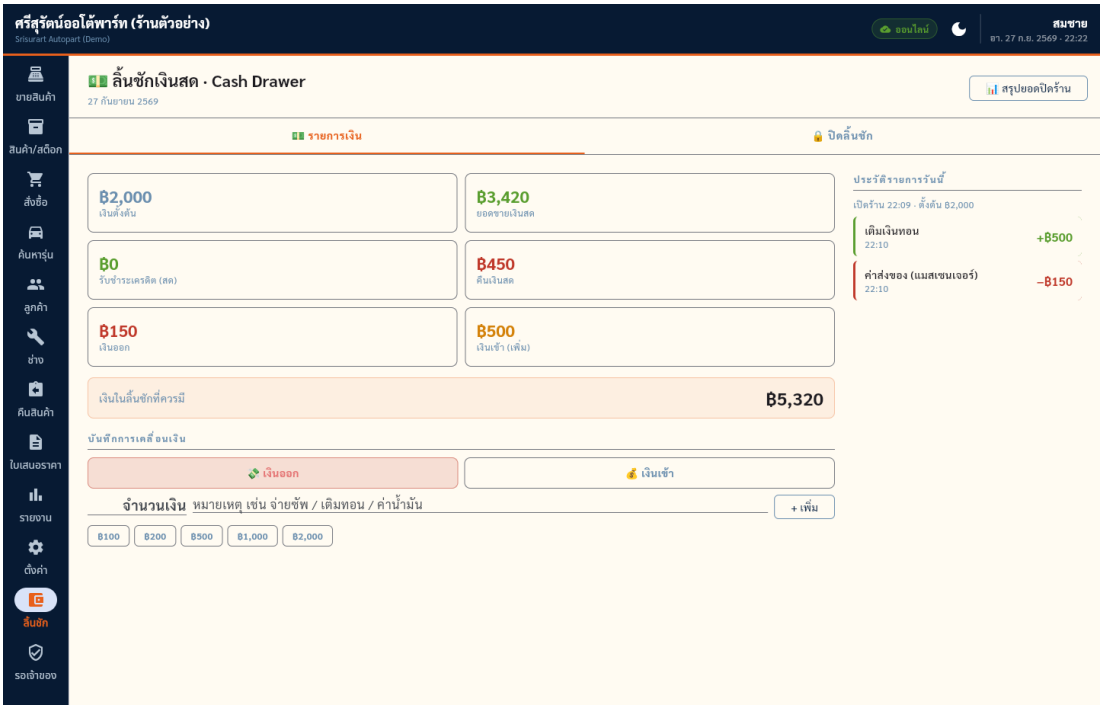
4.2.3 หน้าคืนสินค้า ลิ้นชัก และรายงานปิดร้าน

หน้าคืนสินค้า (`returns_screen.dart` ขนาด 2,057 บรรทัด) ให้ค้นบิลเดิม เลือกรายการคืน และออกใบลดหนี้ตามกติกาในตารางที่ 3.4 พร้อมแท็บประวัติการคืนที่อ้างเลข CN เลขบิล RC และเหตุผล ดังรูปที่ 4.8

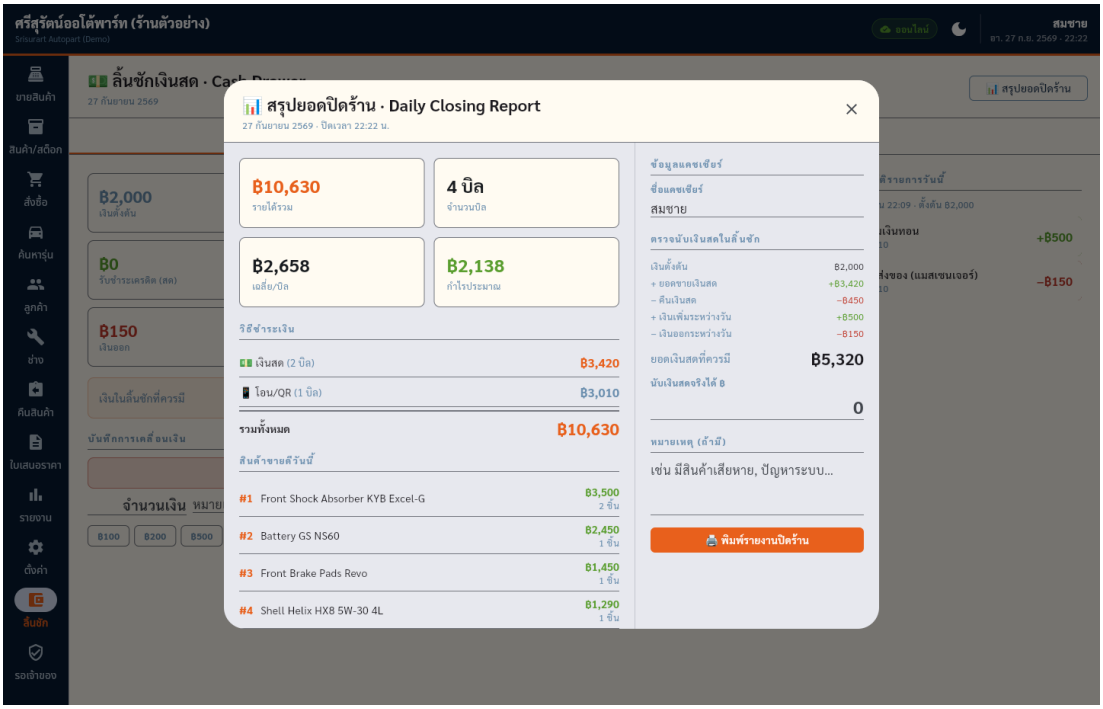


รูปที่ 4.8 หน้าคืนสินค้า

หน้าลิ้นชัก (`cash_drawer_screen.dart`) บังคับให้เปิดกะพร้อมระบุเงินตั้งต้นก่อนรับเงิน จากนั้นบันทึกเงินเข้าและออก และสรุปเงินที่ควรมีในลิ้นชักในการ์ดหกช่อง ดังรูปที่ 4.9 ภายหลังมีการแก้ให้หนึ่งวันมีได้หลายกะ (#453, PR #456) และให้ยอดเงินสดที่คาดหวังนับตามกะด้วยฟังก์ชันเดียวกันทุกที่ (PR #584 แทนกติกาหน้าต่างเวลาของ #452) ผลการทดสอบใช้งานบนเครื่องสาธิตและการแก้ไขหน้าจอนี้เพิ่มเติมสรุปในหัวข้อถัดไป รายงานปิดร้านรวบรวมรายได้ วิธีชำระ สินค้าขายดี และผลตรวจนับเงินในลิ้นชัก ดังรูปที่ 4.10 ภาพนี้ถ่ายก่อนการแก้ #461 ซึ่งเพิ่มแถวเครดิตช่างในรายงาน



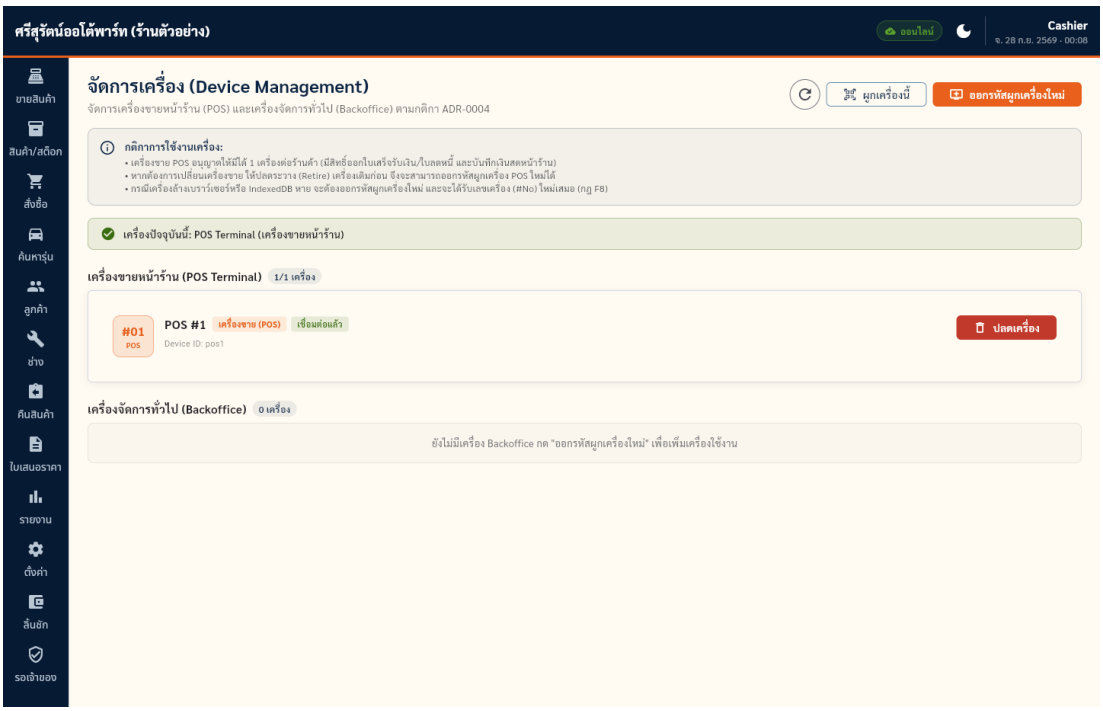
รูปที่ 4.9 แท็บรายการเงินและการ์ดสรุปเงินในลิ้นชัก



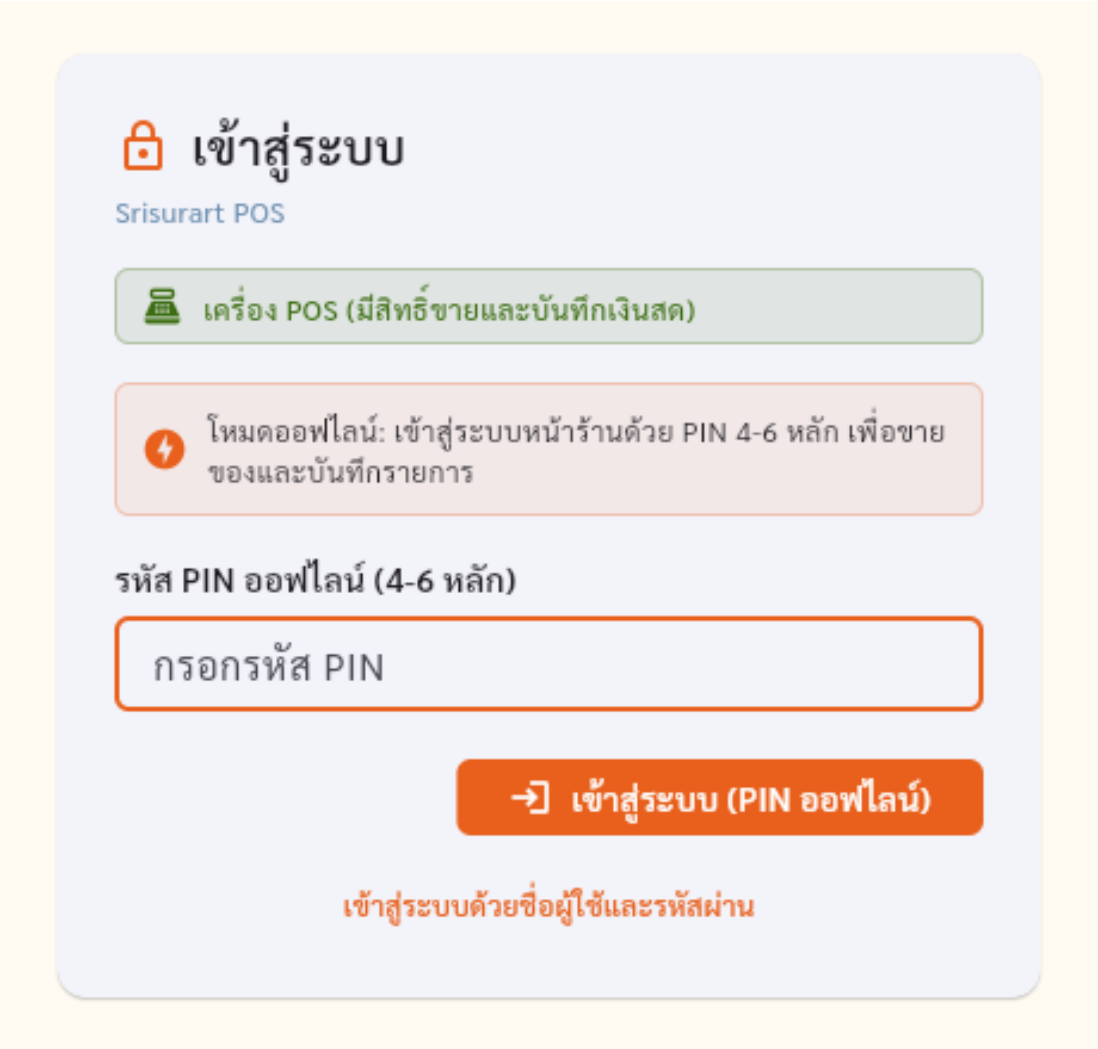
รูปที่ 4.10 รายงานปิดร้าน (Daily Closing Report, ถ่ายก่อนแก้ #461)

4.2.4 หน้าจัดการเครื่อง การเข้าสู่ระบบ และสถานะการเชื่อมต่อ

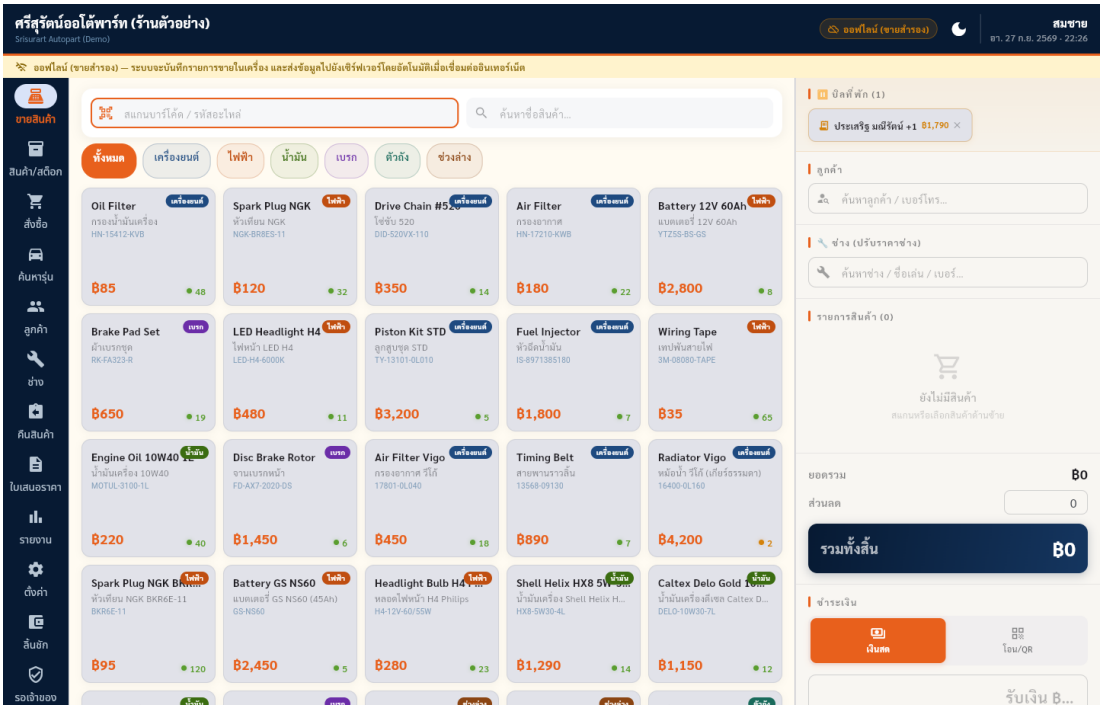
หน้าจัดการเครื่อง (`devices_screen.dart`) ใช้บนบิลด์ที่มีเซิร์ฟเวอร์ เจ้าของร้านผูกเครื่องขายและเครื่องหลังร้านแยกตามบทบาทตาม ADR-0004 ดังรูปที่ 4.11 โดยกด "ออกรหัสผูกเครื่องใหม่" ได้รหัส 8 อักขระ แล้วนำไปกรอกบนเครื่องที่จะผูก ผู้ใช้เข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่าน หรือด้วย PIN ออฟไลน์บนเครื่องขายที่ผูกไว้เมื่อติดต่อเซิร์ฟเวอร์ไม่ได้ ดังรูปที่ 4.12 ป้ายสถานะมุมขวาบนบอกว่าออนไลน์หรือออฟไลน์ และมีแถบเหลืองเตือนเมื่อออฟไลน์ ดังรูปที่ 4.13 ส่วนร้านที่ยังใช้บิลด์ Drift อย่างเดียวไม่มีหน้าเข้าสู่ระบบและไม่ต้องผูกเครื่อง



รูปที่ 4.11 หน้าจัดการเครื่อง แยกเครื่องขาย (POS Terminal) และเครื่องจัดการทั่วไป (Backoffice)



รูปที่ 4.12 ฟอร์มเข้าสู่ระบบด้วย PIN ออฟไลน์บนเครื่องขาย



รูปที่ 4.13 สถานะออฟไลน์ ป้ายขายสำรองและแถบเตือนสีเหลือง

4.2.5 การทดสอบใช้งานบนเครื่องสาธิตและการแก้ไข (3–4 ตุลาคม พ.ศ. 2569)

ระหว่างวันที่ 3–4 ตุลาคม พ.ศ. 2569 ทีมทดสอบการใช้งานผ่านหน้าจอจริงบน `mob04` (`https://172.30.58.20`) ข้อบกพร่องที่พบถูกแก้เป็น PR ขนาดเล็กทีละเรื่อง พร้อมไฟล์ทดสอบที่ล้มเหลวก่อนแก้และผ่านหลังแก้ ดังตารางที่ 4.2 การแก้ทั้งหมดอยู่ฝั่งไคลเอนต์ ยกเว้นการปฏิเสธการจ่ายเงินออกเกินยอดในลิ้นชักและรายการรอตรวจของกรณีออฟไลน์ ซึ่งบังคับที่เซิร์ฟเวอร์ด้วย

ตารางที่ 4.2 ข้อค้นพบจากการทดสอบใช้งานบนเครื่องสาธิตและการแก้ไข

เรื่อง	ปัญหาที่พบ	การแก้ไข	PR
จ่ายเงินออกจากลิ้นชัก	จ่ายออกได้มากกว่าเงินที่ควรมีในลิ้นชัก	ปฏิเสธด้วย `409 DRAWER_INSUFFICIENT_CASH` ที่คำนวณใต้ล็อกของกะ และนับยอดที่คาดหวังตามกะ ส่วนการจ่ายเกินที่เกิดขณะออฟไลน์ไม่ถูกปฏิเสธเมื่อเล่นซ้ำ แต่สร้างรายการรอตรวจ `เงินออกจากลิ้นชักเกินยอดตอนออฟไลน์` (ข้อความที่เจ้าของโครงงานรับรองเมื่อ 3 ตุลาคม พ.ศ. 2569)	#580, #584, #585, #587
ช่องจำนวนเงินในหน้าลิ้นชัก	ช่อง "นับเงินสดจริงได้" รับตัวอักษร เช่น `a400` ถูกอ่านเป็น 0 และแสดงเงินขาดเท่ากับยอดทั้งหมด ส่วน `.` ตัวเดียวเปิดปุ่มยืนยันปิดลิ้นชักได้	ช่องเงินทั้งสามช่องรับเฉพาะจำนวนไม่ติดลบที่มีทศนิยมไม่เกินสองตำแหน่ง และ `.` ตัวเดียวไม่นับเป็นยอด	#587, #590
ช่องจำนวนเงินในหน้าขาย	ช่องราคาต่อหน่วย ส่วนลด และรับเงินยังพิมพ์ `1.2.3` หรือ `1.234` ได้	ย้ายตัวกรองไปไว้ที่ `moneyInputFormatters` ใน `money_input_formatters.dart` ใช้ร่วมกันทั้งหน้าลิ้นชักและหน้าขาย	#591
กะที่เปิดข้ามเที่ยงคืน	กะที่เปิดตอน 20:00 และยังไม่ปิดหายจากหน้าลิ้นชักและรายงานปิดร้านหลังเที่ยงคืน และหน้าจอเสนอให้เปิดกะใหม่ซึ่งจะเก็บกะเดิมที่ยังไม่นับเงิน	ฟังก์ชันเดียว `ShiftsRepository.currentShiftOf` ถือว่ากะที่ยังเปิดเป็นกะปัจจุบันเสมอ และกะที่ปิดแล้วเป็นกะปัจจุบันเมื่อเปิดหรือปิดในวันนี้	#592

เรื่อง	ปัญหาที่พบ	การแก้ไข	PR
ชื่อสินค้า	บันทึกสินค้าที่กรอกเฉพาะชื่อภาษาไทยไม่ได้	ใช้ชื่อภาษาไทยแทนชื่อภาษาอังกฤษเมื่อช่องภาษาอังกฤษว่าง (คอลัมน์ `name` ของเซิร์ฟเวอร์ต้องไม่ว่าง) และเมื่อเปิดแก้ไขจะแสดงช่องภาษาอังกฤษว่างตามเดิม	#593, #595
ส่วนลดท้ายบิล	กรอกได้เฉพาะจำนวนบาท (เจ้าของโครงงานขอให้กรอกเป็นร้อยละได้)	ปุ่มสลับ `฿/%` ข้างช่องส่วนลด ร้อยละรับค่า 0–100 แล้วแปลงเป็นบาทด้วย `round2` และไม่เกินยอดรวม ข้อมูลที่บันทึกยังเป็นจำนวนบาทเช่นเดิม จึงไม่กระทบเซิร์ฟเวอร์ ข้อความคำแนะนำสองข้อความ `ส่วนลดเป็นบาท` และ `ส่วนลดเป็นเปอร์เซ็นต์` ได้รับการรับรองเมื่อ 4 ตุลาคม พ.ศ. 2569	#594

ข้อจำกัดของการแก้ไขชุดนี้ระบุไว้ใน PR เอง ได้แก่ บิลที่พักหรือบันทึกเป็นใบเสนอราคาขณะใช้ส่วนลดแบบร้อยละจะกลับมาเป็นส่วนลดจำนวนบาทคงที่ และหัวข้อ "ประวัติรายการวันนี้" กับวันที่บนหัวรายงานยังเขียนว่า "วันนี้" สำหรับกะที่ข้ามเที่ยงคืน เพราะการแก้ต้องเพิ่มข้อความภาษาไทยใหม่ คู่มือพนักงานใน `docs/tutorial/sri-pos-manual/` ปรับตามการเปลี่ยนแปลงเรื่องลิ้นชัก ช่าง และการเข้าสู่ระบบใน PR #586 และตามส่วนลดแบบร้อยละใน PR #594

ในวันที่ 3 ตุลาคม พ.ศ. 2569 ยังมี PR ฝั่งแอปและแพลตฟอร์มที่รวมแล้วอีกหลายรายการ สรุปตามชื่อ PR ได้แก่ การกู้คืนข้อมูลสำรองด้วยการเลือกไฟล์ `.json` แทนการวาง (#556) และปิดการกู้คืนบนบิลด์ API (#566) การแทนที่เครื่องขายที่สูญหายจากระนาบแพลตฟอร์มและแบนเนอร์บอกเมื่อเซสชันไม่ใช่เครื่อง `pos` (#561, #567) ซึ่งทำให้ตั๋ว #476 ปิด การจบเซสชันเมื่อลงทะเบียนเครื่องขณะเข้าสู่ระบบอยู่ (#562) ตัวดูบันทึกตรวจสอบและการปรับหน้าจอ `platform-ui` (#563, #564, #570) การเพิ่มช่างที่ตอบ 400 และปุ่มบันทึกค้าง (#565) และการห้ามลบช่างที่ยังมียอดค้าง (#578) รายงานปิดร้านและรายงานยอดขายที่หักการคืนและไม่นับบิลที่ยกเลิกด้วยมือ (#

569, #571, #577) ข้อผิดพลาดของหน้าใบสั่งซื้อและใบเสนอราคา (#572, #573) การขายจากตะกร้าของใบเสนอราคาผ่าน `POST /sales` พร้อม `quoteId` ทั้งออนไลน์และออฟไลน์ (#574, #575, #576) และข้อความผิดพลาดภาษาไทยของการเข้าสู่ระบบ (#583) ผลการทดสอบอัตโนมัติฝั่งไคลเอนต์หลังการแก้ชุดนี้ PR #594 รายงานว่า `flutter test` ผ่าน 999 กรณี (ตัวเลขในหัวข้อผลการทดสอบด้านล่างเป็นค่าที่วัดเมื่อ 1 ตุลาคม พ.ศ. 2569)

4.2.6 ความเชื่อมโยงกับปัญหาของร้าน

ตารางที่ 4.3 โยงปัญหา P1–P8 ในบทที่ 1 กับฟังก์ชันที่ตอบปัญหานั้น โดยยังไม่ได้วัดผลกับผู้ใช้จริง

ตารางที่ 4.3 ปัญหาของร้านกับฟังก์ชันและหลักฐานในรายงาน

ปัญหา	ฟังก์ชันหรือหน้าจอที่ตอบ	หลักฐาน
P1 เขียนบิลด้วยมือ	หน้าขายสินค้า ใบเสร็จ และเลขที่ใบเสร็จอัตโนมัติ	รูปที่ 4.2, 4.4
P2 ไม่แน่ใจสต็อก	ตัดสต็อกแบบเข้มงวดเมื่อขาย และหน้าสต็อก	ตารางที่ 3.4, รูปที่ 4.5
P3 ราคาและต้นทุนต่างล็อต	ต้นทุนเฉลี่ยถ่วงน้ำหนักเมื่อรับสินค้า และแท็บคำนวณราคา	ตารางที่ 3.4
P4 ยอดรับ–จ่ายประจำวัน	หน้าลิ้นชักและรายงานปิดร้าน	รูปที่ 4.9, 4.10
P5 ยอดเครดิตช่าง	หน้าช่าง ยอดค้างเทียบวงเงิน และการยืนยันก่อนขายเกินวงเงิน	รูปที่ 4.6, 4.3
P6 ไม่ทราบกำไร	กำไรในรายงานปิดร้านคิดจากต้นทุน ณ เวลาขาย (ADR-0008, PR #408)	`closing_report.dart`
P7 ไม่ทราบยอดขาย	รายงานยอดขายในหน้ารายงาน	`reports_screen.dart`
P8 ไม่ทราบสินค้าขายดี	รายการสินค้าขายดีในหน้ารายงานและรายงานปิดร้าน	`reports_screen.dart`, `closing_report.dart`

4.3 ผลการพัฒนาชั้นเขียนข้อมูลผ่าน API

ชั้นเขียนข้อมูลผ่าน API สร้างตามแนวทางแคชแบบเขียนผ่านของ ADR-0010 (ยอมรับเมื่อ 4 กันยายน พ.ศ. 2569) และกติกาห้าข้อในหัวข้อ Repository แบบ API ของบทที่ 3 งานแบ่งเป็นสี่ขั้น

ซึ่งปิดตั๋วทั้งหมดแล้ว ดังตารางที่ 4.4 ตั๋วงานหลัก (parent issue) #52 ปิดเมื่อ 13 กันยายน พ.ศ. 2569

ตารางที่ 4.4 งานชั้นเชื่อมต่อ API ฝั่งไคลเอนต์ (fe.0 ถึง fe.3)

ขั้น	เนื้อหา	ตั๋ว	PR	ปิดตั๋ว
fe.0	Drift schema v3: `Sales.shiftId` และ `Shifts.id` เป็น TEXT	#53	#58	10 ก.ย. 2569
fe.1	การยืนยันตัวตนฝั่งไคลเอนต์ การเก็บโทเค็น การผูกเครื่อง และข้อความผิดพลาดจากเซิร์ฟเวอร์	#54	#81	13 ก.ย. 2569
fe.2	`ApiRepository` สำหรับการอ่าน: แค็ตตาล็อก ผู้คน การจัดซื้อ ใบเสนอราคา รายงาน	#55	#87 และ #128	14 ก.ย. 2569
fe.3	การเขียน: ขาย สินค้า คืนสินค้า ลิ้นชักเงินสด (`ApiSales`, `ApiReturns`, `ApiShifts`)	#56	#84	12 ก.ย. 2569

กติกาว่า "เฉพาะ 4xx เป็นคำตัดสิน" มีที่มาจากการทบทวน PR #84 ซึ่งพบว่าไคลเอนต์รุ่นแรกยังเขียนในเครื่องหลังได้ 5xx ทั้งที่เซิร์ฟเวอร์อาจบันทึกการเขียนนั้นไปแล้ว (commit `d5d22f8` และ `a5358d1`) ปัจจุบันกติกานี้ถูกบังคับด้วย `api_repository_contract_test.dart` ส่วนการเก็บโทเค็นบนเว็บตามภาคผนวกของ ADR-0009 สร้างเสร็จใน PR #404 (#400 ปิดตั๋ว 25 กันยายน พ.ศ. 2569) โดยโทเค็นที่บิลด์เก่าทิ้งไว้ใน `localStorage` ถูกย้ายครั้งเดียวด้วยลำดับเขียน อ่านกลับ แล้วจึงลบ และเมื่อ 1 ตุลาคม พ.ศ. 2569 PR #534 และ #536 เพิ่มรุ่นของเซสชันใน `ApiClient` เพื่อให้การต่ออายุหรือ 401 ที่ล่าช้าจากผู้ใช้คนก่อนไม่มีผลกับผู้ใช้คนถัดไป

4.4 ผลการพัฒนาระบบทำงานออฟไลน์ เฟส 2 ฝั่งอุปกรณ์

4.4.1 การแบ่งงานและตั๋วที่ปิดแล้ว

งานฝั่งอุปกรณ์ของเฟส 2 ได้แก่ เว็บแอปแบบก้าวหน้า (Progressive Web App: PWA) [40] คิว `outbox_ops` ตัว `SyncService` การออกเลขเอกสาร PIN ออฟไลน์ และการดึงข้อมูล เป็นความรับผิดชอบของเลน B ตามแผน ในการลงมือจริง PR ส่วนหนึ่งเขียนโดยสมาชิกเลน C และงานแก้ข้อบกพร่องหลัง 27 กันยายนเขียนโดยสมาชิกเลน A ดังคอลัมน์ผู้เขียนในตารางที่ 4.5 ส่วนเลน A ส่งมอบสามตั๋วเพื่อปลดล็อกเลนอื่นเมื่อ 17 กันยายน พ.ศ. 2569 คือข้อความภาษาไทยของเฟส 2 (#268, PR #305) อินเทอร์เฟซ `SyncFacade` พร้อมไฟล์ตัวอย่างคำขอ `/sync/push` 18 ไฟล์ (#269, PR #307) และการจำกัดเส้นทางผู้ดูแลแพลตฟอร์ม (#270, PR #308) ตั๋วหลายรายการเป็น "ครึ่ง" ของคู่งานที่แบ่งฝั่งไคลเอนต์และเซิร์ฟเวอร์ ไคลเอนต์จึงทดสอบกับตัวจำลองเซิร์ฟเวอร์ ตารางที่ 4.5 สรุปตั๋วที่ปิดแล้ว (วันที่ปิดตรวจด้วย `gh issue view` เมื่อ 1 ตุลาคม พ.ศ. 2569)

ตารางที่ 4.5 ตั๋วเฟส 2 ฝั่งอุปกรณ์ที่ปิดแล้ว

ตั๋ว	งาน	PR/commit	ผู้เขียน	ปิดตั๋ว
#189	ห้ามออกเลขออฟไลน์เมื่อเครื่องยังไม่เคยเตรียมตัวนับ	`1dff9a9`	LomerAlloys	19 ก.ย. 2569
#273	PWA shell: service worker, persist, แท็บเดียว	`61143b3`	LomerAlloys	19 ก.ย. 2569
#228	`outbox_ops` และ `SyncService` พร้อมการบล็อกหัวคิวต่อ aggregate	PR #324	LomerAlloys	19 ก.ย. 2569
#230	หน้ารายการรอเจ้าของร้านตรวจ 2 แท็บ และการทิ้งรายการ	PR #318	PattaraponKitcharoen	19 ก.ย. 2569
#275	ย้ายการชำระเครดิตเข้า `outbox_ops`	PR #329	PattaraponKitcharoen	19 ก.ย. 2569

ตั๋ว	งาน	PR/commit	ผู้เขียน	ปิดตั๋ว
#194	ส่งความยินยอมขายเกินวงเงินในคิวขายออฟไลน์	PR #327, #328	LomerAlloys, PattaraponKitcharoen	19 ก.ย. 2569
#229	ลูกค้าเข้าคิวได้ ปิดปุ่มที่เป็นออนไลน์เท่านั้น	PR #330	PattaraponKitcharoen	19 ก.ย. 2569
#211	PIN ออฟไลน์บนเครื่อง `pos`	PR #332	PattaraponKitcharoen	19 ก.ย. 2569
#193	contract test ของ `SyncService` กับตัวจำลองเซิร์ฟเวอร์	PR #331	PattaraponKitcharoen	19 ก.ย. 2569
#276	ยกเลิกบิลออฟไลน์ได้เฉพาะบิลที่ขายออฟไลน์ พร้อมเหตุผล	PR #334	PattaraponKitcharoen	20 ก.ย. 2569
#212	ดึงข้อมูลด้วย cursor ของเซิร์ฟเวอร์ ถอยย้อน 30 วินาที ไม่ทับสต็อกใต้งานค้าง	PR #347	PattaraponKitcharoen	20 ก.ย. 2569
#452, #453	ปิดกะต้องให้คิวว่าง หลายกะต่อวัน และคิว `shift.open`, `drawer.entry`, `return.create`	PR #456, #469	NuimanLP	27 ก.ย. 2569
#472, #475	เลข RC ออฟไลน์ใช้กติกาเดียวกับ CN และเตรียมตัวนับให้เครื่องใหม่	PR #484	NuimanLP	28 ก.ย. 2569

ตั๋ว	งาน	PR/commit	ผู้เขียน	ปิดตั๋ว
#489, #490	บันทึกเลขที่เซิร์ฟเวอร์ออกลงตัวนับในเครื่อง; ขึ้นเดือนใหม่เริ่ม 0001	PR #491, #492, #494	NuimanLP	28 ก.ย. 2569
#488	การทิ้งรายการย้อนผลตรงตามค่าที่ใช้จริง (`op_effects`, schema v13)	PR #495	NuimanLP	28 ก.ย. 2569

4.4.2 ผลเพิ่มเติมที่เกิดจากการแก้ข้อบกพร่อง

งานหลังจากตารางข้างต้นเกิดจากข้อบกพร่องที่พบระหว่างทาง ดังนี้

- **การทิ้งรายการในคิว (#473, #488)** การทิ้งรายการขายหรือคืนที่ค้างเดิมไม่ย้อนสต็อกและยอดในเครื่อง PR #483 จึงย้อนสต็อก ลูกหนี้ และสถานะยกเลิกบิลในธุรกรรมเดียวกับการทิ้ง แต่การคำนวณย้อนไม่แม่นยำเมื่อการเขียนครั้งแรกถูกปัดค่าที่ศูนย์ (เช่น เครดิตช่าง 50 บาทกับใบลดหนี้หักเครดิต 180 บาท) PR #495 จึงเพิ่มตาราง `op_effects` ที่เก็บผลต่างที่ใช้จริง และปฏิเสธการทิ้งเมื่อสต็อกที่เกี่ยวข้องถูกขายต่อไปแล้วตามการตัดสินของเจ้าของโครงงาน (รายละเอียดในบทที่ 5)
- **เลขเอกสารซ้ำ (#489)** เลขที่เซิร์ฟเวอร์ออกไม่ถูกบันทึกลงตัวนับในเครื่อง แก้แล้วตามตาราง (บทที่ 5)
- **การดึงข้อมูลกับยอดเงินในคิว (PR #544, 1 ตุลาคม พ.ศ. 2569)** การดึงข้อมูลต้องไม่ทับยอดลูกค้าและช่างที่คำสั่งทางการเงินในคิวยังอ้างอิงอยู่ จึงคุ้มครองเฉพาะคำสั่งที่ย้ายเงินสี่ชนิด (`sale.create`, `sale.void_offline`, `return.create`, `credit_payment.create`)
- **แคชหลายร้าน (PR #539 และ #541, 1 ตุลาคม พ.ศ. 2569)** เครื่องหนึ่งเครื่องอาจถูกใช้กับหลายร้าน จึงผูกแคชในเครื่องกับรหัสร้านและปฏิเสธการสลับร้านเมื่อยังมีงานที่ไม่ได้ส่ง ส่วน PR #541 แก้กรณีบิลด์เก่าที่ยังไม่มีรหัสร้าน ข้อจำกัดที่ยอมรับแล้วคือคำตอบของการเขียนออนไลน์ที่กำลังส่งอยู่ไม่ถูกกั้นด้วยรุ่นของแคช

4.4.3 งานที่ยังไม่เสร็จของฝั่งอุปกรณ์

งานที่ยังเปิดอยู่ ได้แก่ การย้ายร้านจากบิลด์ Drift ไปใช้เซิร์ฟเวอร์ (#231) การสาธิตครบวงจรบน `mob04` (#344) ส่วนข้อความภาษาไทยสามรายการจาก PR #469 และ #470 ที่เคยเป็นร่าง ได้รับการรับรองจากเจ้าของโครงงานแล้วเมื่อ 7 ตุลาคม พ.ศ. 2569 พร้อมข้อความร่างอื่นทั้งหมด (PR #649)

4.5 ผลการทดสอบฝั่งไคลเอนต์

โฟลเดอร์ `frontend/test/` มีไฟล์ทดสอบ (`*_test.dart`) 90 ไฟล์ การรัน `flutter test` เมื่อ 1 ตุลาคม พ.ศ. 2569 รายงานผล "All tests passed!" ที่ 820 กรณี ซึ่งมากกว่าจำนวนจุดนิยาม `test(` และ `testWidgets(` ในซอร์ส (741 จุด) เพราะบางกรณีสร้างในลูปแบบพารามิเตอร์ ส่วน `dart analyze` รายงาน "No issues found!" (ใช้ `dart analyze` แทน `flutter analyze` ด้วยเหตุผลในบทที่ 5)

ชุดทดสอบแบ่งเป็นสี่ระดับ ระดับแรกคือการทดสอบหน่วยของกฎธุรกรรมแต่ละข้อในตารางที่ 3.4 บนฐานข้อมูลในหน่วยความจำ ระดับที่สองคือการทดสอบสัญญา (contract test) ที่บังคับกฎสถาปัตยกรรม เช่น `api_repository_contract_test.dart` และการทดสอบ `SyncService` กับไฟล์ตัวอย่างของ `/sync/push` ระดับที่สามคือการทดสอบวิดเจ็ต เช่น ความกว้างหน้าจอโทรศัพท์ 390 พิกเซล แถบสถานะการซิงก์ และการเปลี่ยนเส้นทางเมื่อเข้าและออกจากระบบ ระดับสุดท้ายคือการทดสอบเส้นทางพื้นฐาน (route smoke test) ของทุกหน้า

ข้อจำกัดของผลนี้มีสองประการ ประการแรก ผลการรันมีข้อความเตือนเรื่องที่เก็บโทเค็นใช้ไม่ได้ ซึ่งมาจากกรณีทดสอบที่จำลองความล้มเหลวของ IndexedDB โดยตั้งใจ ไม่ใช่ข้อผิดพลาด ประการที่สอง การทดสอบฝั่งอุปกรณ์ใช้ตัวจำลองเซิร์ฟเวอร์ ผลผ่านจึงยังไม่พิสูจน์ว่าไคลเอนต์ทำงานถูกต้องกับเซิร์ฟเวอร์จริงทั้งหมด การพิสูจน์ขั้นสุดท้ายคือการสาธิตครบวงจร (#344) ซึ่งยังไม่ได้ดำเนินการ

4.6 ผลการพัฒนาฝั่งเซิร์ฟเวอร์

4.6.1 ภาพรวมของฝั่งเซิร์ฟเวอร์ เฟส 1

ผลลัพธ์ของเฟส 1 คือเซิร์ฟเวอร์ NestJS [25] บน PostgreSQL ที่รับการเขียนข้อมูลธุรกรรมทั้งหมดของร้าน ได้แก่ การขาย การคืนสินค้า การรับสินค้าเข้าตามใบสั่งซื้อ และกะการขาย โดยย้ายกฎความถูกต้องของข้อมูลจากฝั่ง Dart มาบังคับที่ฝั่งเซิร์ฟเวอร์ตามโครงสร้างโมดูลในตารางที่ 3.6

ขนาดของโค้ดและชุดทดสอบสรุปไว้ในตารางที่ 4.13 ท้ายบท ส่วนตารางที่ 4.6 สรุปองค์ประกอบสำคัญคู่กับไฟล์ทดสอบที่เป็นหลักฐาน

ตารางที่ 4.6 สรุปองค์ประกอบสำคัญของฝั่งเซิร์ฟเวอร์และหลักฐานการพิสูจน์

องค์ประกอบ	หน้าที่	หลักฐาน
RLS และคีย์ผสม `(tenant_id, id)`	แยกข้อมูลรายร้านในระดับฐานข้อมูล	`cross-tenant-read.e2e-spec.ts` (25 ตาราง + 12 จุดปลายทาง HTTP)
`runTx` / `runIdempotent`	ธุรกรรมในตัวจัดการคำขอ และการกันการเขียนซ้ำ	สเปกสถาปัตยกรรม 3 ไฟล์, `tx-hold-measure.e2e-spec.ts`
เพดานเวลาคอมมิต 25 วินาที	กันธุรกรรมค้างยาว	`tx-ceiling.e2e-spec.ts`
บทบาทอุปกรณ์ `pos` / `backoffice`	จำกัดสิทธิ์การเขียนบิลตามเครื่อง	`security.e2e-spec.ts`
จำกัดอัตราต่อร้านบน Redis	กันร้านหนึ่งกินทรัพยากรร้านอื่น	`rate-limit.e2e-spec.ts`, `rate-limit-pool.e2e-spec.ts`
ระนาบแพลตฟอร์ม	สร้างร้านและจัดการเครื่องโดยไม่แตะ psql	`platform.e2e-spec.ts`, `platform-cli.e2e-spec.ts`
`POST /sync/push`	เล่นซ้ำงานออฟไลน์ของอุปกรณ์	`sync-push.e2e-spec.ts`, `sync-push-fixtures.e2e-spec.ts`
คิวงาน BullMQ และ Bull-Board	ส่งออกข้อมูลร้านและงานตามเวลา	`queue.e2e-spec.ts`, `worker-jobs.e2e-spec.ts`

4.6.2 การแยกข้อมูลรายร้าน

การพิสูจน์ว่า RLS [10] ทำงานจริงอยู่ในไฟล์ `server/test/cross-tenant-read.e2e-spec.ts` ซึ่งตรวจครบทั้ง 25 ตารางที่มีข้อมูลรายร้าน และทดสอบผ่าน HTTP อีก 12 จุดปลายทาง โดยทุกกรณีต้องได้ 0 แถวหรือสถานะ 404

ระหว่างการตรวจโค้ดทั้งระบบพบข้อบกพร่องสองข้อใน migration ของตาราง `owner_review_items` ข้อแรกคือนโยบาย RLS ไม่ห่อค่าตัวแปรเซสชันด้วย `NULLIF(…, '')` ทำให้เมื่อค่าว่างเกิดข้อผิดพลาดและตอบ HTTP 500 แทนศูนย์แถว ข้อที่สองคือคีย์นอกที่ตั้ง `ON DELETE SET NULL` ทำให้ `tenant_id` ซึ่งเป็น NOT NULL ถูกตั้งเป็นค่าว่างไปด้วย ทั้งสองข้อแก้เมื่อวันที่ 25 กันยายน พ.ศ. 2569 ด้วย migration ใหม่ `1788652804200-`

`OwnerReviewItemsFixes.ts` และพิสูจน์ใน `server/test/schema.e2e-spec.ts` โดยไม่แก้ migration ที่ถูกใช้งานไปแล้ว ปัจจุบันมี migration ทั้งหมด 20 ไฟล์

4.6.3 จุดเชื่อมธุรกรรม (transaction seam) และ idempotency

การย้ายธุรกรรมจาก middleware มาอยู่ใน handler ตามกติกาในบทที่ 3 แบ่งเป็นหกชิ้นงาน ซึ่งรวมเสร็จครบเมื่อวันที่ 15 กันยายน พ.ศ. 2569 การวัดด้วย `tx-hold-measure` หลังย้ายการตรวจ PIN ออกจากธุรกรรม แสดงว่าธุรกรรมที่ยาวที่สุดลดจากราว 101–131 มิลลิวินาที เหลือราว 14–22 มิลลิวินาที และเวลาตอบเลิกเพิ่มเป็นขั้นบันไดตามจำนวนคำขอพร้อมกัน สเปกสถาปัตยกรรมสามไฟล์ (`tenant-door.spec.ts`, `tenant-wrapper.spec.ts` และ `idempotency-routes.spec.ts`) ตรวจโครงสร้างของโค้ดว่าไม่มีเส้นทางเขียนใดหลบเลี่ยงจุดเชื่อมนี้ แต่มีจุดบอดหนึ่งข้อ คือเส้นทางเขียนที่ไม่ได้อ้างสิทธิ์ idempotency เลยจะไม่ปรากฏในสเปกทั้งสามไฟล์ จึงยังต้องอาศัยการทบทวนโดยคน

ในเชิงพฤติกรรม ระบบรับประกันสิ่งต่อไปนี้ซึ่งมีการทดสอบรองรับตามตารางที่ 4.12 การขายที่สต็อกไม่พอหลายบรรทัดตอบข้อความภาษาไทยครบทุกบรรทัดในคำตอบเดียว การขายพร้อมกัน 200 คำขอบนสินค้าที่มี 50 ชิ้นได้บิลสำเร็จ 50 ใบพอดี (วัดบน VM สาธิตเมื่อ 15 กันยายน พ.ศ. 2569 ได้ 201 จำนวน 50 ครั้ง 409 จำนวน 150 ครั้ง และ 5xx ศูนย์ครั้ง) และสภาวะแข่งกัน (race condition) ของสามคำขอบนสินค้าตัวเดียว 200 รอบไม่มีการสูญหายของการอัปเดต นอกจากนี้การนำเข้าสแนปช็อตข้อมูลจริงของร้านผ่านเกณฑ์ตรวจ 44 ข้อ (#185 เมื่อ 17 กันยายน พ.ศ. 2569) โดยไฟล์ที่นำเข้าให้ยอดขาย 33,700 บาทและสต็อกรวม 191 ชิ้น ระบบนำเข้าตรวจข้อมูลก่อน (pre-flight) แล้วจึงปรับค่า เพราะการบีบค่า (clamp) บนข้อมูลที่ยังไม่ตรวจจะเปลี่ยนความเสียหายที่เห็นชัดให้กลายเป็นความเสียหายเงียบ

4.6.4 ระนาบผู้ดูแลแพลตฟอร์ม

การทดสอบ `platform.e2e-spec.ts` ครอบคลุมวงจรเต็มผ่าน HTTP คือสร้างร้าน เข้าสู่ระบบเป็นเจ้าของ สร้างสินค้า ลงทะเบียนเครื่อง `pos` เข้าสู่ระบบบนเครื่องนั้น เปิดกะ และขายจริง ตั๋วงานหลัก #443 มีโค้ดที่รวมแล้วเมื่อ 27 กันยายน พ.ศ. 2569 ได้แก่ เครื่องมือบรรทัดคำสั่งของแพลตฟอร์ม การออกรหัสลงทะเบียนเครื่องใหม่ รหัสผ่านชั่วคราวของเจ้าของร้านพร้อมบังคับเปลี่ยน และแดชบอร์ดเว็บ `platform-ui` การจำกัดเส้นทาง `/api/v1/platform/` สองชั้น (Nginx และแอปพลิเคชัน) พิสูจน์แล้วว่าได้ 403 ทั้งสองชั้นบนเครื่อง `mob04` เมื่อ 30 กันยายน พ.ศ. 2569 ตั๋ว #443 ยัง **ไม่ปิด** เพราะเกณฑ์ข้อ "เฉพาะ `bootstrap:admin` เท่านั้นที่สร้างผู้ดูแลได้"

ขัดกับการซิงก์ผู้ดูแลจากตัวแปร `PLATFORM_ADMINS` ที่เจ้าของโครงงานรับรองแล้ว และยังมีคำถามที่รอเจ้าของโครงงานตัดสิน

4.6.5 เซิร์ฟเวอร์ของเฟส 2

จุดปลายทาง `POST /sync/push` สร้างตามสเปกเฟส 2 และทดสอบฝั่งเซิร์ฟเวอร์กับไฟล์ตัวอย่างคำขอ 18 ไฟล์ชุดเดียวกับที่ฝั่งไคลเอนต์ใช้ (`sync-push-fixtures.e2e-spec.ts` และ `sync-push.e2e-spec.ts`) ทำให้สองเลนพัฒนาแยกกันได้โดยไม่ต้องรอกัน การตรวจโค้ดทั้งระบบเมื่อ 24 กันยายน พ.ศ. 2569 พบข้อบกพร่องระดับสูงสองข้อ ข้อแรกคือค่าสรุปของคำขอ (fingerprint) ของ `/sync/push` ไม่ตรงกับของเส้นทางออนไลน์ (`POST /sales` เทียบกับ `POST /api/v1/sales`) แก้ด้วย PR #413 ข้อที่สองคือเส้นทางออนไลน์เก็บวันที่จากเนื้อหาคำขอของไคลเอนต์แทนเวลาเซิร์ฟเวอร์ แก้ด้วย PR #414 ทั้งสองข้อแก้แล้วเมื่อ 25 กันยายน พ.ศ. 2569 รายงานเดียวกันพบช่องว่างของสเปกระดับกลางสามข้อซึ่งแก้แล้วเมื่อ 27 กันยายน (PR #456, #458, #469) ข้อที่ยังเหลือสรุปไว้ในบทที่ 5

4.7 ผลการพัฒนา CI/CD

4.7.1 ระดับที่ 1 ถึง 3

สามระดับแรกของ CI/CD ตามบทที่ 3 เสร็จสมบูรณ์ ประกอบด้วยเวิร์กโฟลว์ GitHub Actions [31] สามไฟล์ ได้แก่ `flutter.yml`, `server.yml` และ `deploy.yml` โดย Flutter CI ตรวจโค้ด ทดสอบ ตรวจไฟล์ที่สร้างอัตโนมัติ และตรวจว่าไฟล์ฐานข้อมูลบนเว็บตรงกับเวอร์ชันแพ็กเกจ ส่วน Server CI รันการตรวจโค้ด การทดสอบหน่วย การทดสอบรวมระบบกับ PostgreSQL และ Redis จริง และสร้างอิมเมจที่ผ่านการสแกนด้วย Trivy [33] ก่อนเผยแพร่ไปยัง GHCR อิมเมจใน Docker Compose ถูกตรึงด้วย digest (#401, PR #403) และมีการทดสอบ `compose-image-pins.spec.ts` ตรวจเรื่องนี้ การป้องกันสาขา `main` ตั้งแต่ 15 กันยายน พ.ศ. 2569 บังคับให้ผ่าน PR และผ่านงานสถานะสองรายการ

เมื่อวันที่ 1 ตุลาคม พ.ศ. 2569 มีการเพิ่มด่านความปลอดภัยสามรายการ ได้แก่ งานสแกนความลับด้วย gitleaks [34] ใน `server.yml` ที่สแกนคอมมิตที่ PR หรือ push เพิ่มเข้ามาทุกครั้งโดยไม่กรองพาธ และเป็นเงื่อนไขของ `server-ci-status` การเปิด secret scanning และ push protection ของ GitHub และการเปิด Dependabot alerts พร้อม security updates นอกจากนี้กลไกข้ามการทดสอบเมื่อ push เป็นเอกสารล้วนทำงานได้หลังแก้ข้อบกพร่องใน PR #548 ซึ่งอธิบายไว้ในบทที่ 5

เมื่อวันที่ 3 ตุลาคม พ.ศ. 2569 มีการเพิ่มด่านคุณภาพอีกชุด ฝั่ง Flutter (PR #579) ได้แก่ การทดสอบที่ตรวจว่าการเขียนข้อมูลทุกจุดในหน้าจอจัดการข้อผิดพลาด (silent-failure guard) เกณฑ์ความครอบคลุมของการทดสอบที่ลดลงไม่ได้ (coverage ratchet) และกฎ lint ที่เข้มขึ้น ฝั่งเซิร์ฟเวอร์ (PR #581) ได้แก่ ไฟล์ตัวอย่างคำขอที่ไคลเอนต์บันทึกแล้วเซิร์ฟเวอร์เล่นซ้ำในการทดสอบ e2e การตรวจว่า migration ที่มีอยู่แล้วไม่ถูกแก้ การตรวจเวิร์กโฟลว์และสคริปต์ด้วย actionlint และ shellcheck ในงานใหม่ `ci-guards` และเกณฑ์ความครอบคลุมของเซิร์ฟเวอร์ บทเรียนจากวันนั้นคือ PR #579 และ #580 ผ่าน CI แยกกัน แต่เมื่อรวมห่างกัน 29 วินาที `main` กลับเป็นสีแดง เพราะด่านใหม่ตีความการอ่านยอดเงินในลิ้นชักของ #580 ว่าเป็นการเขียน PR #582 แก้ให้ `main` กลับมาผ่าน และเอกสาร `07_CICD_DEPLOY.md` §2c (PR #588, #589) บันทึกกติกาว่าหลังด่านใหม่รวมแล้วต้อง rebase PR ที่เปิดอยู่ก่อนรวม

4.7.2 ระดับที่ 4: การ deploy ด้วย Ansible และ self-hosted runner

self-hosted runner บน `mob04` ติดตั้งเมื่อ 30 กันยายน พ.ศ. 2569 และการ deploy ด้วย Ansible [35] ทำงานจริงในวันเดียวกัน ตั๋ว #67 (การติดตั้ง runner) ปิดในวันนั้นด้วยหลักฐานครบ 15 จาก 15 ข้อ หลักฐานหลักแสดงในตารางที่ 4.7 ข้อที่ต้องระบุคือ เกณฑ์ย่อยด้านการจัดการ PR จาก fork พิสูจน์จากโค้ดและค่าติดตั้งเท่านั้น ไม่มีการรันจริงจาก fork และเจ้าของโครงงานยอมรับหลักฐานระดับนี้

ตารางที่ 4.7 หลักฐานการ deploy บนเครื่อง mob04 (30 กันยายน พ.ศ. 2569)

กรณี	คอมมิต	ผล
deploy จริงครั้งแรก (อนุมัติโดย NuimanLP)	e50f4fa	Ansible failed=0, . current_sha = e50f4fa, /health/ready ตอบ 200
merge เข้า main แล้ว deploy อัตโนมัติ	494ace3	deploy สำเร็จ
ย้อนรุ่นผ่าน workflow_dispatch	e50f4fa	สำเร็จ สคีมาไม่เปลี่ยน
รัน SHA เดิมซ้ำ	SHA ที่ติดตั้งอยู่แล้ว	ข้ามด้วยข้อความ "Skipping duplicate deployment"
readiness ล้มเหลว	–	รันเป็นสีแดง และติดตั้งรุ่นเดิมกลับอัตโนมัติ
สั่ง deploy จากสาขาที่ไม่ใช่ main	–	ถูกปฏิเสธ
deploy การแก้ CORS (PR #516)	00d3488	Ansible failed=0

ประตูอนุมัติบนสภาพแวดล้อม demo ทำงานตามที่ออกแบบ คือทุกรันของ Deploy (demo) เข้าสถานะ "Waiting for review" จนกว่าผู้ตรวจที่กำหนดจะอนุมัติ ข้อควรระวังคือการรัน Deploy (demo) ที่เป็นสีเขียวไม่ใช่หลักฐานว่ามีการ deploy เพราะงาน deploy ถูกข้ามเมื่ออิมเมจของ SHA นั้นยังไม่อยู่บน GHCR โดยเวิร์กโฟลว์ยังรายงานสำเร็จ เช่น เมื่อ 30 กันยายน การตรวจ pnpm audit ระดับสูงทำให้ server-ci-status บน main เป็นสีแดง จึงไม่มีอิมเมจและทุกรัน deploy ถูกข้ามจนกว่า PR #512 จะรวม หลักฐานที่ใช้ได้มีเพียงไฟล์ /opt/pos/.current_sha บนเครื่อง

4.7.3 เวิร์กโฟลว์ Android APK และ TLS แบบ CA ส่วนตัว (3 ตุลาคม พ.ศ. 2569)

เมื่อวันที่ 3 ตุลาคม พ.ศ. 2569 มีการรวม PR สองชุดที่เปิดทางให้แอปบนอุปกรณ์ Android เชื่อมต่อเซิร์ฟเวอร์บน mob04 ได้ ตารางที่ 4.8 สรุปสิ่งที่รวมแล้วและสถานะจริง การออกแบบอยู่ในบทที่ 3

ตารางที่ 4.8 งาน Android APK และ TLS ที่รวมเมื่อ 3 ตุลาคม พ.ศ. 2569 และสถานะ

งาน	สิ่งที่รวมแล้ว	สถานะ
PR #551 เวิร์กโฟลว์ `android-apk.yml`	สั่งรันด้วยมือ เฉพาะ `main` สร้าง APK บิลด์ API ชนิดเดียว ลงนามด้วยความลับ `ANDROID_KEYSTORE_B64` เผยแพร่เป็น pre-release `apk-<sha7>` เพิ่มสิทธิ์ `INTERNET`	รันแล้วสามครั้งเมื่อ 3 ต.ค. 2569: ครั้งแรกล้มเหลว แก้ด้วย PR #560 จากนั้นได้ pre-release `apk-e191755` และ `apk-ff84fd3`
PR #552 CA ส่วนตัวสำหรับ `mob04`	`certgen` รักษา CA ในวอลุ่ม `certs-ca` และออกใบรับรองเซิร์ฟเวอร์ (SAN `localhost`, `127.0.0.1`, `172.30.58.20`) Nginx เชื่อถือ playbook รัน `certgen` มีสคริปต์ทดสอบ `certgen.test.sh` แอปเชื่อถือ CA ที่ฝังมาผ่าน `HttpOverrides`	ติดตั้งบน `mob04` ตั้งแต่การ deploy `7ea0178` และคอมมิต CA ลงแอสเซตแล้ว (PR #559)
PR #553 เอกสาร	ปรับ `CLAUDE.md`, คู่มือการ deploy, คู่มือฝ่ายไอที และบันทึกการศึกษาให้ตรงกับสองรายการข้างต้น	รวมแล้ว (เอกสารล้วน)

ขั้นตอนตามคู่มือ `07_CICD_DEPLOY.md` §5 "TLS" ทำครบในวันเดียวกัน การ deploy `7ea0178` (รัน `37097022857`, Ansible `failed=0`) เป็นครั้งแรกที่ `certgen` สร้าง CA บน `mob04` PR #559 คอมมิตใบรับรอง CA ลงแอสเซต โดยตรวจว่าลายนิ้วมือ SHA-256 ตรงกับ `certs-ca/ca.crt` บนเครื่อง `openssl s_client` ยืนยันใบรับรองของ `172.30.58.20` ได้ และ `HttpClient` ของ Dart ที่เชื่อถือ CA นี้เรียก `/health/ready` ได้ 200 ส่วนเวิร์กโฟลว์ APK รันครั้งแรก (รัน `37099021690`) ล้มเหลวที่ขั้นคอมไพล์ เพราะ `file_picker` รุ่น 11 ไม่คอมไพล์ซอร์ส Kotlin ของตัวเองบน Android Gradle Plugin 9 PR #560 ยกรุ่นเป็น 13.1.0 แล้วการรันครั้งถัดมาสองครั้งสำเร็จ ได้ pre-release `apk-e191755` และ `apk-ff84fd3` ข้อที่ต้องระบุตรงไปตรงมาคือ ที่เก็บโค้ดยังไม่มีบันทึกการติดตั้ง APK บนอุปกรณ์ Android จริงแล้วเข้าสู่ระบบ และ APK ล่าสุด (`ff84fd3`) สร้างก่อนการแก้ไขในหัวข้อ "การทดสอบใช้งานบนเครื่องสาธิตและการแก้ไข" ของบทนี้ จึงยังไม่มีการแก้ไขชุดนั้น

4.7.4 การ deploy และการทดสอบบนเครื่องสาธิต (3–4 ตุลาคม พ.ศ. 2569)

การแก้ไขในตารางที่ 4.2 ถูกนำขึ้น `mob04` ผ่านเส้นทาง CD ปกติ คือรวมเข้า `main` แล้วงาน `deploy` รอการอนุมัติบนสภาพแวดล้อม `demo` ก่อนจะแตะเครื่อง ตารางที่ 4.9 สรุปการ deploy ที่เกี่ยวข้อง รันและการอนุมัติตรวจจาก GitHub ส่วนข้อมูลบนเครื่อง (`.current_sha` และ `/health/ready`) ผู้ทดสอบตรวจผ่าน Secure Shell (SSH) และรายงานไว้ ผู้เขียนรายงานไม่ได้ตรวจซ้ำบนเครื่องเอง

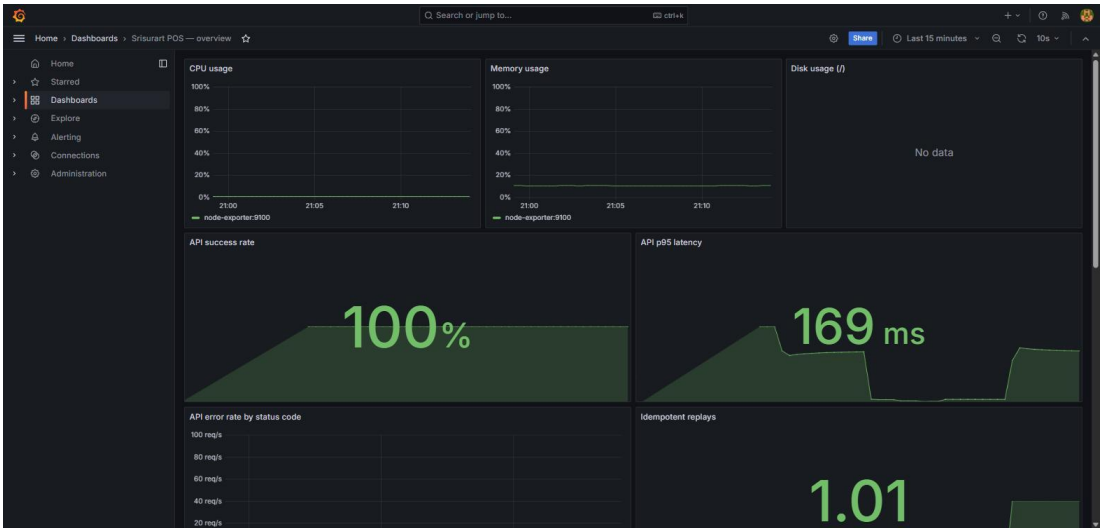
ตารางที่ 4.9 การ deploy ขึ้นเครื่อง mob04 ระหว่าง 3–4 ตุลาคม พ.ศ. 2569

วันที่	คอมมิต	รัน `Deploy (demo)`	ผล
3 ต.ค. 2569	`7ea0178`	`37097022857`	Ansible `failed=0`, `.current_sha` = `7ea0178`, `certgen` สร้าง CA เป็นครั้งแรก
3 ต.ค. 2569	`11265b4`	`37132624209` (อนุมัติโดย `NuimanLP`)	`.current_sha` = `11265b4`, `/health/ready` ตอบ 200
4 ต.ค. 2569	`f2827ed`	`37168492338` (อนุมัติโดย `NuimanLP`)	งาน `deploy to demo` สำเร็จ, `.current_sha` = `f2827ed`, `/health/ready` ตอบ 200

หลังการ deploy `11265b4` การทดสอบบนหน้าจอจริงยืนยันว่าการจ่ายเงินออก 600 บาทจากลิ้นชักที่มี 500 บาทถูกปฏิเสธด้วยข้อความ เงินในลิ้นชักไม่พอ `(มี ฿500)` และตาราง `drawer_entries` ของกะนั้นมีเพียงรายการจ่ายออก 100 บาทที่ยอมรับ ส่วนเส้นทาง 409 ของเซิร์ฟเวอร์ไม่ได้ถูกทดสอบบนเครื่อง เพราะหน้าจอหยุดคำขอไว้ก่อน จึงอาศัยการทดสอบ e2e ใน PR #580 และ #584 แทน หลังการ deploy `f2827ed` การทดสอบช่องจำนวนเงินบนหน้าจอจริงให้ผลตามที่ออกแบบ คือพิมพ์ `a1.2.3` ในช่องส่วนลดได้ `1.23` พิมพ์ `a5.0.009` ในช่องรับเงินได้ `5.00` และพิมพ์ `a5.0.05` ในช่องเงินตั้งต้นของกะได้ `5.00` นอกจากนี้ได้ทดลองเพิ่มสินค้าทดสอบแล้วลบออก โดยไม่มีการขายจริงในรอบนี้ ผลการทดสอบบนหน้าจอเหล่านี้บันทึกตามที่ผู้ทดสอบรายงาน

4.7.5 การติดตามผลด้วย Grafana

การซ้อมสาธิตวันที่ 21 กันยายน พ.ศ. 2569 บันทึกแผงควบคุม Grafana [37] ขณะมีการเข้าสู่ระบบและขายจริงบนสแตกที่ทำงานอยู่ ดังรูปที่ 4.14 แผงแสดงอัตราความสำเร็จ ความหน่วง p95 อัตราข้อผิดพลาดแยกตามรหัสสถานะ และตัวนับการเล่นซ้ำของ idempotency ที่ Prometheus [36] เก็บจากตัวชี้วัดของเซิร์ฟเวอร์ตามการออกแบบในบทที่ 3



รูปที่ 4.14 แผงควบคุม Grafana ระหว่างการซ้อมสาธิต (บันทึกเมื่อ 21 กันยายน พ.ศ. 2569)

4.7.6 การขยายการเฝ้าสังเกตระบบ (1–4 ตุลาคม พ.ศ. 2569)

ระหว่างวันที่ 1–4 ตุลาคม พ.ศ. 2569 มีการพัฒนาส่วนขยายของการเฝ้าสังเกตระบบบนสาขา `develop` ซึ่งแยกจาก `main` และทดสอบบนเครื่องของผู้พัฒนา ก่อนแยกส่วนที่ใช้กับ VM ได้เป็น PR #596 และ #597 โดยใช้ค่าตั้งชุดเดียวกับ `mob04` ตารางที่ 4.10 สรุปงานและสถานะ ณ วันที่ 4 ตุลาคม พ.ศ. 2569 ทั้งสอง PR รวมเข้า `main` แล้วในวันเดียวกัน และ PR #597 ถูก deploy ขึ้น `mob04` แล้ว (`41a8f19`) โดยยืนยันจากไฟล์ `/opt/pos/.current_sha` บนเครื่อง และจาก Prometheus บน VM ที่มีตัวชี้วัดใหม่ครบทั้งสามกลุ่ม ต่อมาในคืนเดียวกันได้ติดตั้งการแจ้งเตือนเมื่อระบบล่มบน `mob04` และปรับตัวอ่านงานในคิว (PR #606) ซึ่ง deploy แล้ว (`7444ea4`)

ตารางที่ 4.10 งานขยายการเฝ้าสังเกตระบบและสถานะ ณ 4 ตุลาคม พ.ศ. 2569

งาน	สิ่งที่ทำ	สถานะ
ตัวชี้วัดใหม่ของเซิร์ฟเวอร์	จำนวนเอกสาร สถานะ pool และงานในคิว	deploy แล้ว (PR #597, `41a8f19`; ปรับตัวอ่านคิว PR #606, `7444ea4`)
แผงควบคุมหน้า Overview	เพิ่มจาก 12 เป็น 29 แผง	deploy แล้ว (PR #597, `41a8f19`)
ชุดทดสอบ	หน่วย 25 กรณี และ e2e 2 กรณี	ผ่าน CI ของ PR #597
ที่เก็บบันทึกและตัวส่งออก	Loki, Alloy, ตัวส่งออก PostgreSQL, Redis และ cAdvisor พร้อมแผงควบคุมหน้าที่สอง 15 แผง	ใช้เฉพาะเครื่องผู้พัฒนา ไม่ deploy ขึ้น VM
การแจ้งเตือนเมื่อระบบล่ม	ส่งสัญญาณจาก cron บน VM ไปยัง Healthchecks.io ทุก 5 นาที	ติดตั้งบน `mob04` แล้ว (PR #596, ติดตั้งด้วยมือ 4 ต.ค.) ยังไม่ทดสอบกรณีแจ้งเตือนจริง

ตัวชี้วัดใหม่มีสามกลุ่ม `pos_documents_total` นับบิลขาย บิลยกเลิก และการคืนสินค้าเมื่อธุรกรรม commit แล้วเท่านั้น ไม่นับคำขอที่เล่นซ้ำด้วยคีย์เดิม และไม่มี label `tenant_id` เช่นเดียวกับตัวนับการเล่นซ้ำ `pos_db_pool_connections` และ `pos_db_pool_max_connections` อ่านสถานะ pool ขณะ Prometheus scrape ผ่านตัวอ่านที่ไม่เปิดให้เข้าถึง DataSource ตามข้อบังคับของสเปกสถาปัตยกรรม และ `pos_queue_jobs` อ่านจำนวนงานในคิว BullMQ ทั้งหกคิวโดยจำกัดเวลา 1 วินาที เพื่อไม่ให้การ scrape ค้างเมื่อ Redis ไม่ตอบ และเนื่องจากการจำกัดเวลาไม่ได้ยกเลิกคำสั่งที่ส่งไปแล้ว PR #606 จึงให้การ scrape ที่พบคำสั่งอ่านเดิมยังค้างอยู่รอผลของคำสั่งนั้นแทนการส่งคำสั่งใหม่ คำสั่งจึงไม่สะสมแล้วถูกส่งออกพร้อมกันเมื่อ Redis กลับมา แผงใหม่บนหน้า Overview ได้แก่ แผงราย instance และรายเส้นทาง แผงจำนวนเอกสาร pool งานในคิว การเข้าสู่ระบบที่ถูกปฏิเสธ และการแยกคำขออ่านกับเขียนตาม HTTP method ชุดทดสอบที่เพิ่มทำให้ความครอบคลุมบรรทัดของการทดสอบหน่วยเพิ่มจากร้อยละ 44.70 เป็น 45.39 ส่วนที่เก็บบันทึก Loki เก็บบันทึกของทุกคอนเทนเนอร์ 7 วัน โดย Alloy อ่านผ่าน Docker และการนำส่วนนี้ขึ้น `mob04` เป็นการตัดสินใจของเจ้าของโครงงาน

การแยกแผงควบคุมเป็นสองหน้ามีเหตุผลจากการ deploy เพราะ `deploy.yml` คัดลอกโฟลเดอร์ `deploy/grafana/` ทั้งโฟลเดอร์ขึ้น VM หากรวมแผงของตัวส่งออกที่มีเฉพาะบนเครื่องผู้พัฒนาไว้ในหน้าเดียวกัน หน้าบน VM จะมีแผงที่ไม่มีข้อมูลปะปนกับแผงที่ใช้งานได้ PR #597

จึงมีเฉพาะหน้า Overview ที่ทุกแผงอาศัยเพียง Prometheus และเซิร์ฟเวอร์ที่มีบน `mob04` อยู่แล้ว สถานะของแผงทั้งหมดเมื่อทดสอบบนเครื่องผู้พัฒนาแสดงดังรูปที่ 4.15



รูปที่ 4.15 สถานะแผงควบคุม 44 แผงจากการทดสอบบนเครื่องผู้พัฒนา (2 ตุลาคม พ.ศ. 2569; 15 แผงของหน้าที่สองมีเฉพาะบนเครื่องผู้พัฒนา)

สำหรับการแจ้งเตือนเมื่อระบบล่ม คณะผู้จัดทำทดลอง Uptime Kuma บนเครื่องผู้พัฒนาก่อน แล้วเปลี่ยนเป็นการส่งสัญญาณจาก VM ออกไปยังบริการภายนอก เหตุผลคือตัวตรวจที่รันบน `mob04` จะหยุดทำงานพร้อม VM จึงแจ้งเตือนกรณี VM ล่มทั้งเครื่องไม่ได้ ในขณะที่การส่งสัญญาณออกไปทำให้การขาดหายของสัญญาณเป็นตัวแจ้งเตือนเอง ครอบคลุมทั้งกรณี VM ล่ม เครือข่ายขาด และ cron หยุดทำงาน เมื่อวันที่ 4 ตุลาคม พ.ศ. 2569 ได้ทดสอบจาก `mob04` แล้วว่าเชื่อมต่อ `hc-ping.com` ผ่านไฟร์วอลล์ของคณะได้ โดยใบรับรองที่ได้รับเป็นของผู้ให้บริการจริง ไม่ใช่ใบรับรองที่ไฟร์วอลล์ออกแทน จากนั้นในคืนวันเดียวกันได้ติดตั้งบน `mob04` ด้วยมือตามขั้นตอนในเอกสารการ deploy แทนการรัน `provision.yml` เพราะ playbook นั้นเขียนทับไฟล์ค่าตั้งทั้งไฟล์ โดยสคริปต์บนเครื่องตรงกับ `main` (ตรวจด้วย SHA-256) URL สำหรับส่งสัญญาณถูกเพิ่มในไฟล์ค่าตั้งผ่าน standard input จึงไม่ปรากฏในบรรทัดคำสั่งที่ผู้ใช้อื่นบนเครื่องมองเห็นได้ และติดตั้ง cron ก่อนทดสอบครั้งแรก การรันด้วยมือและ cron รอบแรก (22:20 น.) ทำงานโดยไม่มีข้อผิดพลาด ส่วนที่ยังไม่ได้พิสูจน์คือการรัน `provision.yml` ส่วนนี้กับเครื่องจริง และการแจ้งเตือนจริงจากบริการเมื่อระบบไม่พร้อม

4.8 ผลการนำระบบขึ้นใช้งานบนเครื่อง mob04 และการติดตามระบบ

4.8.1 สแตกบนเครื่องสาธิต

เครื่อง `mob04` เป็น VM ของภาควิชาที่เป็นทั้งเครื่องสาธิตและเป้าหมายการ deploy ตามการตัดสินใจเมื่อ 15 กันยายน พ.ศ. 2569 ที่ให้ระบบเฟส 1 อยู่ภายในเครือข่ายของมหาวิทยาลัย สแตกประกอบด้วยคอนเทนเนอร์ตามตารางที่ 3.3 เครื่องดึงอิมเมจจาก `ghcr.io` ได้ตั้งแต่ 29–30 กันยายน พ.ศ. 2569 ก่อนหน้านั้นการ deploy อัตโนมัติ (CD) ซึ่งต้องดึงอิมเมจจาก GHCR ถูกไฟร์วอลล์ของคณะปิดกั้น (บทที่ 5)

ตั๋ว #343 (การ deploy จริงครั้งแรก) ปิดเมื่อ 30 กันยายน พ.ศ. 2569 ครบ 5 จาก 5 ข้อ รวมถึงการย้อนกลับด้วย Ansible ด้วยมือไปยัง `e50f4fa` ที่ได้ `failed=0` ส่วนการสาธิตแบบครบวงจร (#344) ยัง **ไม่ได้ดำเนินการ** ข้อติดขัดสรุปไว้ในบทที่ 5

4.8.2 การสำรองข้อมูลและ etcd

สคริปต์ `deploy/scripts/backup-db.sh` dump ฐานข้อมูลลงใน `/opt/pos/backups/` ทุกวันเวลา 03:00 ตั๋ว #346 ปิดเมื่อ 30 กันยายน พ.ศ. 2569 หลังรันสคริปต์ในสภาพแวดล้อมแบบเดียวกับ cron สำเร็จและตรวจไฟล์ dump แล้ว ในคืนวันที่ 29 กันยายน สคริปต์ทำงานล้มเหลวและทิ้งไฟล์ว่างไว้ PR #519 จึงให้สคริปต์เขียนไฟล์ชั่วคราวแล้วเปลี่ยนชื่อเมื่อสำเร็จเท่านั้น พร้อมการทดสอบใน CI และสคริปต์ใหม่ที่ติดตั้งบน `mob04` แล้วได้ไฟล์ dump ที่ผ่านการตรวจความสมบูรณ์ ข้อควรระวังคือมีเพียง playbook `provision.yml` ที่ติดตั้งสคริปต์นี้ การ deploy ผ่าน CD ไม่ได้อัปเดตสคริปต์บนเครื่อง

ต้องระบุอย่างตรงไปตรงมาว่า **การสำรองข้อมูลยังไม่ออกนอก VM** กลไกส่งสำเนาออกนอกเครื่องผ่าน `rclone` ถูกสร้างไว้แต่ยังไม่ได้เชื่อมกับปลายทางจริง (#363 และตั๋วงานหลัก #288 ยังเปิดและถูกพักไว้จนหลังการสาธิต) หากดิสก์ของ `mob04` เสียจะสูญเสียข้อมูลของร้านสาธิต เจ้าของโครงงานรับความเสี่ยงนี้โดยรู้ล่วงหน้า และบันทึกของ cron ที่ไม่มีข้อผิดพลาดไม่ได้พิสูจน์ว่าสำเนาออกนอก VM เพราะเมื่อยังไม่ได้ตั้งปลายทาง สคริปต์จะพิมพ์เพียงคำเตือนแล้วจบด้วยรหัสสำเร็จ

ตั๋ว #365 (etcd ไม่มีการยืนยันตัวตนบน VM) ปิดเมื่อ 30 กันยายน พ.ศ. 2569 ครบ 4 จาก 4 ข้อ โดยเปิดการยืนยันตัวตนตั้งแต่การ deploy จริงครั้งแรก พิสูจน์การยืนยันตัวตนทั้งสองด้านและยืนยันว่า `RuntimeConfigService` อ่านค่า `log_level` จาก etcd ได้

4.9 ผลการทดสอบฝั่งเซิร์ฟเวอร์

ชุดทดสอบของเซิร์ฟเวอร์แบ่งเป็นการทดสอบหน่วย (`*.spec.ts`) ที่อยู่ข้างไฟล์ต้นทาง และการทดสอบรวมระบบ (`*.e2e-spec.ts`) ที่รันกับ PostgreSQL และ Redis จริงภายใต้ `server/test/` จำนวนที่นับเมื่อ 1 ตุลาคม พ.ศ. 2569 แสดงในตารางที่ 4.11 ตัวเลขฝั่งเซิร์ฟเวอร์นับจากบรรทัด `it(` หรือ `test(` ในซอร์ส จึงเป็นค่าประมาณที่ไม่นับกรณีที่สร้างในลูป ส่วนฝั่งไคลเอนต์แสดงทั้งจำนวนจากตัวรันและจากซอร์สเพื่อเทียบกัน

ตารางที่ 4.11 จำนวนไฟล์และกรณีทดสอบของเซิร์ฟเวอร์และไคลเอนต์ (นับเมื่อ 1 ตุลาคม พ.ศ. 2569)

กลุ่มทดสอบ	ไฟล์	กรณีทดสอบ
เซิร์ฟเวอร์ — unit (`*.spec.ts`)	54	ประมาณ 492 (นับจากซอร์ส)
เซิร์ฟเวอร์ — e2e (`*.e2e-spec.ts`)	57	ประมาณ 655 (นับจากซอร์ส)
ไคลเอนต์ Flutter (`*_test.dart`)	90	820 ตามตัวรัน (741 จุดนิยามในซอร์ส)

ชุด e2e ครอบคลุมความปลอดภัยและบทบาทอุปกรณ์ การแยกข้อมูลรายร้าน ธุรกรรมและ idempotency สภาวะแข่งกันของผู้เขียนหลายราย การจำกัดอัตรา แคชและการล่มของ Redis การขาย คืนสินค้า ใบสั่งซื้อ กะ และรายงาน การนำเข้าและการสำรองข้อมูล ระนาบแพลตฟอร์ม คิวและ worker การซิงก์ และเกณฑ์ของสคีมา การรันชุด e2e เต็มเมื่อ 16 กันยายน พ.ศ. 2569 บันทึกว่าผ่าน 490 กรณีและข้าม 2 กรณีจากทั้งหมด 492 กรณี (ไม่มีกรณีล้มเหลว) ซึ่งเป็นจำนวนของชุด e2e ในวันนั้น รายงานนี้ไม่อ้างผลการรันล่าสุดที่ไม่ได้ทำซ้ำ ข้ออ้างว่าชุดทดสอบผ่านในแต่ละช่วงยึดตามสถานะ `server-ci-status` บน `main` ซึ่งเป็นผลจริงของการรันใน CI

4.10 ตรวจสอบเกณฑ์ตรวจรับ (DoD)

เอกสาร `docs/Backend_design/03_ARCHITECTURE.md` หัวข้อ 8 กำหนดเกณฑ์ตรวจรับ (DoD) ของเฟส 1 จำนวน 17 ข้อ การนับซ้ำกับตัวทดสอบเอง (ไม่ใช่สถานะของตั๋ว) เมื่อ 22 กันยายนและอีกครั้งในการเขียนรายงานนี้พบว่า **ผ่านแล้ว 16 ข้อ และยังเปิดอยู่ 1 ข้อ** คือ เกณฑ์ k6 ซึ่งผูกกับตั๋ว #380 ดังตารางที่ 4.12

ตารางที่ 4.12 สถานะเกณฑ์ตรวจรับ (DoD) ของเฟส 1 (17 ข้อ)

ลำดับ	เกณฑ์	สถานะ	หลักฐานหลัก
1	`docker compose up` ครั้งเดียวได้ Nginx + NestJS ×3 + Postgres + Redis + worker + Bull-Board	ผ่าน	#14 (6 ก.ย. 2569)
2	k6 ผ่านเกณฑ์ตาม `02_API_SCREENS.md` หัวข้อ 9	**ยังเปิด**	ตั๋ว #380 วัดจริงครั้งแรก 5 ต.ค. 2569 ยังไม่ได้รับผล (แย่งซื้อ 200 คน p95 ประมาณ 1.5–3 วินาที เกินเกณฑ์ 500 มิลลิวินาที)
3	`POST /sales` พร้อมกัน 200 ครั้งบนสินค้า 50 ชิ้น ขายได้ 50 บิล สต็อก 0	ผ่าน	#184 ส่วน `close.3` (15 ก.ย. 2569, VM สาธิต) 201×50 / 409×150 / 5xx 0
4	อ่านข้ามร้านได้ 0 แถวทุกเคส	ผ่าน	`cross-tenant-read.e2e-spec.ts` (17 ก.ย. 2569)
5	`/health/live` ไม่แตะ DB และ `/health/ready` แตะ DB และ Redis	ผ่าน	#14 (6 ก.ย. 2569)
6	นำเข้าสแนปช็อตร้านจริง ผ่านเกณฑ์ 6 ข้อใน `01_DATABASE.md` หัวข้อ 9	ผ่าน	#185 (17 ก.ย. 2569) 44 checks
7	`redis-cache` แยกจาก `redis-queue` และ Bull-Board มี auth	ผ่าน	#14 (6 ก.ย. 2569)
8	ขายสินค้าไม่พอ 3 บรรทัดได้ข้อความไทยครบ 3 บรรทัดในครั้งเดียว	ผ่าน	`sales.e2e-spec.ts`, #20

ลำดับ	เกณฑ์	สถานะ	หลักฐานหลัก
9	ลบลูกค้าที่มีบิลได้ 200 (soft delete) ไม่ใช่ 500	ผ่าน	`people.e2e-spec.ts`, #17
10	สร้างร้านใหม่ด้วย `POST /platform/tenants` แล้วเข้าสู่ระบบและขายได้โดยไม่แตะ psql	ผ่าน	`platform.e2e-spec.ts` (17 ก.ย. 2569)
11	เครื่อง `backoffice` เรียก `POST /sales` แล้วได้ 403 `DEVICE_ROLE_FORBIDDEN`	ผ่าน	`security.e2e-spec.ts`, #44
12	ระงับร้านแล้วคำขอถัดไปถูกปฏิเสธทันที	ผ่าน	`request-context.e2e-spec.ts`, #20
13	ระงับร้านแล้วงานค้างในคิวของร้านนั้นไม่ถูกรัน	ผ่าน	`queue.e2e-spec.ts`, #34
14	ดับ `redis-cache` แล้วร้านที่ถูกระงับยังถูกปฏิเสธและร้านปกติยังใช้งานได้	ผ่าน	`redis-cache-outage.e2e-spec.ts`, #383 (22 ก.ย. 2569)
15	สภาวะแข่งกันของสามคำขอบนสินค้าตัวเดียว 200 รอบ ไม่มี lost update	ผ่าน	`purchase-orders.e2e-spec.ts`, #196
16	`owner` ปลดเครื่อง `pos` ที่มีกะเปิด: กะปิดในธุรกรรมเดียว token เดิม refresh ไม่ได้ ลงทะเบียนเครื่องใหม่แล้วขายได้	ผ่าน	`shifts.e2e-spec.ts`, #384 (22 ก.ย. 2569)
17	`backoffice` ที่ไม่มี `deviceToken` อ่านสินค้าได้แต่ขายได้ 403	ผ่าน	`security.e2e-spec.ts`, #196

เกณฑ์ข้อ 14 และข้อ 16 ได้รับการยืนยันผ่านใหม่เมื่อมีหลักฐานที่แท้จริง เพราะหลักฐานเดิมของข้อ 14 เพียงจำลอง (spy) การเรียกเมธอดของไคลเอนต์แคชแทนที่จะทำให้ Redis เข้าไม่ถึงจริง และหลักฐานเดิมของข้อ 16 สร้างโทเค็นเองแทนที่จะแลกรหัสลงทะเบียนผ่านจุดปลายทางจริง

เกณฑ์ข้อ 2 ยังเปิด จนถึงวันที่ 4 ตุลาคม พ.ศ. 2569 ยังไม่มีการวัด ตั๋ว #184 เดิมที่ผูกกับการวัดนี้ถูกปิดและเปิดใหม่รวมสามครั้งในสองวัน จนเจ้าของโครงงานปิดเมื่อ 21 กันยายน พ.ศ. 2569 โดยเกณฑ์ย่อยด้านการวัดยังไม่ผ่าน และตั๋ว #380 เข้ามาแทน

วิธีวัดที่ตกลงแล้วคือใช้สามเครื่องรัน k6 โดยแต่ละเครื่องอยู่ภายใต้ขีดจำกัดอัตราต่อ IP ของตัวเอง และส่งผลเข้า Prometheus ของ VM ส่วนการยกเว้น IP ของเครื่องสร้างโหลดจากขีดจำกัดอัตราถูกพิจารณาแล้วปฏิเสธ การวัดจริงครั้งแรกตามวิธีนี้ทำเมื่อ 5 ตุลาคม พ.ศ. 2569 ผลสรุปอยู่ในหัวข้อสถานะล่าสุด 2026-10-07 ของบทที่ 5 และยังไม่มีการทำเครื่องหมายผ่านจนกว่าเจ้าของโครงงานจะตัดสิน

4.11 สรุปผลงานที่มีความก้าวหน้า

ตารางที่ 4.13 สรุปตัวเลขภาพรวมของโครงงาน วัดเมื่อ 1 ตุลาคม พ.ศ. 2569 จากสาขา `origin/main` ด้วย `git rev-list --count`, `git ls-files` ร่วมกับ `wc -l`, `gh pr list` และ `gh issue list` จำนวนบรรทัดนับรวมไฟล์ที่สร้างอัตโนมัติที่ถูกคอมมิตไว้ จึงเป็นเพียงมาตรวัดขนาด ไม่ใช่มาตรวัดผลิตภาพ

ตารางที่ 4.13 สถิติภาพรวมของโครงงาน (วัดเมื่อ 1 ตุลาคม พ.ศ. 2569)

รายการ	ค่าที่วัดได้
จำนวนคอมมิตบน `origin/main`	1,058
คอมมิตแรกของที่เก็บโค้ด	23 มิถุนายน พ.ศ. 2569
Pull request ที่รวมแล้ว	330
Issue ทั้งหมดบน GitHub	209 (ปิดแล้ว 198, เปิดอยู่ 11)
โค้ดไคลเอนต์ `frontend/lib/`	120 ไฟล์ ประมาณ 65,600 บรรทัด
ชุดทดสอบไคลเอนต์ `frontend/test/`	93 ไฟล์ (ไฟล์ทดสอบ 90 ไฟล์) ประมาณ 29,800 บรรทัด
โค้ดเซิร์ฟเวอร์ `server/src/`	213 ไฟล์ ประมาณ 33,500 บรรทัด
ชุดทดสอบเซิร์ฟเวอร์ `server/test/`	82 ไฟล์ ประมาณ 29,800 บรรทัด
การ deploy และโครงสร้างพื้นฐาน `deploy/`	22 ไฟล์ ประมาณ 2,800 บรรทัด
เวิร์กโฟลว์ CI/CD `.github/`	7 ไฟล์ ประมาณ 1,100 บรรทัด
เอกสาร `docs/`	286 ไฟล์ (ไฟล์ข้อความ 215 ไฟล์ ประมาณ 56,200 บรรทัด ที่เหลือเป็นภาพและ PDF)

รายการ	ค่าที่วัดได้
ตารางใน Drift (schemaVersion 13)	26 ตาราง
ตารางใน PostgreSQL	29 ตาราง จาก migration 20 ไฟล์
เกณฑ์ตรวจรับ (DoD) ของเฟส 1	16 จาก 17 ข้อ (เหลือ k6, #380)
บันทึกการตัดสินใจเชิงสถาปัตยกรรม (ADR)	13 หมายเลข (ADR-0001 ถึง ADR-0013)

ตารางที่ 4.14 สรุปผลงานรายบุคคล คอมมิตนับจากอีเมลผู้เขียนคอมมิตบน `main` โดยบัญชี git สามชื่อ (NuiGates, NuiGates_2456 และ Chavatik Thorarit) เป็นของสมาชิกคนเดียวกัน และบัญชี LomerAlloys สองอีเมลเป็นของสมาชิกคนเดียวกัน ส่วน PR นับด้วย `gh pr list --state merged --json author` จำนวนคอมมิตและ PR ไม่ได้สะท้อนขนาดหรือความยากของงาน และคอมมิตส่วนหนึ่งทำร่วมกับเครื่องมือช่วยพัฒนา (บทที่ 3) ตารางนี้จึงแสดงว่าใครลงมือในส่วนใด ไม่ใช่การจัดอันดับ

ตารางที่ 4.14 ผลงานของสมาชิกแต่ละคน (นับเมื่อ 1 ตุลาคม พ.ศ. 2569)

สมาชิก (เลน)	คอมมิตบน `main`	PR ที่รวมแล้ว	ตัวอย่างงานที่ส่งมอบ (PR)
Chavatik Thorarit (NuimanLP, A)	890	273	สแตก Compose และ health (#41), migration และ RLS (#42), idempotency และเส้นทางขาย คืน ลิ้นชัก (#51, #75, #78, #84), `SyncFacade` และไฟล์ตัวอย่าง (#307), แก้เลขเอกสารและการทิ้งรายการในคิว (#483–#495), สคริปต์สำรองข้อมูล (#519)
LomerAlloys (B)	53	15	ระนาบผู้ดูแลแพลตฟอร์มและการสร้างร้าน (#59), การตั้งค่าและ bootstrap (#116), จัดซื้อและต้นทุนถัวเฉลี่ย (#119), แก้ฐานข้อมูลบนเว็บ (#310), `outbox_ops` และ `SyncService` (#324)
Pattarapon Kitcharoen (PattaraponKitcharoen, C)	115	43	JWT และบทบาทอุปกรณ์ (#57), rate limit ต่อร้าน (#77), BullMQ และ worker (#88), Ansible และ runner (#107, #348), `POST /sync/push` (#313), หน้ารอเจ้าของร้านตรวจ (#318), PIN ออฟไลน์ (#332), gitleaks (#526)

ตั๋วงานที่เปิดอยู่ ณ 1 ตุลาคม พ.ศ. 2569 มี 11 ตั๋ว ได้แก่ ตั๋วงานหลักของเฟส 1 สองตั๋ว (#2 และ #196) ซึ่งยังเปิดอยู่ขณะที่เกณฑ์ k6 ยังไม่ผ่าน การวัดภาระ k6 (#380) การสาธิตและสเปกการสาธิตบน `mob04` (#344, #335) การสร้างร้านโดยไม่แตะ psql พร้อมคู่มือ (#338) เครื่องมือผู้ดูแลแพลตฟอร์ม (#443) การสำรองข้อมูลที่ถูกพักโดยเจตนา (#288, #363) งานที่รอเจ้าของโครงงานตัดสิน (#476) และการย้ายร้านไปใช้ระบบใหม่ (#231) ต่อมาตั๋ว #476 ปิดเมื่อ 3 ตุลาคม พ.ศ. 2569 หลังเพิ่มการแทนที่เครื่องขายที่สูญหายจากระนาบแพลตฟอร์ม (PR #561) และพิสูจน์การกู้คืนบน `mob04` จริง ตั๋วที่เปิดอยู่ ณ 4 ตุลาคม พ.ศ. 2569 จึงเหลือ 10 ตั๋ว และยังเป็น 10 ตั๋วเดิม ณ 7 ตุลาคม พ.ศ. 2569

บทที่ 5
ผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

ตารางที่ 5.1 เทียบวัตถุประสงค์ทั้งเจ็ดข้อในบทที่ 1 กับผลที่ได้ หลักฐาน และสถานะ ณ วันที่ 4 ตุลาคม พ.ศ. 2569 (ตัวเลขสถิติยังเป็นค่าที่วัดเมื่อ 1 ตุลาคม พ.ศ. 2569) การเปลี่ยนแปลงระหว่างวันที่ 5–7 ตุลาคม พ.ศ. 2569 สรุปไว้ในหัวข้อสถานะล่าสุด 2026-10-07 การเปลี่ยนแปลงระหว่างวันที่ 8–12 ตุลาคม พ.ศ. 2569 สรุปไว้ในหัวข้อสถานะล่าสุด 2026-10-12

ตารางที่ 5.1 ผลการดำเนินงานเทียบกับวัตถุประสงค์

วัตถุประสงค์	ผลที่ได้	หลักฐาน	สถานะ
1. แอป Flutter offline-first ที่คงกติกาเดิม	พอร์ตกฎธุรกรรมครบหน้าจอ 14 ไฟล์ รายงานยอดขาย กำไร และสินค้าขายดี ชุดทดสอบ 820 กรณีผ่านทั้งหมด (1 ต.ค. 2569)	ตารางที่ 3.4 และ 4.3	เสร็จบนเว็บ สร้าง APK ได้แล้ว (#551, #560) แต่ยังไม่มีบันทึกการทดสอบบนอุปกรณ์ Android/iOS จริง
2. เซิร์ฟเวอร์ที่ PostgreSQL เป็นแหล่งความจริง	กฎธุรกรรมบังคับที่ฝั่งเซิร์ฟเวอร์ ชุดทดสอบประมาณ 492 กรณีหน่วยและ 655 กรณีแบบ e2e	ตารางที่ 4.6 และ 4.11	เสร็จ
3. แยกข้อมูลรายร้านด้วย RLS	อ่านข้ามร้านได้ 0 แถว ครบ 25 ตารางและ 12 จุดปลายทาง	ตารางที่ 4.12 ข้อ 4	เสร็จ
4. การเขียนที่ปลอดภัยเมื่อซ้ำหรือพร้อมกัน	ขายพร้อมกัน 200 คำขอบนสินค้า 50 ชิ้นได้ 50 บิลพอดี ธุรกรรมที่ยาวที่สุดลดจากราว 101–131 เหลือ 14–22 มิลลิวินาที	ตารางที่ 4.12 ข้อ 3 และ 15	เสร็จ
5. ทำงานออฟไลน์ (เฟส 2)	`outbox_ops`, `SyncService`, `POST /sync/push`, เลข RC/CN ฝั่งเครื่อง และ PIN ออฟไลน์ รวมโค้ดแล้ว แต่ละฝั่งทดสอบกับไฟล์ตัวอย่างคำขอชุดเดียวกัน 18 ไฟล์	ตารางที่ 4.5	โค้ดเสร็จ ยังไม่พิสูจน์แบบครบวงจรกับเซิร์ฟเวอร์จริง (#344)

วัตถุประสงค์	ผลที่ได้	หลักฐาน	สถานะ
6. CI/CD และการเฝ้าสังเกตระบบ	CI สองเวิร์กโฟลว์พร้อมสแกนความลับและช่องโหว่ การ deploy ผ่านการอนุมัติของผู้ตรวจที่กำหนดบนสภาพแวดล้อม `demo` การย้อนรุ่นอัตโนมัติ และแดชบอร์ด Grafana	ตารางที่ 4.7 และรูปที่ 4.14	เสร็จ ยกเว้นการสำรองข้อมูลออกนอก VM (#363)
7. บันทึกการตัดสินใจเป็น ADR	ADR 13 หมายเลข	หัวข้อ ADR ในบทที่ 3	เสร็จ

สิ่งที่ **ยังไม่เสร็จ** ได้แก่ การวัดโหลด k6 (#380) ซึ่งเป็นเกณฑ์ตรวจรับข้อเดียวที่ยังเปิด (วัดจริงครั้งแรกแล้วเมื่อ 5 ตุลาคม พ.ศ. 2569 แต่ยังไม่ได้รับผล) การสาธิตครบวงจร (#344) การสำรองข้อมูลออกนอก VM (#363, #288) และการย้ายร้านไปใช้ระบบใหม่ (cutover, #231) ร้านยังใช้บิลด์ Drift แบบเดิม ผลงานจึงเป็นระบบที่พัฒนากับร้านสาธิตและผ่านการทดสอบอัตโนมัติ ยังไม่ได้ใช้งานจริงในร้านและยังไม่มีการประเมินผลจากผู้ใช้อย่างเป็นทางการ

5.2 สถานะล่าสุด 2026-10-07

หัวข้อนี้สรุปการเปลี่ยนแปลงระหว่างวันที่ 5–7 ตุลาคม พ.ศ. 2569 ซึ่งเกิดหลังสถานะในตารางที่ 5.1 ทุกข้อตรวจกับประวัติ git ของสาขา `develop` และ `main` สถานะตั๋วบน GitHub และค่าที่อ่านบนเครื่อง `mob04` โดยตรงเมื่อ 7 ตุลาคม พ.ศ. 2569 ดังตารางที่ 5.2

ตารางที่ 5.2 การเปลี่ยนแปลงระหว่างวันที่ 5–7 ตุลาคม พ.ศ. 2569

เรื่อง	ผลที่ได้	หลักฐาน	สถานะ
การวัดโหลด k6 สามเครื่อง (#380)	วัดจริงครั้งแรกบน `mob04` ตามวิธีใน `03_ARCHITECTURE.md` หัวข้อ 8.1 การอ่านสินค้ามีค่า p95 16–33 มิลลิวินาที (เกณฑ์ไม่เกิน 200) การแย่งซื้อสินค้าชิ้นเดียว 200 คนมีค่า p95 ประมาณ 1.5–3 วินาที (เกณฑ์ไม่เกิน 500 มิลลิวินาที) แต่ข้อมูลถูกต้องทุกครั้ง	ความเห็นในตั๋ว #380 (5 ต.ค. 2569) และ `docs/handoff_log/session-2026-10-05-k6-capacity-run.md`	วัดแล้ว ยังไม่ได้รับการตัดสินจากเจ้าของโครงงาน
รหัสระบุข้อมูลแบบ UUIDv7 (#616)	รหัสของเอนทิตีเปลี่ยนจากข้อความเป็น UUIDv7 ทั้งสองฝั่ง (ยกเว้น `tenants`, `users` และ `platform_admins` ที่ใช้ UUID v4) ข้อมูลบน `mob04` ถูกสำรองแล้วล้าง ก่อนรัน migration `EntityIdsToUuid1788652804900`	PR #617, PR #628 (`65861ea`), `docs/handoff_log/session-2026-10-06-uuid-cutover-mob04.md`	deploy แล้ว 6 ต.ค. 2569
สาขา `develop` และกติกาการรวมโค้ด	PR งานทุกรายการเข้า `develop` และ `develop` เข้า `main` ด้วย merge commit เท่านั้น (ruleset `24564072`) ทั้งสองสาขาบังคับ PR และการตรวจสถานะสองรายการ CI รันเมื่อ push เข้า `develop` ด้วย แต่สร้างอิมเมจและ deploy เฉพาะจาก `main`	PR #637, PR #647, `gh api .../rulesets`	ใช้งานแล้ว
การเล่นซ้ำของ `POST /sync/push`	ลำดับต่อรายการคือเล่นซ้ำด้วย `Idempotency-Key` แล้วด้วยรหัสของไคลเอนต์ ก่อนตรวจรูปแบบข้อมูล จึงเล่นซ้ำได้แม้คีย์หมดอายุแล้ว มีการทดสอบ e2e ครบทั้งเจ็ดชนิดรายการ	`08_PHASE2_SPEC.md` หัวข้อ 8.3, PR #630, #638, #641	รวมแล้ว
การจัดการข้อผิดพลาดฝั่งแอป	`ApiException` ไม่ถึงหน้าจออีก และมีการทดสอบเฝ้าไว้ ความล้มเหลวแบบ 5xx, 429 และ `IN_FLIGHT` ไม่เข้าคิว แต่พักความพยายามไว้ด้วยรหัสและคีย์เดิม ครอบคลุมถึงการเขียนข้อมูลลูกค้าและการรับชำระเครดิต	PR #642, #644, #645, #646, `08_PHASE2_SPEC.md` หัวข้อ 5	รวมแล้ว

เรื่อง	ผลที่ได้	หลักฐาน	สถานะ
เครื่องที่ถูกปลดหรือไม่รู้จักขณะเข้าสู่ระบบ	ตอบ 401 `DEVICE_RETIRED` หรือ `DEVICE_TOKEN_INVALID` ก่อนตรวจรหัสผ่าน แอปล้างโทเค็นเครื่องและกลับเป็นโหมด Backoffice	PR #611, #613 (#609), PR #625, #629 (#612)	deploy แล้ว
การเปิดกะซ้ำด้วยรหัสเดิม	`POST /shifts/open` ที่ส่ง `id` เดิมแต่เงินเริ่มต้นต่างกันได้ 409 `CLIENT_ID_REUSED`	PR #610	deploy แล้ว
ข้อความภาษาไทย	เจ้าของโครงงานรับรองข้อความที่คณะผู้จัดทำร่างไว้ทั้งหมด และเติมข้อความข้อผิดพลาดที่เหลือ	PR #649, PR #651, PR #653	เสร็จ
การ deploy ล่าสุด	`/opt/pos/.current_sha` = `dd659e2` และ `/health/ready` ตอบ 200	Deploy run `37585778195` (งาน `deploy to demo` สำเร็จ) ส่วน run `37585675913` ของ SHA เดียวกันถูกข้าม	ยืนยันบน VM 7 ต.ค. 2569

ผลการวัดโหลดรอบหาขีดความสามารถ (15 ขั้น) ไม่พบการตอบ 429 ข้อผิดพลาด 5xx หรือการรีสตาร์ตของคอนเทนเนอร์ทั้ง 14 ตัว บิล 3,752 ใบมีเลขที่ใบเสร็จไม่ซ้ำกันครบ 3,752 เลข การอ่านและการใช้งานแบบผสมรับได้อย่างน้อย 72 คำขอต่อวินาที ซึ่งเป็นเพดานของเครื่องสร้างโหลด ไม่ใช่ของระบบ การแย่งซื้อสินค้าชิ้นเดียวในร้านเดียวกันคงค่า p95 ไม่เกิน 500 มิลลิวินาทีได้ถึงประมาณ 25 คน หลังจากนั้นเวลาเพิ่มเป็นเส้นตรงเพราะคำขอต่อคิวรอล็อกแถวสินค้า หน่วยความจำรวมของคอนเทนเนอร์สูงสุด 648 MiB จากหน่วยความจำของเครื่อง 5,920 MiB ทั้งนี้ร้านจริงมีเครื่องขายเพียงเครื่องเดียวต่อร้าน (ADR-0004) จึงไม่น่าเกิดคิวลักษณะนี้ในร้านเดียว แต่ผลยังไม่ผ่านเกณฑ์ในข้อนี้ และยังไม่มีการทำเครื่องหมายผ่านในเกณฑ์ตรวจรับข้อใด

สิ่งที่ **ยังไม่เสร็จ** ณ วันที่ 7 ตุลาคม พ.ศ. 2569 ได้แก่ การรับผลการวัด k6 (#380) การสาธิตครบวงจร (#344) ซึ่งต้องเริ่มใหม่ทั้งหมดเพราะร้านที่ใช้สาธิตถูกลบไปพร้อมการล้างข้อมูลเมื่อ 6 ตุลาคม การสำรองข้อมูลออกนอก VM (#363, #288) ซึ่งยังถูกพักไว้ สำเนาข้อมูลก่อนล้างมีเพียงไฟล์บน VM และสำเนาที่คัดลอกด้วยมือหนึ่งชุดบนเครื่องของเจ้าของโครงงาน การย้ายร้านไปใช้ระบบใหม่ (#231) และเครื่องมือผู้ดูแลแพลตฟอร์ม (#443) ตั๋วที่เปิดอยู่จึงยังเป็น 10 ตั๋วเดิม ส่วนเกณฑ์ตรวจรับของเฟส 1 ที่นับซ้ำเมื่อ 7 ตุลาคม พ.ศ. 2569 ยังผ่าน 16 จาก 17 ข้อ

5.3 สถานะล่าสุด 2026-10-12

หัวข้อนี้สรุปการเปลี่ยนแปลงระหว่างวันที่ 8–12 ตุลาคม พ.ศ. 2569 ซึ่งเกิดหลังสถานะในตารางที่ 5.2 ช่วงวันที่ 7–10 ตุลาคมมีข้อตกลงให้หยุดเพิ่มฟีเจอร์ใหม่เพื่อส่งงานในรายวิชา โดยเจ้าของโครงงานอนุญาตงานเฉพาะรายการเป็นกรณี ๆ ไป และยกเลิกข้อตกลงนี้เมื่อวันที่ 10 ตุลาคม จากนั้นงานกลับมาผ่านสาขา `develop` ตามปกติ ทุกข้อตรวจกับประวัติ git ของสาขา `develop` และ `main` บันทึกส่งมอบงาน (handoff) ของแต่ละวัน และค่าที่อ่านบนเครื่อง `mob04` เมื่อ 12 ตุลาคม พ.ศ. 2569 ดังตารางที่ 5.3

ตารางที่ 5.3 การเปลี่ยนแปลงระหว่างวันที่ 8–12 ตุลาคม พ.ศ. 2569

เรื่อง	ผลที่ได้	หลักฐาน	สถานะ
นำเข้าข้อมูลสำรองโดยเจ้าของร้าน และการดึงซัพพลายเออร์	เจ้าของร้านนำเข้าไฟล์สำรองเองจากหน้าตั้งค่า ส่วนสำรอง/กู้คืน ผ่าน POST /backup/import โหมดแทนที่ทั้งร้านต้องพิมพ์ชื่อร้านยืนยัน และทำในธุรกรรมเดียว เลขที่เอกสารล่าสุดถูกยกขึ้นตามไฟล์ที่นำเข้า แอปดึงรายการซัพพลายเออร์จากเซิร์ฟเวอร์มาแสดงแล้ว	PR #664, #665 (847e7ef), PR #668, #669 (6a38c87), session-2026-10-08-owner-import.md	deploy แล้ว ยังไม่มีบันทึกการนำเข้าจริงบน mob04
ติดดาวสินค้าโปรดบนหน้าขาย	ติดดาวสินค้าได้ทั้งในหมวด "ทั้งหมด" และในแต่ละหมวด สินค้าที่ติดดาวขึ้นก่อน มีตัวกรองสินค้าที่ติดดาว เก็บเฉพาะเครื่องนี้	PR #672 (1d70d1c)	deploy แล้ว
บัญชีรับเงิน QR และ QR พร้อมเพย์ที่หน้าขาย	ตั้งบัญชี QR ได้ไม่เกิน 5 บัญชี เฉพาะเจ้าของร้านแก้ไข หน้าขายแสดง QR พร้อมเพย์พร้อมยอดเงิน บิลบันทึกว่าเงินเข้าบัญชีใด และรายงานปิดร้านแยกรายบัญชี มีตาราง payment_accounts และ Drift เป็นรุ่นที่ 14	PR #676, #677 (ef07e27), migration 1788652805000	deploy แล้ว ยังไม่ได้ลองสแกนด้วยแอปธนาคารจริง
รูปสินค้าบนการ์ดหน้าขาย	หนึ่งรูปต่อสินค้า เจ้าของร้านเท่านั้นอัปโหลดหรือลบ แต่ละรูปมีที่อยู่เดาไม่ได้ (key เป็นแฮชของไฟล์) ไฟล์สำรองของเจ้าของร้านเปลี่ยนเป็น ZIP รอบแรก (ac9d4dd) ถูกย้อนกลับอัตโนมัติ เพราะไลบรารีรูปที่สร้างไว้ล่วงหน้าต้องการ CPU ระดับ x86-64-v2 แต่ mob04 เป็น qemu64 จึงสร้าง libvips และ sharp เองและเพิ่มการตรวจใน CI	PR #680, #681, #682, #683 (e77356f), migration 1788652805100, Drift รุ่นที่ 15	deploy แล้ว (run 38066101954) ยังไม่มีการอัปโหลดรูปจริงผ่านแอป

เรื่อง	ผลที่ได้	หลักฐาน	สถานะ
ขยายทั้งหน้าจอสำหรับผู้สูงอายุ	ขยายทั้งหน้าจอ (ไม่ใช่แค่ตัวอักษร) ได้ถึง 3 เท่าที่จอ 1920×1080 โดยโค้ดจำกัดไม่ให้หน้าจอตรรกะเล็กกว่า 360×320 หน่วยเพดานจึงอยู่ที่ราว 2.4 เท่าบนจอ 1366×768 และราว 1.08 เท่าบนโทรศัพท์ที่กว้าง 390 หน่วย ทดสอบทุกหน้าจอที่ 3 เท่าบน 1920×1080 และ 2 กับ 3 เท่าบน 1366×768	PR #686, #687 (bf02582), run 38100697523	deploy แล้ว ยังไม่ได้ลองพิมพ์ภาษาไทยในเบราว์เซอร์จริง
ค้นหาซัพพลายเออร์ตอนสร้างใบสั่งซื้อ	หน้าต่างสร้างใบสั่งซื้อค้นหาซัพพลายเออร์ได้และใช้ง่ายขึ้น	PR #690, #691 (2dabbcf), run 38103562796	deploy แล้ว
ปรับ UI/UX ทุกหน้าจอ และรับรองข้อความไทย	ปรับการแสดงผลทุกหน้าจอโดยไม่เปลี่ยนตรรกะ ผลตรวจอยู่ใน docs/research/ui-ux-audit-2026-10-11/ เจ้าของโครงงานรับรองข้อความไทยใหม่ทั้งหมด และรวมข้อความที่ซ้ำกันสามรายการ	PR #693, #694 (2d6a575, run 38187206166); PR #695, #696 (ec31121, run 38189738827)	deploy แล้ว
สแกนบาร์โค้ดด้วยกล้อง	ปุ่มกล้องในหน้าสินค้า หน้าขาย และหน้าต่างสร้างใบสั่งซื้อ ใช้ mobile_scanner 7.4.2 บนอุปกรณ์เคลื่อนที่ ใช้ BarcodeDetector ของเบราว์เซอร์เมื่อมี และใช้ตัวถอดรหัส zxing-wasm 3.1.3 ที่แนบมากับแอป (ไม่โหลดจาก CDN) เมื่อไม่มี จึงใช้ได้บน Chrome หรือ Edge ของเครื่อง Windows	PR #697 (35a79cc)	deploy แล้ว (ใน c3e33aa) ยังไม่ได้ลองสแกนด้วยกล้องจริง
ภาพถ่ายเป็นหลักฐานการขาย	หลังรับเงิน ถ่ายภาพสินค้าเพิ่มได้ไม่เกิน 5 ภาพต่อบิล (ไม่บังคับ ไม่ขวางการชำระเงิน) เวอร์ชันแรกใช้ได้เมื่อออนไลน์เท่านั้น เก็บบนเซิร์ฟเวอร์ ตารางใหม่ sale_photos (Postgres รวม 31 ตาราง) มี RLS และ pos_app เพิ่มได้อย่างเดียว การเปิดดูต้องยืนยันตัวตนและตอบ Cache-Control: no-store โวลุ่ม sale-photos แยกจาก product-images และ nginx ไม่เมานต์	PR #698 (4c992a2), migration SalePhotos1788652805200, docs/Backend_design/10_SALE_PHOTO_EVIDENCE.md	deploy แล้ว ยังไม่ได้ลองถ่ายภาพจริง

เรื่อง	ผลที่ได้	หลักฐาน	สถานะ
การ deploy ล่าสุด	main = c3e33aa (PR #699 แบบ merge commit) ตรวจผ่าน SSH ว่า /opt/pos/.current_sha ตรงกัน /health/ready ตอบ 200 คอนเทนเนอร์ทำงานครบ (api สามตัว healthy) และ migration SalePhotos1788652805200 ถูกใช้แล้ว	Deploy run 38260777086 (run คู่ 38260594779 ถูกข้ามตามปกติ)	ยืนยันบน VM 12 ต.ค. 2569
ผลการทดสอบ	Flutter ผ่าน 1,928 กรณี (dart analyze ไม่มีข้อเตือน) ฝั่งเซิร์ฟเวอร์ unit test ผ่าน 806 กรณี e2e ที่เกี่ยวข้องผ่าน 133 กรณีบนเครื่องผู้พัฒนา และชุด e2e ทั้งหมดผ่านใน CI	session-2026-10-12-photo-evidence-barcode.md	เสร็จ
สคริปต์สำรองข้อมูล	backup-db.sh ติดตั้งบน mob04 ด้วยมือและตรวจแล้ว: รันในสภาพแวดล้อมของ cron สำเร็จ ได้ไฟล์ pos_sale_photos_*.tar.gz พร้อมไฟล์ sha256 (ว่างเพราะยังไม่มีภาพ) ส่วน restore-db.sh ยังไม่กู้รูปสินค้าและภาพถ่ายการขาย	session-2026-10-12-photo-evidence-barcode.md	ติดตั้งแล้ว ยังไม่มีสำเนาออกนอก VM

กระบวนการสร้างฟีเจอร์ภาพถ่ายหลักฐานการขายเริ่มจากเอกสารออกแบบ 10_SALE_PHOTO_EVIDENCE.md แล้วตรวจด้วย /scrutinize ซึ่งสรุปว่าให้ดำเนินการโดยมีข้อแก้ไข (พบว่าในเวอร์ชันแรกดูภาพได้เฉพาะบนเครื่องเดียวกับที่ขาย และงานคิวออฟไลน์เป็นสัดส่วนราวร้อยละ 60 ของงานฝั่งไคลเอนต์) เจ้าของโครงงานจึงตัดสินใจว่า ถ่ายภาพหลังรับเงินและไม่บังคับเก็บบนเซิร์ฟเวอร์ ตัดโหมดออฟไลน์ออกจากเวอร์ชันแรก ดูภาพได้ที่หน้าคืนสินค้าและประวัติการขายในหน้ารายงาน ไม่มีใครลบภาพได้ ไม่ลบอัตโนมัติ ภาพไม่อยู่ในไฟล์สำรอง ZIP และการนำเข้าแบบแทนที่ไม่ลบภาพ จากนั้นเอกสารถูกแก้เป็นสัญญา API ฉบับเดียวที่ผูกมัดทั้งสองฝั่ง ฝั่งเซิร์ฟเวอร์และฝั่งไคลเอนต์สร้างขนานกันตามสัญญานั้น แต่ละฝั่งผ่าน /code-review (มาตรฐานและข้อกำหนด) แล้วรวมกับงานสแกนบาร์โค้ดบนสาขาเดียวเพื่อทดสอบรวม ก่อนออกเป็น PR #699

สิ่งที่ ยังไม่เสร็จ ณ วันที่ 12 ตุลาคม พ.ศ. 2569 ได้แก่ การทดลองใช้จริงของฟีเจอร์สแกนด้วยกล้องและภาพถ่ายหลักฐานบนโทรศัพท์ แท็บเล็ต และเครื่องของร้าน (รายการ X1 ในเอกสารออกแบบ) การสร้าง Android release ที่มี image_picker (ยังไม่ได้สร้าง) การรับรองข้อความไทยที่

คณะผู้จัดทำร่างไว้ (กล่องสแกน แผ่นและตัวดูภาพถ่าย ข้อความ SALE_PHOTO_LIMIT และข้อความขอสิทธิ์กล้องบน iOS) และการตัดสินใจของเจ้าของโครงงานเรื่องประกาศแจ้งลูกค้าตาม PDPA เกี่ยวกับการถ่ายภาพ ส่วนงานค้างเดิมยังเป็นเช่นเดิม ได้แก่ การสำรองข้อมูลออกนอก VM (#363, #288) ซึ่งยังถูกพักไว้ การรับผลการวัด k6 ซ้ำ (#380) และการสาธิตครบวงจรที่ต้องเริ่มใหม่ (#344)

5.4 ปัญหาและอุปสรรค

หัวข้อนี้เลือกเฉพาะปัญหาที่เกิดขึ้นจริงและมีบันทึกในเอกสารของโครงงาน แต่ละข้อแสดงเป็นลำดับ ปัญหา สาเหตุ วิธีแก้ และบทเรียน

5.4.1 พาธที่มีอักษรไทยทำให้ build_runner ล้มเหลว

ปัญหา ที่เก็บโค้ดอยู่ในโฟลเดอร์ชื่อภาษาไทยที่ซิงก์ผ่าน BeeStation คำสั่ง `build_runner` [41] สำหรับสร้างโค้ดของ Drift และคำสั่ง `flutter analyze` ล้มเหลวด้วยข้อความว่าเขียนไฟล์ `build.dart.aot` ไม่ได้ **สาเหตุ** คอมไพเลอร์ AOT ของ Dart แปลงพาธที่มีอักษรนอก ASCII เป็นเครื่องหมายคำถาม และ Windows junction แก้ไม่ได้เพราะ `build_runner` ใช้พาธจริง **วิธีแก้** คัดลอกที่เก็บโค้ดไปยังพาธ ASCII เพื่อรัน `build_runner` แล้วคอมมิตไฟล์ `*.g.dart` ที่สร้างใหม่ และใช้ `dart analyze` แทน `flutter analyze` เสมอ **บทเรียน** ข้อจำกัดของเครื่องมือเชิงพาธต้องบันทึกเป็นกฎของโครงงานตั้งแต่แรก และงาน `codegen-check` ใน CI ช่วยตรวจว่าไฟล์ที่สร้างตรงกับสคีมา

5.4.2 ไฟร์วอลล์ของคณะปิดกั้นการดึงอิมเมจจาก ghcr.io

ปัญหา การ deploy อัตโนมัติไปยัง `mob04` ถูกปิดกั้น เครื่องดึงอิมเมจจากรีจิสทรีภายนอกไม่ได้และเกิดข้อผิดพลาด `x509: certificate is not valid for any names` **สาเหตุ** ไฟร์วอลล์ FortiGate ของคณะตรวจและแทนที่ใบรับรองของการเชื่อมต่อ TLS ด้วยใบรับรองที่ไม่มีชื่อตรงกับโดเมนปลายทาง **วิธีแก้** แก้ได้เมื่อ 29–30 กันยายน พ.ศ. 2569 หลังทีมเครือข่ายเพิ่มชื่อทางเลือก (Subject Alternative Name: SAN) ในใบรับรอง หากอาการกลับมา ทางแก้ที่ถูกคือให้ทีมเครือข่ายยกเว้นรีจิสทรีเหล่านี้สำหรับ IP ของ VM ส่วนการคัดลอกอิมเมจด้วยมือเป็นเพียงทางแก้ฉุกเฉินในวันสาธิต ไม่ใช่ CD **บทเรียน** การออกแบบการ deploy ต้องรวมความเสี่ยงของเครือข่ายที่ทีมไม่ได้ควบคุม และกำหนดขอบเขตของทางสำรองไว้ล่วงหน้า

5.4.3 pool ฐานข้อมูลหมดเมื่อมีคำขอพร้อมกัน (#162)

ปัญหา เมื่อมีคำขอพร้อมกันเท่ากับขนาด pool ระบบหยุดตอบสนอง **สาเหตุ** guard แบบ global ที่ทำงานกับทุกคำขอและอ่านแผนของร้านตอนแคชว่าง ขอการเชื่อมต่อจาก pool เป็นครั้งที่สองภายในคำขอที่ถือการเชื่อมต่อแรกอยู่แล้ว ทุกคำขอจึงรอกันเอง **วิธีแก้** กำหนดเป็นกฎว่าไม่มีส่วนประกอบใดขอการเชื่อมต่อที่สองในคำขอเดียว และรวม `runTx` ที่ทำพร้อมกันเป็นตัวเดียว พร้อมสเปกสถาปัตยกรรมที่บังคับจุดเชื่อมนี้ **บทเรียน** ความผิดพลาดของทรัพยากรที่จำกัดไม่ปรากฏเมื่อทดสอบทีละคำขอ ต้องมีการทดสอบที่ใช้จำนวนคำขอพร้อมกันเท่าขนาด pool (เช่น `rate-limit-pool.e2e-spec.ts` และ `health-pool.e2e-spec.ts`)

5.4.4 การทิ้งรายการในคิวย้อนผลไม่ตรง (#473, #488)

ปัญหา เมื่อเจ้าของร้านทิ้งรายการขายหรือคืนที่ค้างในคิว สต็อกและยอดในเครื่องไม่ถูกย้อน และเมื่อแก้ให้ย้อนแล้ว ยอดที่ย้อนยังไม่ตรงในบางกรณี **สาเหตุ** การย้อนคำนวณจากข้อมูลปัจจุบัน แต่การเขียนครั้งแรกอาจถูกปัดค่าที่ศูนย์ เช่น ช่างมีเครดิตค้าง 50 บาทแต่ใบลดหนี้หักเครดิต 180 บาท ยอดจึงถูกปัดเป็น 0 และการย้อนคืนยอดเป็น 180 บาท ซึ่งสร้างหนี้ 130 บาทที่ไม่มีอยู่จริง **วิธีแก้** PR #483 ย้อนผลในธุรกรรมเดียวกับการทิ้ง และ PR #495 เก็บผลต่างที่ใช้จริงในตาราง `op_effects` (Drift schema v13) แล้วย้อนตามค่านั้น **บทเรียน** การย้อนผลที่ถูกต้องต้องบันทึกสิ่งที่เกิดขึ้นจริง ณ เวลาเขียน ไม่ใช่คำนวณกลับจากสถานะปัจจุบัน

5.4.5 เลขที่เอกสารซ้ำระหว่างการออกเลขออนไลน์กับออฟไลน์ (#489)

ปัญหา เครื่องอาจออกเลขใบเสร็จหรือใบลดหนี้แบบออฟไลน์ซ้ำกับเลขที่เซิร์ฟเวอร์เคยออกให้ **สาเหตุ** เลข RC หรือ CN ที่เซิร์ฟเวอร์ออกในคำตอบออนไลน์ไม่ถูกบันทึกลงตัวนับ `DocCounters` ในเครื่อง ตัวนับในเครื่องจึงไม่รู้เลขสูงสุดที่ใช้ไปแล้วจริง **วิธีแก้** บันทึกเลขที่เซิร์ฟเวอร์ออกผ่าน `DocNumberService.commitServerIssued` ทั้งในคำตอบออนไลน์ (PR #491, #492) และในการเล่นซ้ำของ `/sync/push` (PR #494) **บทเรียน** เมื่อมีผู้ออกเลขสองฝ่าย ทุกเลขที่ฝ่ายหนึ่งออกต้องไหลกลับไปยังอีกฝ่าย กรณีนี้ยังเผยปัญหาด้านกระบวนการในหัวข้อการควบคุมการรวมโค้ดด้านล่าง

5.4.6 การควบคุมการรวมโค้ดและหลักฐาน

ปัญหา พบสามกรณี กรณีแรก ครึ่งหนึ่งของการแก้ #489 (การบันทึกเลขเมื่อ `/sync/push` เล่นซ้ำ) ไม่เข้า `main` ทั้งที่ PR #491 และ #492 รวมแล้ว และคอมมิตแก้ตามการทบทวน

ของ PR #486 ก็หายในลักษณะเดียวกัน กรณีที่สอง ลิงก์ PR ที่ GitHub แสดงบนตั๋วงานให้ข้อสรุปผิดว่าบางตั๋วไม่มีงานส่งมอบ และตั๋ว #184 ถูกปิดและเปิดใหม่สามครั้ง ทั้งที่เกณฑ์ด้านการวัด latency และหน่วยความจำของตั๋วไม่เคยมีผลวัด กรณีที่สาม สองสาขาที่ไม่มี PR เก็บสำเนาเดียวของเอกสารวิจัย (424 บรรทัด) ขณะกำลังจะถูกลบพร้อมสาขาเก่า 44 สาขาเมื่อ 22 กันยายน พ.ศ. 2569 **สาเหตุ** คอมมิตถูก push หลังกดรวม PR แล้ว [42] GitHub ไม่ผูก PR ที่ไม่มีคำหลักปิดตั๋วหรือมีคำหลักเฉพาะในชื่อ PR (93 จาก 201 PR ที่รวมแล้ว ณ 22 กันยายน) และการตัดสินว่าสาขารวมแล้วจากชื่อสาขาเป็นการเดา **วิธีแก้** cherry-pick คอมมิตที่ตกหล่นไปยัง PR #494 กู้การแก้ของ PR #486 ด้วย PR #521 ตรวจงานส่งมอบด้วย `git log --all --grep` และตัวทดสอบเอง และรวมเอกสารวิจัยผ่าน PR #386 ก่อนลบ โดยตรวจทุกสาขาด้วย `git merge-base --is-ancestor` **บทเรียน** ก่อนรวมต้องตรวจว่า head SHA ของ PR ตรงกับที่ทบทวน สถานะของตั๋วไม่ใช่หลักฐาน (การนับเกณฑ์ตรวจรับในรายงานนี้จึงอ้างไฟล์ทดสอบ) และการลบที่ย้อนกลับไม่ได้ต้องพิสูจน์ก่อนว่ารวมแล้ว

5.4.7 การทดสอบที่ผูกกับเดือนปัจจุบัน (#525)

ปัญหา เมื่อขึ้นเดือนใหม่ `main` กลายเป็นสีแดง ซึ่งทำให้ไม่มีอิมเมจบน GHCR และทุกการรัน deploy ถูกข้าม **สาเหตุ** ไฟล์ `sync-push.e2e-spec.ts` ฝังเลขที่ใบเสร็จของเดือนกันยายนไว้ตายตัว แต่เซิร์ฟเวอร์ประทับวันที่ตามเวลาจริง เมื่อเดือนเปลี่ยน เซิร์ฟเวอร์จึงตั้งธงวันที่ผิดปกติตามสเปก **วิธีแก้** PR #525 คำนวณงวดของเลขเอกสารจากเวลาปัจจุบันในเขตเวลา `Asia/Bangkok` **บทเรียน** การทดสอบที่พึ่งเวลาต้องอ้างเวลาเดียวกับระบบที่ทดสอบ และสีแดงของ `main` ปิดกั้นการส่งมอบทั้งสาย ไม่ใช่แค่เรื่องของการทดสอบ

5.4.8 ค่า 0 เป็นเท็จในนิพจน์ของ GitHub Actions

ปัญหา เงื่อนไขข้ามการทดสอบเมื่อ push เป็นเอกสารล้วนไม่เคยทำงาน **สาเหตุ** ในนิพจน์ของ GitHub Actions [43] ตัวเลข `0` เป็นเท็จ นิพจน์รูปแบบ `cond && 0 || 1` จึงได้ `1` เสมอ ทำให้ขั้นตอน checkout ดึงประวัติเพียงคอมมิตเดียว ขั้นจำแนกประเภทของ push จึงมองไม่เห็นคอมมิตก่อนหน้า **วิธีแก้** ใช้สตริง `'0'` และ `'1'` แทนตัวเลข (PR #548 รวมเมื่อ 1 ตุลาคม พ.ศ. 2569) **บทเรียน** ชนิดข้อมูลในภาษานิพจน์ของเครื่องมือ CI ทำงานต่างจากภาษาโปรแกรมทั่วไป และการข้ามการทดสอบแบบเงียบต้องมีหลักฐานจากการรันที่สังเกตได้ว่าเงื่อนไขทำงานจริง

5.5 งานที่จะดำเนินการต่อไป

งานที่เหลือเรียงตามลำดับความสำคัญได้ดังนี้ โดยตั๋วที่อ้างถึงยังเปิดอยู่ ณ วันที่ 7 ตุลาคม พ.ศ. 2569 (ข้อ 12 เพิ่มเมื่อ 12 ตุลาคม พ.ศ. 2569)

1. **การสาธิตครบวงจรบน `mob04` (#344)** รันตามรายการตรวจ `docs/handoff_log/demo-344-checklist-2026-09-30.md` โดยต้องจัดการข้อติดขัดที่ระบุไว้ก่อน ได้แก่ รหัสผ่านชั่วคราวของเจ้าของร้านที่ต้องตั้งรหัสใหม่ภายใน 10 นาทีหลังเข้าสู่ระบบครั้งแรก ร้านใหม่ที่ยังไม่มีสินค้า การที่แอปยังส่ง `Idempotency-Key` เดิมซ้ำไม่ได้ และเกณฑ์ย่อยสามข้อที่พิสูจน์บน VM ไม่ได้ (ตั๋ว #476 ที่เคยเป็นข้อติดขัดปิดแล้วเมื่อ 3 ตุลาคม พ.ศ. 2569) การเดินครั้งแรกเมื่อ 5 ตุลาคม พ.ศ. 2569 ผ่านเพียงสองเกณฑ์ย่อยแรก และผู้ดูแลแพลตฟอร์มเข้าสู่ระบบ `platform-ui` จริงเป็นครั้งแรกในวันนั้น แต่ร้านที่ใช้สาธิตถูกลบไปพร้อมการล้างข้อมูลบน `mob04` เมื่อ 6 ตุลาคม การสาธิตจึงต้องเริ่มใหม่ทั้งหมด โดยสร้างร้านใหม่ผ่าน `platform-ui` และล้างข้อมูลในทุกไคลเอนต์ก่อน
2. **การรับผลการวัดโหลด k6 (#380)** การวัดจริงครั้งแรกทำเมื่อ 5 ตุลาคม พ.ศ. 2569 (หัวข้อสถานะล่าสุด 2026-10-07) งานที่เหลือคือให้เจ้าของโครงงานตัดสินผล โดยเฉพาะกรณีแย่งซื้อ 200 คนที่ไม่ผ่านเกณฑ์ p95 ข้อบกพร่องของเครื่องมือวัดสามข้อ (การแปลงหน่วยใน `measure-container-rss.sh` คีย์ `Idempotency-Key` ที่ชนกันข้ามเครื่อง และการคำนวณ burst ของสคริปต์เล่นซ้ำ) แก้แล้วใน PR #654 และ #655 บนสาขา `develop` เมื่อ 7 ตุลาคม พ.ศ. 2569 งานที่เหลือคือนำเข้า `main` ติดตั้งสคริปต์บน `mob04` (การ deploy อัตโนมัติไม่อัปเดตสคริปต์ใน `/opt/pos/scripts`) แล้ววัดซ้ำ
3. **การสำรองข้อมูลออกนอก VM (#363 และ #288)** ถูกพักไว้จนหลังการสาธิต เมื่อกลับมาทำต้องทดสอบเส้นทางเครือข่ายจาก `mob04` ไปยังร้านก่อนสร้างข้อมูลรับรอง เลือกโปรโตคอล (อุปกรณ์ BeeStation ไม่เปิด SSH/SFTP ที่ใช้ได้ จึงต้องเลือกระหว่างการเชื่อมผ่านโปรโตคอล Server Message Block (SMB) กับการซื้ออุปกรณ์จัดเก็บข้อมูลบนเครือข่าย (Network-Attached Storage: NAS) ที่รองรับ SSH File Transfer Protocol (SFTP)) ติดตั้ง `rclone` ส่งสำเนาจริงครั้งแรก และกู้คืนจากสำเนานอก VM ให้ได้ ระหว่างนี้ระบบจึงยังไม่มีสำเนาข้อมูลนอก VM

4. **การแสดงผลของกะที่ข้ามเที่ยงคืน** หัวข้อ "ประวัติรายการวันนี้" และวันที่บนหัวรายงานปิดร้านยังเขียนว่า "วันนี้" สำหรับกะที่เปิดก่อนเที่ยงคืน (PR #592) การแก้ต้องมีข้อความภาษาไทยใหม่ที่เจ้าของโครงงานรับรอง
5. **การย้ายร้านไปใช้ระบบใหม่ (cutover, #231)** ซึ่งเป็นตั๋วสุดท้ายของลำดับงานเริ่มต้นเฟส 2 ต้องมีแผนทดสอบก่อนสลับและแผนย้อนกลับ
6. **ข้อค้นพบที่เหลือจากการตรวจโค้ดทั้งระบบ** ได้แก่ ข้อค้นพบด้านมาตรฐานโค้ดที่ยังไม่ได้จัดลำดับ (ข้อบกพร่องระดับต่ำเรื่อง `POST /shifts/open` ที่ละเลย `startingCash` เมื่อส่ง `id` เดิม แก้แล้วด้วย PR #610 เมื่อ 5 ตุลาคม พ.ศ. 2569)
7. **ตั๋วงานหลัก #443** ต้องให้เจ้าของโครงงานตัดสินว่าเกณฑ์ "เฉพาะ `bootstrap:admin` เท่านั้นที่สร้างผู้ดูแลได้" จะปรับให้สอดคล้องกับการซิงก์ `PLATFORM_ADMINS` อย่างไร
8. **ฮาร์ดแวร์เฉพาะร้าน** เครื่องพิมพ์ใบเสร็จแบบความร้อน ลิ้นชักเก็บเงิน และการสแกนบาร์โค้ด ซึ่งต้องทำที่หน้าร้านจริง และงานเสริมความมั่นคงที่ยังเหลือ เช่น การให้เว็บไม่ต้องดึงฟอนต์สำรองจากภายนอก และ service worker ที่ยังไม่ precache ทรัพยากร (การสแกนด้วยกล้องมีแล้วตั้งแต่ 12 ตุลาคม พ.ศ. 2569 แต่ยังไม่ได้ทดลองกับกล้องจริง)
9. **การรับรองข้อความภาษาไทย** เจ้าของโครงงานรับรองข้อความที่คณะผู้จัดทำร่างไว้ทั้งหมดแล้วเมื่อ 7 ตุลาคม พ.ศ. 2569 (PR #649 และ PR #653 สำหรับข้อความสิบรายการที่ PR #651 เพิ่ม) รวมถึงข้อความสามชุดจาก PR #469 และ #470 งานที่เหลือคือข้อความใหม่ที่จะเกิดขึ้น เช่น ข้อความของข้อ 4 ซึ่งต้องผ่านการรับรองแบบเดียวกัน
10. **การทดสอบบน Android และ iOS** CI ยังสร้างเฉพาะบิลด์เว็บ ส่วน Android มีเวิร์กโฟลว์ `android-apk.yml` ที่สั่งรันด้วยมือ และหลังติดตั้ง CA บน `mob04` (PR #559) สร้าง APK ได้แล้วสองรุ่น หลังการเปลี่ยนรหัสเป็น UUID มีการสร้าง APK จาก `main` ที่ `65861ea` แล้ว (run `37416623383`) งานที่เหลือคือติดตั้งบนอุปกรณ์จริงเข้าสู่ระบบ และทดสอบการขาย รวมถึงทดลองกู้คืนข้อมูลสำรองบนอุปกรณ์จริง (#556) และเก็บสำรองกุญแจลงนาม APK ไว้นอกเครื่องพัฒนา สำหรับ iOS ยังไม่มีงานใด
11. **การเฝ้าสังเกตระบบส่วนขยาย (PR #596 และ #597)** PR #596 (การแจ้งเตือนเมื่อระบบล่ม) และ PR #597 (ตัวชี้วัดและแผงควบคุม) รวมแล้วเมื่อ 4 ตุลาคม พ.ศ. 2569 และ PR #597 deploy ขึ้น `mob04` แล้ว การแจ้งเตือนเมื่อระบบล่มติดตั้งบน `mob04` ด้วยมือแล้วในวันเดียวกัน งานที่เหลือคือทดสอบกรณีระบบไม่พร้อมเพื่อยืนยันว่าบริการแจ้งเตือนจริง และติดตั้งซ้ำทุกครั้งที่แก้สคริปต์ เพราะการ deploy อัตโนมัติไม่อัปเดตสคริปต์บนเครื่อง ส่วนการนำที่เก็บบันทึกและตัวส่งออกขึ้น `mob04` ต้องประเมินงบ

หน่วยความจำของ VM ก่อน เพราะ ณ วันที่ 4 ตุลาคม พ.ศ. 2569 เพดานหน่วยความจำของคอนเทนเนอร์ที่ตั้งไว้รวม 4,256 MB จากหน่วยความจำของเครื่อง 5,920 MB และเครื่องไม่มี swap แม้การใช้งานจริงรวมเพียงประมาณ 460 MB ส่วนขยายนี้จะเพิ่มเพดานอีก 1,152 MB ข้อมูลประกอบการประเมินคือผลการวัด k6 เมื่อ 5 ตุลาคม พ.ศ. 2569 ซึ่งหน่วยความจำรวมของคอนเทนเนอร์ภายใต้โหลดสูงสุด 648 MiB

12. การทดลองใช้จริงของฟีเจอร์ใหม่วันที่ 12 ตุลาคม พ.ศ. 2569 (รายการ X1) ได้แก่ สแกนบาร์โค้ดด้วยกล้อง และถ่ายภาพหลักฐานการขายบนโทรศัพท์ แท็บเล็ต และเครื่องของร้านกับ `mob04` สร้าง Android release ที่มี `image_picker` รับรองข้อความไทยที่ร่างไว้ ตัดสินเรื่องประกาศแจ้งลูกค้าตาม PDPA และทำให้ `restore-db.sh` กู้รูปสินค้ากับภาพถ่ายการขายได้

นอกจากนี้ต้องตรวจบันทึกการสำรองข้อมูลรอบ 03:00 บน `mob04` เป็นระยะ และปรับไฟล์ฐานข้อมูลบนเว็บทุกครั้งที่ยกระดับ `drift` หรือ `sqlite3`

บรรณานุกรม

[1] "01 — Product & Pitch," ไฟล์ docs/study/01_pitch.md ของที่เก็บโค้ดโครงงาน srisurart-pos-flutter, 2026.

[2] Mozilla Developer Network, "Window: localStorage property." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage. Accessed: Oct. 1, 2026.

[3] "02 — System Architecture Overview," ไฟล์ docs/study/02_architecture.md ของที่เก็บโค้ดโครงงาน srisurart-pos-flutter, 2026.

[4] Google, "Flutter documentation." [Online]. Available: https://docs.flutter.dev/. Accessed: Oct. 1, 2026.

[5] S. Binder, "Drift documentation." [Online]. Available: https://drift.simonbinder.eu/. Accessed: Oct. 1, 2026.

[6] SQLite Consortium, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html. Accessed: Oct. 1, 2026.

[7] "03 — Architecture options and phase-1 plan," ไฟล์ docs/Backend_design/03_ARCHITECTURE.md, 2026.

[8] "ADR-0004: Device roles," ไฟล์ docs/Backend_design/adr/0004-device-roles.md, 2026.

[9] "ADR-0010: Client write-through cache," ไฟล์ docs/Backend_design/adr/0010-client-write-through-cache.md, 2026.

[10] The PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html. Accessed: Oct. 1, 2026.

[11] "ADR-0012: Replace PostgreSQL with CouchDB (Rejected)," ไฟล์ docs/Backend_design/adr/0012-couchdb-replaces-postgres.md, 2026.

[12] "08 — Phase 2 spec: offline shell + production ในมหาวิทยาลัย," ไฟล์ docs/Backend_design/08_PHASE2_SPEC.md, 2026.

[13] "ADR-0013: CI/CD toolchain," ไฟล์ docs/Backend_design/adr/0013-cicd-toolchain.md, 2026.

[14] "ADR-0001: Tenant provisioning," ไฟล์ docs/Backend_design/adr/0001-tenant-provisioning.md, 2026.

[15] "09 — Phase 2: lane split and ticket list," ไฟล์ docs/Backend_design/09_PHASE2_LANES.md, 2026.

[16] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!), 2019. [Online]. Available: https://www.inkandswitch.com/local-first/. Accessed: Oct. 1, 2026.

[17] W. Vogels, "Eventually consistent," Commun. ACM, vol. 52, no. 1, pp. 40-44, Jan. 2009.

[18] F. Angelov, "bloc / flutter_bloc documentation." [Online]. Available: https://bloclibrary.dev/. Accessed: Oct. 1, 2026.

[19] Flutter Team, "go_router package." [Online]. Available: https://pub.dev/packages/go_router. Accessed: Oct. 1, 2026.

[20] PostgreSQL Global Development Group, "PostgreSQL documentation: Concurrency control, explicit locking, deadlocks." [Online]. Available: https://www.postgresql.org/docs/current/mvcc.html. Accessed: Oct. 1, 2026.

[21] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP Semantics," RFC 9110, IETF, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110. Accessed: Oct. 1, 2026.

[22] J. Jena and S. Dalal, "The Idempotency-Key HTTP Header Field," IETF Internet-Draft, draft-ietf-httpapi-idempotency-key-header. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/. Accessed: Oct. 1, 2026.

[23] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519. Accessed: Oct. 1, 2026.

[24] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, IETF, Sep. 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9106. Accessed: Oct. 1, 2026.

[25] NestJS, "NestJS documentation." [Online]. Available: https://docs.nestjs.com/. Accessed: Oct. 1, 2026.

[26] Redis Ltd., "Redis documentation." [Online]. Available: https://redis.io/docs/. Accessed: Oct. 1, 2026.

[27] Taskforce.sh, "BullMQ documentation." [Online]. Available: https://docs.bullmq.io/. Accessed: Oct. 1, 2026.

[28] F5 NGINX, "NGINX documentation." [Online]. Available: https://nginx.org/en/docs/. Accessed: Oct. 1, 2026.

[29] Docker Inc., "Docker documentation." [Online]. Available: https://docs.docker.com/. Accessed: Oct. 1, 2026.

[30] Docker Inc., "Docker Compose overview." [Online]. Available: https://docs.docker.com/compose/. Accessed: Oct. 1, 2026.

[31] GitHub, "GitHub Actions documentation." [Online]. Available: https://docs.github.com/en/actions. Accessed: Oct. 1, 2026.

[32] GitHub, "Working with the Container registry." [Online]. Available: https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry. Accessed: Oct. 1, 2026.

[33] Aqua Security, "Trivy documentation." [Online]. Available: https://trivy.dev/. Accessed: Oct. 1, 2026.

[34] Gitleaks, "gitleaks: detect secrets in git repositories." [Online]. Available: https://github.com/gitleaks/gitleaks. Accessed: Oct. 1, 2026.

[35] Red Hat, "Ansible documentation." [Online]. Available: https://docs.ansible.com/. Accessed: Oct. 1, 2026.

[36] Prometheus Authors, "Prometheus documentation." [Online]. Available: https://prometheus.io/docs/. Accessed: Oct. 1, 2026.

[37] Grafana Labs, "Grafana documentation." [Online]. Available: https://grafana.com/docs/. Accessed: Oct. 1, 2026.

[38] M. Nygard, "Documenting architecture decisions," Cognitect Blog, Nov. 2011. [Online]. Available: https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Accessed: Oct. 1, 2026.

[39] Redis Ltd., "Redis documentation: Key eviction," [Online]. Available: https://redis.io/docs/latest/develop/reference/eviction/. Accessed: Oct. 1, 2026.

[40] Mozilla, "Progressive web apps," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps. Accessed: Oct. 1, 2026.

[41] Dart Team, "build_runner," pub.dev. [Online]. Available: https://pub.dev/packages/build_runner. Accessed: Oct. 1, 2026.

[42] GitHub, "About pull request merges," [Online]. Available: https://docs.github.com/en/pull-requests. Accessed: Oct. 1, 2026.

[43] GitHub, "Expressions — GitHub Actions," [Online]. Available: https://docs.github.com/en/actions/learn-github-actions/expressions. Accessed: Oct. 1, 2026.